



UNIVERSIDAD
DE GRANADA

Facultad de Ciencias
E.T.S. Ingenierías Informática y de Telecomunicación

DOBLE GRADO EN INGENIERÍA INFORMÁTICA Y MATEMÁTICAS

TRABAJO DE FIN DE GRADO

Análisis Topológico de Datos con especial atención a Mapper y a los datos de alta dimensionalidad

Presentado por:
Jaime Corzo Galdó

Curso académico 2025-2026



Análisis Topológico de Datos con especial atención a Mapper y a los datos de alta dimensionalidad

Jaime Corzo Galdó

Jaime Corzo Galdó *Análisis Topológico de Datos con especial atención a Mapper y a los datos de alta dimensionalidad.*

Trabajo de fin de Grado. Curso académico 2025-2026.

**Responsable de
tutorización**

David Ruiz Aguilar
Análisis Matemático

Daniel Molina Cabrera
*Departamento de Ciencias de la
Computación e Inteligencia Artificial*

Doble Grado en Ingeniería
Informática y Matemáticas

Facultad de Ciencias
E.T.S. Ingenierías
Informática y de
Telecomunicación

Universidad de Granada

DECLARACIÓN DE ORIGINALIDAD

D./Dña. Jaime Corzo Galdó

Declaro explícitamente que el trabajo presentado como Trabajo de Fin de Grado (TFG), correspondiente al curso académico 2025-2026, es original, entendido esto en el sentido de que no he utilizado para la elaboración del trabajo fuentes sin citarlas debidamente.

En Granada a 14 de julio de 2026

Fdo: Jaime Corzo Galdó

A mis padres

Agradecimientos

En la portada de este trabajo no aparecería mi nombre de no ser por todos los que me habéis acompañado y apoyado durante esta etapa y que, de una manera u otra, habéis contribuido a la realización del mismo. Por ello, no puedo empezar sin dedicaros antes unas palabras de agradecimiento.

En primer lugar, a mis tutores, David Ruiz Aguilar y Daniel Molina Cabrera, por sus valiosos consejos y por aceptar el reto de acompañarme en un tema que escapaba a su área de especialización. Algo que no dudaron en hacer desde que contacté con ellos hace más de un año con apenas un esbozo de idea y que, poco a poco, fue tomando forma hasta convertirse en el documento que hoy tenéis entre manos.

En segundo lugar, a todos mis profesores de una etapa universitaria que recordaré con mucho cariño. Gracias, especialmente, a aquellos que, además de transmitirme conocimientos, han sabido contagiarme su pasión y estimular mi curiosidad y mis ganas de seguir aprendiendo. También quiero aprovechar para agradecer la labor de todo el personal de los dos centros en los que he tenido la oportunidad de estudiar, ya que, aunque haya habido algún que otro momento de estrés, sin su trabajo, este doble grado no sería posible tal y como lo conocemos.

En tercer lugar, a todos mis compañeros de clase, porque tengo claro que no estaría a un paso de poder presumir de ser info-matemático de no ser por vosotros. Me habéis enseñado que lo importante no es solo llegar a la meta, sino hacerlo bien acompañado. Y, aunque quizá no me corresponda decirlo, tengo la suerte de haber formado parte de la mejor promoción que podría haber imaginado. Más aún cuando me llevo amigos que ojalá lo sean para toda la vida.

También quería hacer una mención a mi Erasmus en Salerno y a todas las personas que formaron parte de esa experiencia. Más allá del aprendizaje académico (que también, mamá), me llevo recuerdos y amistades que conservaré siempre. Todo ello me ha acompañado también durante la realización de este trabajo.

Por último, y aún más importante, a mis amigos, los de siempre y los que han ido apareciendo por el camino, porque, especialmente en los momentos difíciles, cuando piensas que no vas a ser capaz, cuando no tienes ganas de seguir, me hacen darme cuenta de lo afortunado que soy de tenerlos. Y, por supuesto, a mi familia, y en especial a mis padres, porque sin su ayuda, su paciencia y su apoyo incondicional durante todo este tiempo no habría sido capaz de llegar hasta aquí. Gracias por estar siempre ahí.

Como sé que estáis deseosos de empezar a leer este apasionante trabajo, que estoy seguro de que os dejará sin palabras, no os entretengo más.

GRACIAS A TODOS.

Summary

This Final Degree Project studies the application of Topological Data Analysis (TDA) to high-dimensional financial data from both a mathematical and a computational perspective. The main objective is to understand the mathematical foundations of TDA and to apply them to an experimental portfolio-construction problem, comparing the resulting strategies with a simple classical benchmark. The work therefore combines three complementary dimensions: theoretical study, software development and empirical evaluation.

The motivation comes from the difficulty of analysing datasets with many variables and complex relationships. In high-dimensional settings, direct visualisation is impossible and distances may become less informative. Financial markets provide a natural example, since each asset can be represented by a time series and the relationships between assets evolve over time. Traditional measures such as return, volatility and correlation remain essential, but they may not fully describe the global structure of the data. TDA offers a complementary viewpoint by studying the shape, connectivity and multiscale structure of point clouds.

The mathematical part of the project introduces the main concepts needed to understand this approach. It covers basic topological notions, simplicial complexes, simplicial homology and the construction of topological objects from point clouds. Particular attention is given to filtrations and persistent homology, which make it possible to track how topological features appear and disappear as the scale changes. Persistence diagrams, barcodes and persistence landscapes are also introduced because they provide practical representations of persistent information and support the market-regime signal later used in the financial application.

The computational part translates these ideas into a reproducible Python workflow. A dedicated library was developed to prepare financial data, construct Mapper representations, compute persistent-homology features, generate portfolio weights, run causal back-tests and save experimental results. The project was organised as an installable Python package, published on PyPI under the name `tda-finance-mapper`, and documented with Sphinx. A user manual and API documentation were also prepared so that the main functionality can be used without reviewing the complete implementation.

Two financial universes are considered. The first is the Kenneth French 49 Industry Portfolios dataset, referred to as KF49, which provides a stable set of aggregated sector portfolios. The second is a universe of individual S&P 500 stocks obtained through CRSP/WRDS. These datasets allow the method to be evaluated in two different settings: a smaller and more aggregated sector universe, and a larger universe of individual stocks with a more complex structure.

At each rebalancing date, every asset is represented by its returns over a recent historical window. This produces a high-dimensional point cloud in which each point corresponds to one asset. Returns are standardised by asset before constructing Mapper. The filter or *lens* combines PCA and one-dimensional UMAP, using a correlation-based metric. Mapper then applies a cover to the filtered space, performs local clustering within each cover element and builds a graph in which nodes represent local groups of assets and edges appear when nodes share at least one asset because of the overlap of the cover.

Since Mapper itself does not define portfolio weights, an additional rule is required to

transform the graph into an allocation. Several design choices were studied during the development of the model. DBSCAN and hierarchical agglomerative clustering, referred to as HACA in the project, were compared as local clustering methods. Two graph-to-portfolio rules were also analysed: a macrocluster-based method and a node-overlap method. In addition, two defensive responses to the persistent-homology signal were compared, and a limited sensitivity analysis was carried out for the main cover parameters. The final configuration selected HACA, node overlap, temporary replacement by an equal-weight portfolio when the persistent-homology signal is anomalous, and a dataset-specific cover configuration.

Persistent homology is used as an auxiliary market-regime signal rather than as an independent portfolio constructor. At each rebalancing date, a correlation-based distance matrix is built from a recent return window. Persistence diagrams are computed and the L_2 -norm of the H_1 persistence landscape is used as a scalar summary of the current topological structure. This value is compared with its recent history. When it exceeds the 95th percentile, the regime is treated as potentially anomalous and the Mapper portfolio is temporarily replaced by an equal-weight portfolio. Otherwise, the Mapper allocation is maintained.

The final comparison considers three strategies: Mapper, Mapper with persistent-homology regime control, and equal-weight. The equal-weight portfolio is used as the main benchmark because it is simple, transparent and widely recognised as a demanding reference in asset-allocation problems. All strategies are evaluated under the same causal protocol. At each rebalancing date, only information available up to that moment is used. Mapper uses a 60-month historical window, the persistent-homology signal uses a shorter 20-month window, portfolios are rebalanced every three months and transaction costs of 5 basis points proportional to turnover are included. Performance is evaluated over five temporal blocks: 1975–1984, 1985–1994, 1995–2004, 2005–2014 and 2015–2025.

The main evaluation measures are annualised return, annualised volatility, Sharpe ratio, Sortino ratio, maximum drawdown, final net asset value and turnover. The experimental design is intended to reduce look-ahead bias and make the comparison reproducible. The same general procedure is applied to all strategies, results are exported to structured files, and the configuration is kept fixed within each dataset during the final evaluation.

The results depend clearly on the financial universe considered. In KF49, the two topological strategies slightly outperform the equal-weight benchmark in several metrics. Mapper achieves an average annualised return of 13.45%, an annualised volatility of 16.01%, a Sharpe ratio of 0.609, a Sortino ratio of 0.922 and an average maximum drawdown of -28.99%. Mapper with persistent homology reaches an annualised return of 13.50%, a Sharpe ratio of 0.612 and a maximum drawdown of -28.98%. The equal-weight portfolio obtains an annualised return of 13.04%, a Sharpe ratio of 0.575 and a maximum drawdown of -30.36%. The differences are modest, but they indicate that the topological representation can be competitive in an aggregated sector universe.

The outcome is different in the S&P 500 universe. The equal-weight portfolio achieves the highest annualised return and Sharpe ratio, with values of 11.19% and 0.574, respectively. Mapper reaches an annualised return of 8.76% and a Sharpe ratio of 0.495, while Mapper with persistent homology obtains 8.66% and 0.475. However, Mapper shows a lower average maximum drawdown, at -17.34%, compared with -23.70% for equal-weight. This suggests that Mapper behaves more defensively in this universe, even though it does not provide the highest return.

The persistent-homology regime signal also behaves differently across datasets. In KF49 it provides a small improvement over Mapper, while in the S&P 500 it slightly worsens the

main performance measures. This shows that persistent homology should not be treated as an automatic enhancement. Its usefulness depends on the dataset, the topological feature used and the way an anomaly signal is translated into a portfolio decision.

Overall, the project shows that TDA can be integrated into a causal and reproducible financial workflow. Its main contribution lies in connecting mathematical foundations, software implementation and experimental portfolio evaluation within a single project. The work provides an installable Python library, a reproducible backtesting framework and a direct comparison between topological strategies and a classical benchmark. The results do not support a general claim that topological methods always outperform equal weighting, but they show that Mapper can provide competitive results in some settings and may help reduce drawdown in others.

Several limitations remain. Only two financial universes are considered, so the conclusions cannot be generalised to all markets, time periods or asset classes. KF49 consists of academic portfolios rather than directly tradable assets. The final configuration was chosen after a limited sensitivity analysis, which reduces the scope for extensive tuning but does not rule out the existence of better alternatives. The backtest includes transaction costs but does not model every aspect of real trading, such as liquidity, bid–ask spreads, market impact or taxes. Finally, the persistent-homology regime signal requires further study because its behaviour is not consistent across the two datasets.

Future work could extend the analysis to additional markets and asset universes, investigate alternative ways of transforming Mapper graphs into portfolio weights, study the persistent-homology signal in greater depth and compare the proposed approach with additional classical portfolio methods. The software could also be extended with more examples, broader automated testing and simpler experiment-configuration tools.

Índice general

Agradecimientos	V
Summary	VII
Índice de figuras	XV
Índice de tablas	XVII
Introducción	XIX

I. Fundamentos matemáticos del análisis topológico de datos	1
1. Motivación matemática	3
1.1. El reto de los datos masivos	3
1.2. Limitaciones de la geometría clásica	3
1.3. La forma de los datos: el Análisis Topológico de Datos	4
2. Herramientas topológicas necesarias	5
2.1. Conectividad por caminos e información cualitativa	5
2.2. Homotopía y deformación continua	6
2.3. Equivalencia homotópica y espacios contráctiles	7
2.4. De la homotopía a la homología	8
2.5. Comentario sobre computabilidad	8
3. Complejos simpliciales y homología simplicial	11
3.1. Símplices geométricos	11
3.2. Complejos simpliciales	12
3.3. Orientación de símlices	14
3.4. Grupos de cadenas	14
3.5. El operador borde	15
3.6. La identidad $\partial_{n-1} \circ \partial_n = 0$	16
3.7. Ciclos, bordes y grupos de homología	17
3.8. Los números de Betti	18
3.9. Característica de Euler	19
3.10. Ejemplos básicos	19
3.10.1. La circunferencia	20
3.10.2. El disco	20
3.10.3. La esfera m -dimensional	21
3.10.4. El toro	22
4. De nubes de puntos a objetos topológicos	23
4.1. El problema de reconstrucción topológica	23

4.2. Sistemas finitos de conjuntos	24
4.3. El nervio de una familia de conjuntos	24
4.4. El teorema del nervio	25
4.5. El complejo de Čech	27
4.6. El complejo de Vietoris–Rips	28
4.7. Comparación entre los complejos de Čech y de Vietoris–Rips	28
4.8. Condiciones de muestreo y reconstrucción	29
4.9. Escala y filtraciones	30
5. Filtraciones y homología persistente	33
5.1. Filtraciones de complejos simpliciales	33
5.2. Aplicaciones inducidas y homología persistente	34
5.3. Nacimiento, muerte y persistencia	35
5.4. Un ejemplo elemental	36
5.5. Códigos de barras	37
5.6. Diagramas de persistencia	38
5.7. Paisajes de persistencia	39
5.8. Estabilidad e interpretación	41
II. Aplicación computacional a finanzas	43
6. Motivación y objetivos computacionales	45
6.1. Objetivo general	45
6.2. Objetivos específicos	45
6.2.1. Disponer de una herramienta modular y reutilizable para TDA aplica- do a finanzas	46
6.2.2. Representar las relaciones entre activos mediante herramientas topoló- gicas	46
6.2.3. Obtener decisiones de cartera a partir de la información topológica	46
6.2.4. Comprobar experimentalmente el funcionamiento del método sobre datos financieros reales	46
6.2.5. Evaluar la utilidad de la información topológica frente a una estrategia de referencia	46
6.3. Requisitos del sistema	47
6.3.1. Requisitos funcionales	47
6.3.2. Requisitos no funcionales	47
6.4. Alcance del trabajo	48
7. Planificación y presupuesto	49
7.1. Descomposición de tareas	49
7.2. Estimación económica	49
7.3. Planificación temporal	51
8. Implementación de la librería	53
8.1. Objetivos de la implementación	53
8.2. Estructura del repositorio	53

8.3. Módulos principales	54
8.3.1. Preprocesamiento de datos	54
8.3.2. Módulo topológico	54
8.3.3. Módulo de cartera	55
8.3.4. Módulo de experimentos	55
8.4. Librerías y herramientas utilizadas	55
8.5. Empaquetado e instalación	57
8.6. Documentación	58
8.7. Calidad del código y reproducibilidad	58
9. Diseño experimental	61
9.1. Objetivo de la evaluación	61
9.2. Conjuntos de datos	61
9.3. Preprocesamiento y representación de los activos	61
9.4. Protocolo experimental común	62
9.4.1. Esquema común de construcción de carteras Mapper	63
9.5. Métricas de evaluación	63
9.6. Modelos comparados	64
9.6.1. Mapper	64
9.6.2. Mapper con homología persistente	65
9.6.3. Cartera equiponderada	65
9.7. Control temporal y reproducibilidad	66
10. Resultados experimentales	67
10.1. Evolución del modelo	67
10.1.1. Clustering local: DBSCAN frente a HACA	67
10.1.2. Asignación de pesos: macrocluster frente a solapamiento de nodos	68
10.1.3. Control de régimen: liquidez frente a cartera equiponderada	70
10.1.4. Sensibilidad respecto al recubrimiento	70
10.2. Configuración final de los modelos	73
10.3. Resultados sobre las 49 carteras industriales	73
10.4. Resultados sobre el S&P 500	74
10.5. Comparación global	75
10.6. Tiempos de ejecución	77
10.7. Análisis por periodos	78
10.8. Discusión de los resultados	79
10.9. Limitaciones	80
III. Cierre del trabajo	81
11. Conclusiones y vías futuras	83
11.1. Conclusiones generales	83
11.2. Cumplimiento de los objetivos	83
11.3. Aprendizaje académico y profesional	84
11.4. Limitaciones	85
11.5. Vías futuras	86

A. Resultados auxiliares de la evolución del proyecto	87
A.1. Comparación completa entre DBSCAN y HACA	87
A.2. Comparación completa entre macrocluster y solapamiento de nodos	88
A.3. Comparación completa entre liquidez y cartera equiponderada	88
A.4. Resultados del estudio de sensibilidad del recubrimiento	89
B. Resultados completos de la comparación final	91
B.1. Resultados completos por periodo en KF49	91
B.2. Resultados completos por periodo en S&P 500	91
C. Manual de usuario	93
C.1. Formas de uso	93
C.2. Uso del paquete con datos propios	93
C.2.1. Crear una carpeta de trabajo	93
C.2.2. Crear un entorno virtual	93
C.2.3. Instalar el paquete	94
C.3. Formato de los datos de entrada	94
C.3.1. Caso 1: el archivo contiene precios	94
C.3.2. Caso 2: el archivo contiene retornos	94
C.4. Script de ejemplo	95
C.5. Ejecución del experimento	99
C.6. Interpretación de los resultados	99
C.7. Representación gráfica de las curvas NAV	100
C.8. Uso del repositorio	100
C.9. Reproducción de los experimentos de la memoria	101
C.10. Parámetros principales	101
C.11. Descripción de las estrategias	103
C.11.1. Mapper	103
C.11.2. Mapper con control PH	103
C.11.3. Cartera equiponderada	103
C.12. Problemas frecuentes	103
C.12.1. ModuleNotFoundError: No module named 'tda_finance'	103
C.12.2. Serie temporal demasiado corta	103
C.12.3. Ruta incorrecta al archivo de datos	103
C.12.4. Resultados inesperados	104
C.13. Advertencia sobre uso financiero	104
Glosario	105
Bibliografía	111

Índice de figuras

1.	Evolución del número de documentos relacionados con TDA y finanzas obtenidos en Scopus mediante la búsqueda TITLE-ABS-KEY("topological data analysis" AND finance). Fuente: elaboración propia a partir de Scopus, consulta realizada el 7 de julio de 2026.	xx
2.1.	Comparación entre una circunferencia y un disco. Ambos espacios son conexos por caminos, pero presentan distinta estructura global. Fuente: elaboración propia.	6
2.2.	Esquema de una retracción por deformación de un anillo sobre una circunferencia contenida en él. Fuente: elaboración propia.	7
3.1.	Ejemplos de símlices geométricos en dimensiones 0, 1, 2 y 3. Fuente: elaboración propia.	12
3.2.	A la izquierda, un complejo simplicial válido, ya que los dos triángulos comparten una cara común. A la derecha, una configuración que no define un complejo simplicial, porque la intersección de los triángulos no es una cara común. Fuente: elaboración propia.	13
4.1.	Ejemplo de nervio asociado a una familia de conjuntos. El complejo simplicial de la derecha registra qué subfamilias tienen intersección común no vacía. Fuente: elaboración propia.	25
4.2.	Comparación entre los complejos de Čech y Vietoris–Rips para tres puntos equidistantes. Las bolas de radio r tienen intersecciones por pares, pero no intersección triple. Por ello, en $\check{C}_r(S)$ aparecen las tres aristas, pero no el 2-símplice. En cambio, como en $VR_r(S)$ solo se comprueban distancias por pares, sí aparece el 2-símplice. Fuente: elaboración propia.	29
5.1.	Filtración elemental de un triángulo. Primero aparecen los tres vértices, después se añaden aristas, se forma un ciclo de dimensión 1 y finalmente el ciclo se rellena al añadir la cara triangular. Fuente: elaboración propia.	37
5.2.	Código de barras y diagrama de persistencia asociados a la filtración del triángulo. En dimensión 0 aparecen tres componentes, dos de las cuales mueren al conectarse con la componente persistente. En dimensión 1, el ciclo nace al cerrar el triángulo y muere al añadir la cara. Fuente: elaboración propia.	39
5.3.	Ejemplo esquemático de un paisaje de persistencia. Cada intervalo finito genera una función triangular, y las funciones $\lambda_1, \lambda_2, \dots$ se obtienen ordenando las alturas para cada valor de u . Fuente: elaboración propia.	40
7.1.	Diagrama de Gantt del desarrollo del trabajo.	52
10.1.	Comparación del Sharpe medio entre DBSCAN y HACA.	68
10.2.	Comparación del Sharpe medio entre macrocluster y solapamiento de nodos.	69

Índice de figuras

10.3. Comparación del Sharpe medio entre liquidez y cartera equiponderada como acción defensiva.	71
10.4. Sensibilidad del Sharpe medio respecto a los parámetros del recubrimiento en KF49.	72
10.5. Sensibilidad del Sharpe medio respecto a los parámetros del recubrimiento en S&P 500.	72
10.6. Evolución encadenada del valor liquidativo para los modelos comparados en KF49.	74
10.7. Evolución encadenada del valor liquidativo para los modelos comparados en S&P 500.	75
10.8. Retorno anualizado medio por modelo y conjunto de datos.	76
10.9. Sharpe medio por modelo y conjunto de datos.	76
10.10 Máximo drawdown medio en valor absoluto por modelo y conjunto de datos.	77
10.11 Sharpe por periodo para los modelos comparados en KF49.	78
10.12 Sharpe por periodo para los modelos comparados en S&P 500.	79

Índice de tablas

7.1. Estimación de dedicación por tareas principales.	50
7.2. Estimación económica orientativa del proyecto.	51
8.1. Principales librerías y herramientas utilizadas en la implementación.	57
9.1. Conjuntos de datos utilizados en los experimentos.	61
9.2. Configuración común del backtesting utilizado en los experimentos.	63
9.3. Métricas utilizadas en las comparaciones experimentales.	64
9.4. Modelos comparados en la evaluación final.	65
10.1. Comparación resumida entre DBSCAN y HACA. Las métricas son medias sobre los cinco periodos de evaluación.	68
10.2. Comparación resumida entre macrocluster y solapamiento de nodos. Las métricas son medias sobre los cinco periodos de evaluación.	69
10.3. Comparación resumida entre las dos acciones defensivas asociadas a la señal de homología persistente. Las métricas son medias sobre los cinco periodos de evaluación.	70
10.4. Configuración final del recubrimiento seleccionada tras el análisis de sensibilidad.	71
10.5. Configuración seleccionada tras las comparaciones realizadas durante la evolución del modelo.	73
10.6. Resultados medios sobre las 49 carteras industriales. Las métricas son medias sobre los cinco periodos de evaluación.	73
10.7. Resultados medios sobre el S&P 500. Las métricas son medias sobre los cinco periodos de evaluación.	74
10.8. Resumen de los mejores modelos por métrica principal.	76
10.9. Tiempos de ejecución de los modelos en KF49.	77
10.10. Tiempos de ejecución de los modelos en S&P 500.	78
A.1. Comparación completa por periodo entre DBSCAN y HACA.	87
A.2. Comparación completa por periodo entre macrocluster y solapamiento de nodos.	88
A.3. Comparación completa por periodo entre liquidez y cartera equiponderada como acción defensiva.	88
A.4. Resultados medios del estudio de sensibilidad del recubrimiento en KF49.	89
A.5. Resultados medios del estudio de sensibilidad del recubrimiento en S&P 500.	89
B.1. Resultados completos por periodo para KF49.	91
B.2. Resultados completos por periodo para S&P 500.	91
C.1. Métricas principales generadas por el script de ejemplo.	100
C.2. Parámetros principales de MapperParams.	102

C.3. Parámetros principales del backtesting.	102
--	-----

Introducción

Este trabajo parte de la siguiente idea: muchos conjuntos de datos actuales son demasiado grandes y complejos como para entenderlos únicamente mirando sus variables por separado. En muchos casos aparecen muchas dimensiones, ruido, relaciones difíciles de visualizar y patrones que cambian según la escala a la que se observen los datos. Por ello, además de estudiar cada componente de forma aislada, puede ser útil analizar cómo se relacionan entre sí y qué estructura global forman en conjunto. Esto ocurre en ámbitos muy distintos, como la biología, la medicina, la informática o las finanzas.

El Análisis Topológico de Datos, conocido habitualmente como TDA por sus siglas en inglés, propone estudiar los datos desde el punto de vista de su forma y de las relaciones entre sus elementos, buscando describir propiedades más globales, como la aparición de agrupamientos, componentes conectadas, ciclos o estructuras de conectividad. Esta perspectiva aparece en el trabajo de Carlsson [7] y en el algoritmo Mapper de Singh, Mémoli y Carlsson [42].

Este Trabajo Fin de Grado combina dos partes. La primera es matemática y está dedicada a entender las herramientas necesarias para trabajar con TDA: complejos simpliciales, homología, filtraciones y homología persistente. La segunda es computacional y aplicada: se desarrolla una librería en Python para utilizar estas ideas sobre datos financieros y evaluar estrategias experimentales de cartera.

Objetivos del trabajo

La propuesta inicial del trabajo planteaba varios objetivos. En primer lugar, familiarizarse con los conceptos matemáticos necesarios para comprender el Análisis Topológico de Datos. En segundo lugar, estudiar la motivación y el funcionamiento general del TDA. En tercer lugar, entender el algoritmo Mapper y hacer uso de él en un contexto aplicado. Por último, aplicar estas herramientas a datos reales de alta dimensionalidad y, si el desarrollo del trabajo lo permitía, comparar el enfoque topológico con técnicas o referencias más clásicas.

Como problema real, se ha optado por centrar la parte aplicada en finanzas, ya que se trata de un contexto donde aparecen de forma natural series temporales multivariantes, relaciones cambiantes entre activos y problemas de alta dimensión.

Además, con el objeto de permitir que usuarios sin conocimientos avanzados de topología puedan utilizar estas herramientas matemáticas, se ha diseñado una librería propia en Python que integra las distintas etapas del trabajo: preparación de datos, construcción topológica, generación de pesos de cartera, backtesting, evaluación de resultados y documentación del software.

Contexto y motivación

Dentro del TDA, uno de los desarrollos más influyentes ha sido la homología persistente, que permite estudiar cómo cambian las características topológicas de un conjunto de datos a

distintas escalas. Junto a ella, el algoritmo Mapper se ha consolidado como una herramienta especialmente útil para obtener representaciones simplificadas de nubes de puntos de alta dimensión, combinando funciones filtro, recubrimientos y clustering local [42]. Estas herramientas han contribuido a hacer del TDA un área cada vez más activa tanto en su dimensión teórica como en su vertiente aplicada [7, 15].

En el ámbito financiero, el interés por estas técnicas ha crecido en los últimos años. La razón es que los mercados financieros generan datos ruidosos, multivariantes y dinámicos, en los que no siempre es fácil identificar patrones relevantes con herramientas tradicionales. En este contexto, algunos trabajos han utilizado homología persistente para estudiar cambios de régimen o episodios de inestabilidad. Por ejemplo, Gidea y Katz analizan señales topológicas asociadas a crisis financieras a partir de paisajes de persistencia [20]. Otros trabajos han explorado el uso de información topológica en problemas relacionados con construcción de carteras y seguimiento de índices [23, 22, 24].

Este trabajo se sitúa dentro de esa línea general. Su propósito es estudiar de manera rigurosa si las técnicas topológicas pueden incorporarse a un flujo de trabajo reproducible y si la información que proporcionan resulta útil al aplicarlas a un problema concreto de construcción de carteras.

Las referencias principales del trabajo se pueden agrupar en tres bloques. Para la parte matemática se han utilizado principalmente los textos de Carlsson [7] y Edelsbrunner y Harer [15], junto con el trabajo original de Mapper [42]. Para la parte financiera han sido importantes los trabajos de Gidea y Katz sobre señales topológicas en mercados financieros [20] y los trabajos de Goel y colaboradores sobre TDA aplicado a decisiones de inversión y construcción de carteras [23, 22, 24]. Por último, para la comparación con una estrategia clásica se ha tomado como referencia la cartera equiponderada estudiada por DeMiguel, Garlappi y Uppal [13].

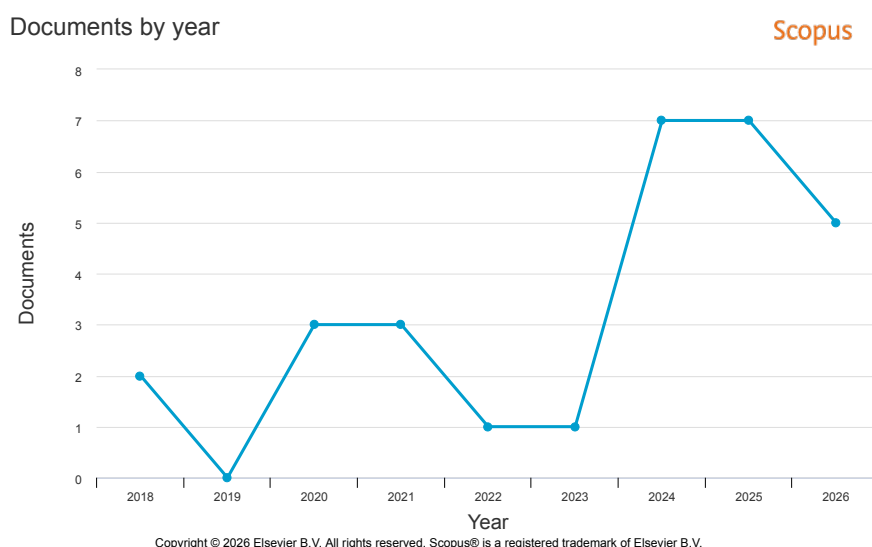


Figura 1.: Evolución del número de documentos relacionados con TDA y finanzas obtenidos en Scopus mediante la búsqueda TITLE-ABS-KEY("topological data analysis" AND finance). Fuente: elaboración propia a partir de Scopus, consulta realizada el 7 de julio de 2026.

Problema abordado

La cuestión principal que se estudia en esta memoria es si la información topológica extraída de series financieras puede utilizarse para construir estrategias de cartera y compararlas de forma razonable con una estrategia sencilla de referencia.

Para ello, cada activo se representa mediante sus retornos en una ventana temporal, de modo que en cada fecha de rebalanceo se obtiene una nube de puntos en la que cada punto corresponde a un activo financiero. Sobre esa nube se aplica Mapper, obteniendo un grafo que resume relaciones estructurales entre los activos. A partir de ese grafo se construye una regla de asignación de pesos de cartera.

Además, se considera una segunda variante en la que se incorpora homología persistente como señal auxiliar de control de régimen. En esta versión, la homología persistente se utiliza para decidir si, en determinados momentos, conviene sustituir temporalmente la cartera obtenida con Mapper por una cartera equiponderada.

La evaluación de estas estrategias se realiza mediante backtesting causal, es decir, utilizando en cada fecha únicamente la información disponible hasta ese momento. De esta forma, la comparación entre modelos se lleva a cabo bajo un protocolo homogéneo y reproducible.

Estructura de la memoria

La memoria se divide en dos grandes partes. La primera está dedicada a los fundamentos matemáticos del Análisis Topológico de Datos. En el Capítulo 1 se presenta la motivación del TDA y el problema de los datos de alta dimensión. El Capítulo 2 introduce herramientas topológicas previas, como conectividad, homotopía y equivalencia homotópica. El Capítulo 3 desarrolla los complejos simpliciales y la homología simplicial. En el Capítulo 4 se estudia cómo pasar de nubes de puntos a objetos topológicos mediante construcciones como el nervio y los complejos de Čech y Vietoris–Rips. El Capítulo 5 introduce las filtraciones y la homología persistente, junto con códigos de barras, diagramas y paisajes de persistencia.

La segunda parte se centra en la aplicación computacional a finanzas. El Capítulo 6 fija los objetivos computacionales y los requisitos del proyecto. El Capítulo 7 recoge la planificación y el presupuesto. El Capítulo 8 presenta la implementación de la librería desarrollada. El Capítulo 9 define el diseño experimental, incluyendo los conjuntos de datos, el preprocesamiento, el protocolo de evaluación, las métricas y los modelos comparados. El Capítulo 10 describe la evolución del modelo, presenta la configuración final seleccionada y recoge los resultados experimentales y su discusión.

La memoria concluye con un capítulo final de conclusiones y vías futuras, seguido de la bibliografía y de varios apéndices con resultados complementarios y el manual de usuario de la librería.

Parte I.

Fundamentos matemáticos del análisis topológico de datos

1. Motivación matemática

1.1. El reto de los datos masivos

Hoy en día es posible trabajar con conjuntos de datos cada vez más grandes y con un número creciente de variables. Sin embargo, disponer de más información no hace necesariamente que el análisis sea más sencillo. La presencia de muchas variables, ruido o valores faltantes puede dificultar la identificación de patrones y limitar la eficacia de algunos métodos estadísticos tradicionales [33].

Una de las principales dificultades aparece cuando el número de dimensiones es elevado. En estas situaciones, las distancias entre los puntos pueden perder parte de su capacidad para distinguir qué elementos son realmente cercanos o lejanos. Este problema forma parte de la conocida como *maldición de la dimensionalidad* [3] y puede dificultar la detección de estructuras relevantes dentro de los datos [4].

Además, cuando el número de variables es grande, deja de ser posible visualizar directamente los datos y resulta más difícil entender cómo se relacionan en conjunto. Una posibilidad es estudiarlos desde un punto de vista geométrico, representándolos como una nube de puntos y analizando las relaciones de proximidad y la estructura global que forman [7].

1.2. Limitaciones de la geometría clásica

Aunque la geometría proporciona una primera forma de estudiar nubes de puntos, también presenta algunas limitaciones cuando los datos son complejos. Carlsson señala varios motivos por los que puede resultar útil adoptar una perspectiva topológica [7]:

- **Información cualitativa.** En algunos problemas interesa comprender primero cómo están organizados los datos. Carlsson pone como ejemplo una base de datos de pacientes con diabetes, donde una primera cuestión relevante sería observar si aparecen distintos tipos de pacientes antes de construir un método cuantitativo para clasificarlos [7]. La topología permite estudiar precisamente este tipo de estructura global, por ejemplo cómo se agrupan o conectan los datos.
- **Dependencia de la métrica.** En algunas áreas, la distancia utilizada viene determinada por el propio problema. En otras, su elección responde a criterios más prácticos o intuitivos. La topología permite centrarse menos en los valores exactos de las distancias y más en las relaciones de cercanía y conectividad entre los datos [7].
- **Dependencia de las coordenadas.** Los datos suelen representarse mediante vectores numéricos, pero las coordenadas elegidas no siempre tienen un significado natural. La topología estudia propiedades que no dependen directamente del sistema de coordenadas y puede ayudar a describir aspectos más globales de la estructura de los datos [7].

- **Estudio multiescala.** Muchos métodos geométricos o de agrupamiento requieren escoger un parámetro de escala. La topología permite estudiar qué ocurre al variar ese parámetro y observar qué estructuras aparecen, desaparecen o se mantienen. Esta idea será fundamental más adelante al introducir la homología persistente [7].

1.3. La forma de los datos: el Análisis Topológico de Datos

El Análisis Topológico de Datos, o TDA por sus siglas en inglés, parte de la idea de que un conjunto de datos puede tener una estructura global que no siempre se aprecia estudiando cada variable, distancia o estadístico de forma aislada. Su objetivo es utilizar herramientas de la topología algebraica para estudiar esa estructura y describir la forma que presentan los datos en conjunto [7].

Esto puede ser especialmente útil cuando se trabaja con datos de alta dimensión, ruidosos o difíciles de visualizar. El TDA permite estudiar aspectos como la existencia de agrupamientos, componentes conectadas, ciclos o cavidades. Estas estructuras ofrecen una forma de resumir cómo se organizan los datos y pueden aportar información que no siempre se observa mediante métodos geométricos o estadísticos clásicos [7].

En este trabajo se utilizarán principalmente dos herramientas. La primera es la homología persistente, que estudia cómo aparecen y desaparecen determinados rasgos homológicos al variar un parámetro de escala [15]. La segunda es Mapper, un algoritmo que construye una representación en forma de grafo y permite obtener un resumen de la estructura de datos complejos [42].

El TDA se ha aplicado en campos muy diferentes. Algunos ejemplos son la clasificación de enfermedades pulmonares [2], el análisis del plegamiento de proteínas [28], el estudio de epidemias [29], el análisis de redes neuronales complejas [21] y el estudio de grandes bases de datos [30].

En este trabajo se desarrollarán primero las herramientas matemáticas necesarias para entender estos métodos y, posteriormente, se aplicarán al contexto financiero, en particular a la detección de regímenes de mercado y a la construcción de carteras [23].

2. Herramientas topológicas necesarias

La topología algebraica estudia propiedades globales de los espacios topológicos que permanecen invariantes bajo deformaciones continuas. Esta perspectiva encaja bien con el análisis topológico de datos, ya que en muchas aplicaciones lo que interesa es la estructura cualitativa del conjunto observado: número de componentes, existencia de lazos, cavidades y patrones de conectividad de distintas dimensiones [7].

La primera información topológica asociada a un espacio es su descomposición en componentes conexas por caminos. Sin embargo, esta información no basta para distinguir espacios que, aun siendo conexos por caminos, tienen estructuras globales distintas. Por ejemplo, una circunferencia y un disco son ambos conexos por caminos, pero la circunferencia contiene un lazo esencial, mientras que el disco puede contraerse a un punto. Para formalizar esta diferencia hacen falta invariantes más finos, capaces de detectar estructuras internas como agujeros o cavidades.

En este capítulo se introducen algunas nociones topológicas previas que servirán para motivar la homología. Primero se presentan los conceptos de camino, conectividad por caminos, homotopía y equivalencia homotópica. Después se comenta el papel de los espacios contráctiles y se explica por qué la homología, aunque pierde parte de la información que capturan los grupos de homotopía, resulta más manejable desde el punto de vista algebraico y computacional. La construcción formal de la homología se dejará para el capítulo siguiente, donde se trabajará con complejos simpliciales [27, 7, 15].

2.1. Conectividad por caminos e información cualitativa

Una de las primeras preguntas que puede plantearse sobre un espacio topológico es si está formado por una sola componente o por varias componentes desconectadas. Esta idea conduce a la noción de conectividad por caminos, que constituye un primer nivel de información cualitativa sobre el espacio [7].

Definición 2.1. Sea X un espacio topológico. Un *camino* en X es una aplicación continua

$$\gamma: [0, 1] \rightarrow X.$$

Si $\gamma(0) = x$ y $\gamma(1) = y$, diremos que γ une x con y .

Definición 2.2. Un espacio topológico X se dice *conexo por caminos* si para cualesquiera dos puntos $x, y \in X$ existe un camino en X que une x con y .

La relación “estar unidos por un camino” es una relación de equivalencia en X , y sus clases de equivalencia reciben el nombre de *componentes conexas por caminos*. El número de estas componentes constituye, por tanto, un primer invariante topológico.

Observación 2.3. La conectividad por caminos detecta únicamente el número de componentes conexas por caminos de un espacio. En particular, no distingue entre espacios conexos por caminos con estructuras globales diferentes. Así, una circunferencia S^1 y un disco D^2

2. Herramientas topológicas necesarias

son ambos conexos por caminos, pero no tienen la misma forma desde el punto de vista topológico.

Dos espacios conexos por caminos con distinta estructura global



Figura 2.1.: Comparación entre una circunferencia y un disco. Ambos espacios son conexos por caminos, pero presentan distinta estructura global. Fuente: elaboración propia.

Carlsson subraya precisamente este punto al presentar la descomposición en componentes conexas como un primer invariante cualitativo, insuficiente todavía para capturar información de orden superior, como la existencia de lazos o cavidades [7].

2.2. Homotopía y deformación continua

La topología algebraica amplía la noción de conectividad estudiando además de puntos y caminos, deformaciones continuas entre aplicaciones. Esta idea se formaliza mediante la noción de homotopía [27].

Definición 2.4. Sean X e Y espacios topológicos y sean $f, g: X \rightarrow Y$ aplicaciones continuas. Se dice que f y g son *homotópicas* si existe una aplicación continua

$$H: X \times [0, 1] \rightarrow Y$$

tal que

$$H(x, 0) = f(x), \quad H(x, 1) = g(x)$$

para todo $x \in X$. En tal caso se escribe $f \simeq g$, y a H se le llama una *homotopía* entre f y g .

Intuitivamente, una homotopía describe una deformación continua de la aplicación f hasta la aplicación g . Desde este punto de vista, dos aplicaciones homotópicas representan el mismo comportamiento global.

Un caso especialmente útil es el de las deformaciones de un espacio sobre uno de sus subespacios.

Definición 2.5. Sea X un espacio topológico y sea $A \subseteq X$ un subespacio. Una *retracción por deformación* de X sobre A es una homotopía

$$F: X \times [0, 1] \rightarrow X$$

tal que

$$F(x, 0) = x \quad \text{para todo } x \in X, \quad F(x, 1) \in A \quad \text{para todo } x \in X,$$

y

$$F(a, t) = a \quad \text{para todo } a \in A \text{ y todo } t \in [0, 1].$$

En la **Figura 2.2** se muestra un ejemplo intuitivo: el anillo se deforma radialmente hasta una circunferencia central que permanece fija durante toda la deformación.

Retracción por deformación de un anillo sobre una circunferencia

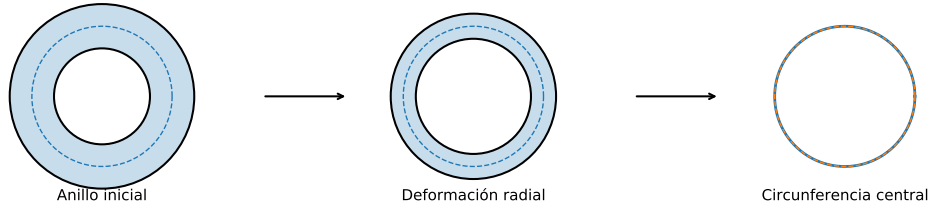


Figura 2.2.: Esquema de una retracción por deformación de un anillo sobre una circunferencia contenida en él. Fuente: elaboración propia.

Esta noción expresa que el espacio X puede deformarse continuamente sobre el subespacio A dejando fijo a A durante todo el proceso. Hatcher presenta las retracciones por deformación como una de las herramientas geométricas básicas para comparar espacios desde el punto de vista topológico [27].

2.3. Equivalencia homotópica y espacios contráctiles

La equivalencia homotópica es una forma flexible de comparar espacios topológicos. Dos espacios pueden no ser iguales desde el punto de vista geométrico y, aun así, tener la misma forma desde el punto de vista de la homotopía [27, 7].

Definición 2.6. Sean X e Y espacios topológicos. Se dice que X e Y son *homotópicamente equivalentes* si existen aplicaciones continuas

$$f: X \rightarrow Y, \quad g: Y \rightarrow X$$

tales que

$$g \circ f \simeq \text{id}_X, \quad f \circ g \simeq \text{id}_Y.$$

En este caso, f se denomina una *equivalencia homotópica* y se escribe

$$X \simeq Y.$$

Por ejemplo, si un espacio X se retrae por deformación sobre un subespacio A , entonces X y A son homotópicamente equivalentes [27].

Definición 2.7. Un espacio topológico X se dice *contráctil* si es homotópicamente equivalente a un punto.

2. Herramientas topológicas necesarias

Equivalentemente, un espacio es contráctil si su identidad es homotópica a una aplicación constante. En muchos ejemplos sencillos, esto coincide con la idea intuitiva de poder deformar el espacio de forma continua hasta un único punto [27].

Observación 2.8. La importancia de la equivalencia homotópica es que permite sustituir un espacio por otro más sencillo sin cambiar los invariantes topológicos que se estudiarán después, como los grupos de homología.

2.4. De la homotopía a la homología

Aunque los grupos de homotopía contienen información topológica muy rica, en la práctica son difíciles de calcular, especialmente en dimensiones altas. Incluso para espacios sencillos como la esfera, el cálculo de $\pi_i(S^n)$ con $i > n$ no es inmediato [27].

Una de las dificultades principales es que los grupos de homotopía superiores no se leen directamente a partir de una descomposición celular o simplicial del espacio. Hatcher señala, por ejemplo, que aunque la esfera S^2 no tiene celdas de dimensión mayor que 2, sus grupos de homotopía $\pi_n(S^2)$ son no nulos para infinitos valores de n [27].

Por este motivo resulta natural introducir la homología. La homología no conserva toda la información de la homotopía, pero sí recoge una parte importante de la estructura global del espacio y resulta más manejable desde el punto de vista algebraico y computacional.

La homología adopta un enfoque distinto. En vez de trabajar directamente con clases de aplicaciones continuas, parte de una descripción más combinatoria del espacio, por ejemplo mediante celdas o símlices. A partir de esa descripción se construyen combinaciones formales, llamadas cadenas, y se estudia cuáles tienen borde nulo y cuáles aparecen como borde de una cadena de dimensión superior [27].

El precio de esta mayor manejabilidad es que la definición formal de los grupos de homología requiere introducir cierta maquinaria algebraica previa: grupos de cadenas, operadores borde, ciclos, bordes y grupos cociente. Esta construcción se desarrollará en el capítulo siguiente.

Observación 2.9. La naturaleza abeliana de los grupos de homología es una de las razones de su utilidad. Frente a la complejidad de los grupos de homotopía, la homología se expresa en términos algebraicos más manejables y, en contextos discretos adecuados, puede calcularse mediante álgebra lineal [15].

2.5. Comentario sobre computabilidad

Una de las razones por las que la homología resulta importante en topología aplicada es que puede calcularse de forma efectiva cuando el espacio está descrito mediante un complejo simplicial finito. En ese caso, las cadenas se representan mediante vectores y el operador borde se representa mediante matrices.

De este modo, el cálculo de la homología se reduce al cálculo de núcleos e imágenes de aplicaciones lineales. Edelsbrunner y Harer explican que, a partir de estas matrices frontera, se obtienen los rangos de los grupos de ciclos y bordes, y sus diferencias proporcionan los números de Betti [15].

Por tanto, la homología ocupa una posición intermedia útil: es más fina que la conectividad por caminos, pero más tratable que la homotopía desde un punto de vista algorítmico.

Esta combinación explica su papel central tanto en topología algebraica clásica como en análisis topológico de datos.

En el capítulo siguiente se desarrollará esta idea en el caso simplicial. Para ello se introducirán los símlices, los complejos simpliciales, los grupos de cadenas, el operador borde, los ciclos, los bordes y los grupos cociente que dan lugar a la homología simplicial.

3. Complejos simpliciales y homología simplicial

En el capítulo anterior se ha visto que la conectividad por caminos no basta para distinguir algunos espacios topológicos sencillos. También se ha explicado que interesa trabajar con invariantes que tengan significado topológico, pero que puedan calcularse de forma efectiva. La homología responde precisamente a esta idea.

Para definirla de manera concreta, en este capítulo se trabajará con complejos simpliciales. La idea es representar los espacios mediante símlices, que son objetos geométricos sencillos, y estudiar cómo se unen entre sí. A partir de esta estructura se construyen los grupos de cadenas, el operador borde, los ciclos, los bordes y, finalmente, los grupos de homología. Seguimos aquí la construcción estándar de la homología simplicial, como puede encontrarse en [27, 15].

3.1. Símlices geométricos

Los símlices constituyen la generalización en dimensión arbitraria de los segmentos, triángulos y tetraedros.

Definición 3.1. Sean $v_0, \dots, v_n \in \mathbb{R}^m$ puntos afínmente independientes. El n -símplice geométrico generado por dichos vértices es el subconjunto

$$[v_0, \dots, v_n] = \left\{ \sum_{i=0}^n \lambda_i v_i \mid \lambda_i \geq 0, \sum_{i=0}^n \lambda_i = 1 \right\}.$$

La condición de independencia afín garantiza que el símplice tenga realmente dimensión n . Equivale a exigir que los vectores

$$v_1 - v_0, v_2 - v_0, \dots, v_n - v_0$$

sean linealmente independientes. De este modo se evita que la figura degenera en un subespacio de dimensión menor.

Los casos de baja dimensión son familiares:

- un 0-símplice es un punto;
- un 1-símplice es un segmento;
- un 2-símplice es un triángulo sólido;
- un 3-símplice es un tetraedro sólido.

Geométricamente, un símplice es la envolvente convexa de sus vértices. En este sentido, el símplice constituye el objeto geométrico de caras planas más elemental en cada dimensión.

Definición 3.2. Una *cara* de un símplice $[v_0, \dots, v_n]$ es cualquier símplice de la forma

$$[v_{i_0}, \dots, v_{i_k}], \quad 0 \leq i_0 < \dots < i_k \leq n.$$

3. Complejos simpliciales y homología simplicial

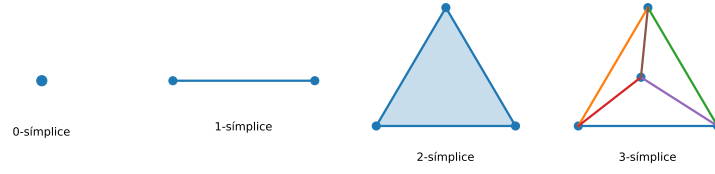


Figura 3.1.: Ejemplos de símlices geométricos en dimensiones 0, 1, 2 y 3. Fuente: elaboración propia.

Si $k = n - 1$, la cara se denomina *cara de codimensión uno*.

Por ejemplo, las caras de un triángulo son sus vértices, sus aristas y el propio triángulo. Las caras de un tetraedro son sus vértices, sus aristas, sus caras triangulares y el propio tetraedro.

3.2. Complejos simpliciales

La idea de un complejo simplicial consiste en construir espacios a partir de símlices unidos entre sí de manera compatible.

Definición 3.3. Un *complejo simplicial geométrico* K^{geo} es una colección de símlices geométricos en $\mathbb{R}^m$ tal que:

1. toda cara de un símplex de K^{geo} pertenece también a K^{geo} ;
2. la intersección de dos símlices de K^{geo} es vacía o bien una cara común de ambos.

La dimensión de un complejo simplicial geométrico es la máxima dimensión de los símlices que contiene.

La condición de compatibilidad entre símlices se ilustra en la Figura 3.2. En particular, la intersección de dos símlices solo puede ser vacía o una cara común de ambos. En términos intuitivos, esta segunda condición asegura que la intersección de dos elementos del complejo es, a su vez, una subestructura bien definida del mismo.

Estrictamente hablando, K^{geo} es una colección de símlices geométricos. La unión de todos ellos, dotada de la topología subespacio heredada del espacio euclídeo ambiente, se denomina *espacio subyacente* del complejo y se denota por

$$|K^{\text{geo}}|.$$

Ejemplo 3.4. El borde de un triángulo, formado por tres vértices y tres aristas, es un complejo simplicial geométrico de dimensión 1. Su espacio subyacente es homeomorfo a la circunferencia S^1 .

Ejemplo 3.5. Un triángulo con su interior incluido, junto con sus aristas y vértices, es un complejo simplicial geométrico de dimensión 2. Su espacio subyacente es homeomorfo al disco D^2 .

Ejemplo 3.6. La superficie de un tetraedro, formada por sus cuatro caras triangulares, sus aristas y sus vértices, es un complejo simplicial geométrico de dimensión 2. Su espacio subyacente es homeomorfo a la esfera S^2 .

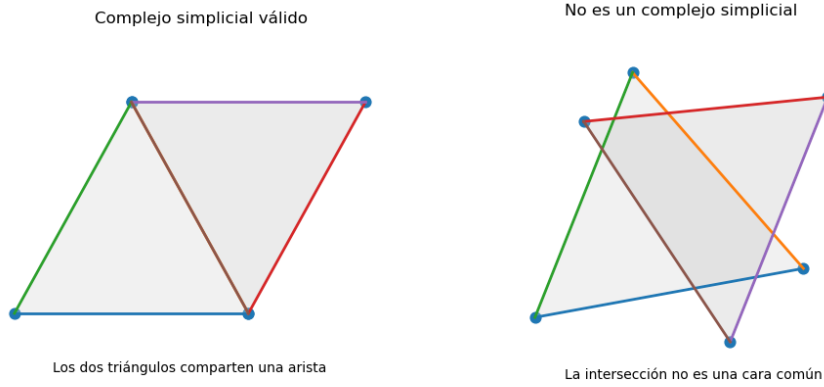


Figura 3.2.: A la izquierda, un complejo simplicial válido, ya que los dos triángulos comparten una cara común. A la derecha, una configuración que no define un complejo simplicial, porque la intersección de los triángulos no es una cara común. Fuente: elaboración propia.

Además del punto de vista geométrico, también resulta útil la formulación puramente combinatoria.

Definición 3.7. Sea V un conjunto de vértices. Un *complejo simplicial abstracto* sobre V es una familia K de subconjuntos finitos no vacíos de V tal que, si $\sigma \in K$ y

$$\emptyset \neq \tau \subseteq \sigma,$$

entonces $\tau \in K$.

Los elementos de K se denominan *símplices abstractos*. Si un símplex σ contiene $k + 1$ vértices, se define su dimensión como k .

Un complejo simplicial geométrico determina un complejo simplicial abstracto al olvidar la posición concreta de sus vértices y conservar únicamente qué conjuntos de vértices forman cada símplex. En sentido contrario, un complejo simplicial abstracto puede representarse mediante símplexes geométricos unidos de acuerdo con su estructura combinatoria. Un complejo geométrico obtenido de esta forma se denomina una *realización geométrica* del complejo abstracto [15].

Cuando sea necesario considerar el espacio topológico asociado a una realización geométrica de K , utilizaremos la notación $|K|$.

Ejemplo 3.8. Consideremos el conjunto de vértices $V = \{x, y, z\}$. La familia

$$K = \{\{x\}, \{y\}, \{z\}, \{x, y\}, \{y, z\}, \{z, x\}\}$$

es un complejo simplicial abstracto de dimensión 1. En efecto, contiene todos los subconjuntos no vacíos de cada uno de sus símplexes y no contiene el subconjunto $\{x, y, z\}$, por lo que no aparece ningún símplex de dimensión 2.

Una realización geométrica de K puede obtenerse tomando como símplexes los tres vértices y los tres lados de un triángulo. El espacio subyacente de esta realización es homeomorfo a la circunferencia S^1 .

3. Complejos simpliciales y homología simplicial

Esta perspectiva combinatoria será especialmente útil más adelante, cuando se trabaje con estructuras discretas obtenidas a partir de datos [15].

Observación 3.9. A partir de este punto, salvo que se indique lo contrario, por *complejo simplicial* entenderemos un complejo simplicial abstracto K . Cuando sea necesario considerar el espacio topológico asociado a una realización geométrica de K , utilizaremos la notación $|K|$.

3.3. Orientación de símplexes

Para construir la homología simplicial es necesario introducir orientaciones, ya que el operador borde debe distinguir las caras con signo.

Definición 3.10. Sea

$$\sigma = \{v_0, \dots, v_n\} \in K$$

un n -símplex. Una *orientación* de σ es una clase de equivalencia de ordenaciones de sus vértices, donde dos ordenaciones se consideran equivalentes si difieren en una permutación par.

Esta relación de equivalencia divide las $(n+1)!$ ordenaciones posibles de los vértices en exactamente dos clases. Intercambiar dos vértices invierte la orientación del símplex.

Si se representa un símplex orientado escribiendo sus vértices entre paréntesis, entonces se cumple la relación

$$(v_0, \dots, v_i, \dots, v_j, \dots, v_n) = -(v_0, \dots, v_j, \dots, v_i, \dots, v_n).$$

Ejemplos.

- En dimensión 1, un segmento con vértices v_0, v_1 tiene dos orientaciones:

$$(v_0, v_1) \quad \text{y} \quad (v_1, v_0),$$

que representan sentidos opuestos.

- En dimensión 2, las dos orientaciones posibles de un triángulo corresponden a recorrer sus vértices en sentido horario o antihorario.

La orientación será la responsable de la aparición de signos alternados en la definición del operador borde.

3.4. Grupos de cadenas

Una vez fijado un complejo simplicial K , la siguiente idea consiste en considerar combinaciones lineales formales de símplexes orientados.

Definición 3.11. Sea K un complejo simplicial. El *grupo de n -cadenas* de K , denotado por $C_n(K)$, es el grupo abeliano libre generado por los n -símplexes orientados de K , con la relación

$$(v_{\sigma(0)}, \dots, v_{\sigma(n)}) = \text{sgn}(\sigma) (v_0, \dots, v_n)$$

para toda permutación $\sigma \in S_{n+1}$, donde $\text{sgn}(\sigma)$ denota el signo de la permutación (+1 si es par, -1 si es impar).

Esta relación algebraica formaliza lo expuesto en la sección anterior: un cambio de orientación invierte el signo del símplice en el grupo. De este modo, un símplice recorrido en sentido opuesto actúa como el inverso aditivo del original, cumpliéndose por ejemplo que $(v_1, v_0) = -(v_0, v_1)$.

Un elemento de $C_n(K)$ es, por tanto, una suma formal finita

$$c = \sum_i a_i \sigma_i, \quad a_i \in \mathbb{Z},$$

donde los σ_i son n -símplices orientados. El calificativo “formal” indica que los símplices actúan de forma análoga a los elementos de una base en un espacio vectorial; los coeficientes a_i simplemente indican la multiplicidad o el “peso” con el que cada símplice aparece en la cadena, y su signo determina la dirección en la que se recorre.

Interpretación geométrica.

- Los elementos de $C_0(K)$ son combinaciones enteras de vértices.
- Los elementos de $C_1(K)$ son combinaciones enteras de aristas orientadas.
- Los elementos de $C_2(K)$ son combinaciones enteras de triángulos orientados.

En esta definición se han tomado coeficientes enteros, que constituyen la elección clásica en homología simplicial. Más generalmente, puede construirse un grupo de cadenas $C_n(K; G)$ con coeficientes en un grupo abeliano G . Cuando $G = \mathbb{F}$ es un cuerpo, los grupos de cadenas, ciclos, bordes y homología adquieren estructura de espacios vectoriales. Este caso será útil para definir los números de Betti como dimensiones de espacios vectoriales y será también el considerado más adelante al introducir la homología persistente.

Observación 3.12. En topología computacional es frecuente trabajar con coeficientes en el cuerpo $\mathbb{Z}_2$. En este caso los coeficientes solo pueden tomar los valores 0 y 1, y se tiene $1 = -1$, por lo que los signos que aparecen en el operador borde se identifican. Esto simplifica los cálculos [15].

3.5. El operador borde

La estructura esencial que permite pasar de geometría a álgebra es el operador borde.

Definición 3.13. Para $n \geq 1$, el *operador borde*

$$\partial_n : C_n(K) \rightarrow C_{n-1}(K)$$

se define sobre un n -símplice orientado $(v_0, \dots, v_n)$ por

$$\partial_n(v_0, \dots, v_n) = \sum_{i=0}^n (-1)^i (v_0, \dots, \widehat{v_i}, \dots, v_n),$$

donde $\widehat{v_i}$ indica que el vértice v_i se omite. Por convenio, se define $\partial_0 = 0$.

Esta fórmula expresa que el borde de un símplice es la suma alternada de sus caras de codimensión uno.

3. Complejos simpliciales y homología simplicial

Ejemplos.

- Si (v_0, v_1) es una arista orientada, entonces

$$\partial_1(v_0, v_1) = v_1 - v_0.$$

- Si (v_0, v_1, v_2) es un triángulo orientado, entonces

$$\partial_2(v_0, v_1, v_2) = (v_1, v_2) - (v_0, v_2) + (v_0, v_1).$$

- Si (v_0, v_1, v_2, v_3) es un tetraedro orientado, entonces

$$\partial_3(v_0, v_1, v_2, v_3) = (v_1, v_2, v_3) - (v_0, v_2, v_3) + (v_0, v_1, v_3) - (v_0, v_1, v_2).$$

La linealidad extiende ∂_n a cualquier n -cadena.

3.6. La identidad $\partial_{n-1} \circ \partial_n = 0$

La propiedad fundamental del operador borde es que el borde de un borde es siempre nulo.

Proposición 3.14. Para todo $n \geq 1$, se tiene

$$\partial_{n-1} \circ \partial_n = 0.$$

Demostración. Para $n = 1$, la igualdad es inmediata, ya que $\partial_0 = 0$. Supongamos entonces $n \geq 2$. Basta probar la igualdad sobre los generadores, es decir, sobre un n -símplice orientado $\sigma = (v_0, \dots, v_n)$. Aplicando la definición del operador borde dos veces, obtenemos:

$$\begin{aligned} \partial_{n-1}(\partial_n(\sigma)) &= \partial_{n-1} \left(\sum_{i=0}^n (-1)^i (v_0, \dots, \widehat{v}_i, \dots, v_n) \right) \\ &= \sum_{i=0}^n (-1)^i \partial_{n-1}(v_0, \dots, \widehat{v}_i, \dots, v_n). \end{aligned}$$

Al desarrollar el segundo operador borde, hay que tener cuidado con los índices. Si eliminamos un segundo vértice v_j , su posición en la lista depende de si estaba antes o después de v_i . Por tanto, separamos la suma en dos casos ($j < i$ y $j > i$):

$$\begin{aligned} \partial_{n-1}(\partial_n(\sigma)) &= \sum_{i=0}^n (-1)^i \left[\sum_{j=0}^{i-1} (-1)^j (v_0, \dots, \widehat{v}_j, \dots, \widehat{v}_i, \dots, v_n) \right. \\ &\quad \left. + \sum_{j=i+1}^n (-1)^{j-1} (v_0, \dots, \widehat{v}_i, \dots, \widehat{v}_j, \dots, v_n) \right]. \end{aligned}$$

Agrupando los términos, observamos que cada cara de dimensión $n-2$, de la forma $(v_0, \dots, \widehat{v}_k, \dots, \widehat{v}_l, \dots, v_n)$ con $k < l$, aparece exactamente dos veces en la suma doble:

- Una vez cuando $i = l$ y $j = k$, con signo $(-1)^{l+k}$.
- Otra vez cuando $i = k$ y $j = l$, con signo $(-1)^{k+l-1}$.

Como $(-1)^{l+k}$ y $(-1)^{k+l-1}$ tienen signos opuestos, las dos contribuciones se cancelan mutuamente para cada par de vértices eliminados, resultando en que la suma total es idénticamente nula. $\square$

Esta identidad es el núcleo algebraico de la teoría. Su consecuencia inmediata es que toda cadena que sea borde de una cadena de dimensión superior tiene borde nulo.

3.7. Ciclos, bordes y grupos de homología

La proposición anterior permite introducir dos subgrupos fundamentales dentro del grupo de cadenas.

Definición 3.15. Sea K un complejo simplicial. Para cada $n \geq 0$, se definen:

- El grupo de n -ciclos, formado por las cadenas cuyo borde es nulo:

$$Z_n(K) = \ker \partial_n.$$

- El grupo de n -bordes, formado por las cadenas que son imagen del operador borde aplicado a una cadena de dimensión superior:

$$B_n(K) = \text{Im } \partial_{n+1}.$$

La identidad $\partial_{n-1} \circ \partial_n = 0$ implica directamente que la imagen de ∂_{n+1} está contenida en el núcleo de ∂_n . Esto se traduce en la inclusión de grupos:

$$B_n(K) \subseteq Z_n(K),$$

lo que equivale a afirmar que todo borde es, necesariamente, un ciclo.

Sin embargo, la implicación recíproca no tiene por qué ser cierta. Puede haber ciclos que no sean el borde de ninguna cadena de dimensión superior. Esta diferencia es precisamente la que mide la homología.

Definición 3.16. El n -ésimo grupo de homología simplicial de K se define como el grupo cociente:

$$H_n(K) = Z_n(K) / B_n(K).$$

Interpretación geométrica. Los elementos de $H_n(K)$ son clases de equivalencia de ciclos, denominadas *clases de homología*. Dos n -ciclos z_1 y z_2 pertenecen a la misma clase (y se dicen *homólogos*) si su diferencia es el borde de una cadena de dimensión $n+1$, es decir, si $z_1 - z_2 \in B_n(K)$. Geométricamente, esto significa que ambos ciclos representan el mismo rasgo homológico dentro del complejo.

Observación 3.17. Al realizar el cociente por $B_n(K)$, cualquier ciclo que sea borde de una cadena de dimensión superior se anula en el grupo de homología ($[z] = 0$). Por tanto, los elementos no nulos de $H_n(K)$ representan ciclos que no están rellenados dentro del complejo. Esta es la idea que permite interpretar la homología como una forma algebraica de detectar agujeros.

3. Complejos simpliciales y homología simplicial

Observación 3.18. Sean K y L dos complejos simpliciales. Si sus realizaciones geométricas son homotópicamente equivalentes,

$$|K| \simeq |L|,$$

entonces sus grupos de homología son isomorfos:

$$H_n(K) \cong H_n(L) \quad \text{para todo } n \geq 0.$$

Por tanto, la homología permite comparar la estructura topológica de los espacios representados por los complejos [27, 15].

Observación 3.19. Una propiedad importante para este trabajo aparece cuando un complejo simplicial K es subcomplejo de otro complejo L . La inclusión

$$\iota: K \hookrightarrow L$$

permite considerar toda cadena de K como una cadena de L . Con coeficientes en un grupo abeliano G , esto define aplicaciones

$$\iota_{\#}: C_n(K; G) \rightarrow C_n(L; G).$$

Estas aplicaciones son compatibles con el operador borde:

$$\partial_n^L \circ \iota_{\#} = \iota_{\#} \circ \partial_n^K.$$

Por tanto, envían ciclos en ciclos y bordes en bordes, e inducen un homomorfismo

$$\iota_*: H_n(K; G) \rightarrow H_n(L; G), \quad \iota_*([z]) = [\iota_{\#}(z)].$$

Esta idea será fundamental en el capítulo dedicado a la homología persistente, donde se compararán los grupos de homología de distintos complejos dentro de una misma filtración [27, 15].

3.8. Los números de Betti

Cuando los grupos de homología se consideran con coeficientes en un cuerpo, adquieren estructura de espacios vectoriales y su dimensión proporciona una medida numérica de la información homológica del complejo [15, 27].

Definición 3.20. Sea K un complejo simplicial y sea $\mathbb{F}$ un cuerpo. El n -ésimo número de Betti de K con coeficientes en $\mathbb{F}$ se define como

$$\beta_n(K; \mathbb{F}) = \dim_{\mathbb{F}} H_n(K; \mathbb{F}).$$

Dado que

$$H_n(K; \mathbb{F}) = Z_n(K; \mathbb{F}) / B_n(K; \mathbb{F}),$$

se obtiene

$$\beta_n(K; \mathbb{F}) = \dim_{\mathbb{F}} Z_n(K; \mathbb{F}) - \dim_{\mathbb{F}} B_n(K; \mathbb{F}).$$

En bajas dimensiones, estos números admiten la siguiente interpretación geométrica:

- β_0 es el número de componentes conexas por caminos;
- β_1 es el número de clases de homología independientes representadas por 1-ciclos que no son bordes;
- β_2 es el número de clases de homología independientes representadas por 2-ciclos que no son bordes.

3.9. Característica de Euler

Además de los números de Betti, existe otro invariante clásico asociado a un complejo simplicial finito: la característica de Euler. Si K es un complejo simplicial finito y f_n denota el número de n -símplices de K , se define

$$\chi(K) = \sum_{n \geq 0} (-1)^n f_n.$$

Como K es finito, esta suma tiene solo un número finito de términos no nulos.

Cuando se trabaja con coeficientes en un cuerpo, la característica de Euler también puede expresarse en términos de los números de Betti:

$$\chi(K) = \sum_{n \geq 0} (-1)^n \beta_n.$$

Esta relación muestra que la característica de Euler resume parte de la información homológica del complejo en un único número [27, 15].

En este trabajo no se utilizará directamente como herramienta computacional, pero resulta útil mencionarla porque conecta la información combinatoria del complejo, dada por el número de símplices, con la información homológica, dada por los números de Betti.

3.10. Ejemplos básicos

Se calculan a continuación los grupos de homología simplicial de varios ejemplos fundamentales. Los cálculos siguen directamente las definiciones anteriores y la construcción estándar de la homología simplicial presentada en [27, 15].

En los ejemplos siguientes se calcularán los grupos de homología con coeficientes enteros. En todos los casos considerados, estos grupos son de la forma

$$H_n(K; \mathbb{Z}) \cong \mathbb{Z}^r$$

para algún $r \geq 0$. Por tanto, el número de Betti viene dado por

$$\beta_n = \text{rank } H_n(K; \mathbb{Z}) = r.$$

Equivalentemente, si se trabaja con coeficientes en un cuerpo $\mathbb{F}$, entonces en estos ejemplos se obtiene

$$H_n(K; \mathbb{F}) \cong \mathbb{F}^r,$$

y por tanto

$$\beta_n = \dim_{\mathbb{F}} H_n(K; \mathbb{F}) = r.$$

3. Complejos simpliciales y homología simplicial

Cuando un espacio topológico X admite una triangulación, existe un complejo simplicial abstracto K y una realización geométrica de K cuyo espacio subyacente es homeomorfo a X . Los grupos de homología obtenidos no dependen de la triangulación elegida. Por ello, en estos casos escribiremos $H_n(X)$ para referirnos a los grupos de homología del espacio [27, 15].

3.10.1. La circunferencia

Consideremos el complejo simplicial abstracto

$$K = \{\{x\}, \{y\}, \{z\}, \{x, y\}, \{y, z\}, \{z, x\}\}.$$

Una realización geométrica de K puede obtenerse tomando como símlices los tres vértices y los tres lados de un triángulo. El espacio subyacente de esta realización es homeomorfo a S^1 .

Elegimos las siguientes orientaciones para sus tres aristas:

$$a = (x, y), \quad b = (y, z), \quad c = (z, x).$$

Los grupos de cadenas son

$$C_0(K) = \langle x, y, z \rangle \cong \mathbb{Z}^3, \quad C_1(K) = \langle a, b, c \rangle \cong \mathbb{Z}^3, \quad C_n(K) = 0 \text{ para } n \geq 2.$$

El operador borde en dimensión 1 viene dado por

$$\partial_1(a) = y - x, \quad \partial_1(b) = z - y, \quad \partial_1(c) = x - z.$$

De aquí se obtiene

$$Z_0(K) = C_0(K), \quad B_0(K) = \langle y - x, z - y, x - z \rangle,$$

por lo que

$$H_0(K) \cong \mathbb{Z}.$$

Además,

$$Z_1(K) = \ker \partial_1 = \langle a + b + c \rangle \cong \mathbb{Z}, \quad B_1(K) = 0,$$

y por tanto

$$H_1(K) \cong \mathbb{Z}.$$

Finalmente,

$$H_n(K) = 0 \quad \text{para todo } n \geq 2.$$

En consecuencia,

$$\beta_0 = 1, \quad \beta_1 = 1, \quad \beta_n = 0 \text{ para } n \geq 2.$$

3.10.2. El disco

Consideremos el complejo simplicial abstracto

$$K = \{\{x\}, \{y\}, \{z\}, \{x, y\}, \{y, z\}, \{z, x\}, \{x, y, z\}\}.$$

Una realización geométrica de K puede obtenerse tomando un triángulo sólido junto con todas sus caras. El espacio subyacente de esta realización es homeomorfo al disco D^2 .

Elegimos las orientaciones

$$a = (x, y), \quad b = (y, z), \quad c = (z, x)$$

para las aristas, y

$$A = (x, y, z)$$

para el 2-símplice.

Los grupos de cadenas relevantes son

$$C_0(K) \cong \mathbb{Z}^3, \quad C_1(K) \cong \mathbb{Z}^3, \quad C_2(K) \cong \mathbb{Z}.$$

El borde de la cara viene dado por

$$\partial_2(A) = a + b + c.$$

Como antes,

$$H_0(K) \cong \mathbb{Z}.$$

Para la homología de dimensión 1, se tiene

$$Z_1(K) = \langle a + b + c \rangle, \quad B_1(K) = \langle a + b + c \rangle,$$

por lo que

$$H_1(K) = 0.$$

Además,

$$Z_2(K) = 0, \quad B_2(K) = 0,$$

de donde

$$H_2(K) = 0.$$

Por tanto,

$$H_0(K) \cong \mathbb{Z}, \quad H_n(K) = 0 \text{ para } n \geq 1.$$

Esto concuerda con el hecho de que el disco es contráctil.

3.10.3. La esfera m -dimensional

Para la esfera m -dimensional S^m , con $m \geq 1$, el resultado estándar es el siguiente [27]:

$$H_n(S^m) \cong \begin{cases} \mathbb{Z}, & n = 0, \\ \mathbb{Z}, & n = m, \\ 0, & \text{en otro caso.} \end{cases}$$

En términos geométricos, $H_0(S^m) \cong \mathbb{Z}$ refleja que la esfera es conexa, mientras que $H_m(S^m) \cong \mathbb{Z}$ representa su clase fundamental de dimensión m . La anulación de los demás grupos expresa que no aparecen clases homológicas independientes en dimensiones intermedias.

3. Complejos simpliciales y homología simplicial

En particular, para $m = 2$ se recupera

$$H_0(S^2) \cong \mathbb{Z}, \quad H_1(S^2) = 0, \quad H_2(S^2) \cong \mathbb{Z}.$$

3.10.4. El toro

Como ejemplo adicional, recordamos el resultado estándar para el toro T^2 [27]:

$$H_0(T^2) \cong \mathbb{Z}, \quad H_1(T^2) \cong \mathbb{Z} \oplus \mathbb{Z}, \quad H_2(T^2) \cong \mathbb{Z},$$

y

$$H_n(T^2) = 0 \quad \text{para } n \geq 3.$$

En consecuencia, los números de Betti del toro son

$$\beta_0 = 1, \quad \beta_1 = 2, \quad \beta_2 = 1.$$

Geométricamente, esto expresa que el toro es conexo, posee dos clases independientes de lazos esenciales y tiene una clase de homología de dimensión 2 asociada a la propia superficie. Este ejemplo ilustra de manera clara que la homología permite distinguir espacios que, aun siendo ambos conexos, tienen estructuras globales notablemente distintas.

Los ejemplos anteriores muestran cómo la homología simplicial permite traducir información geométrica global en grupos abelianos calculables a partir de un complejo. En el capítulo siguiente se abordará el problema que aparece en el contexto de datos: dado que normalmente partimos de una nube finita de puntos, será necesario explicar cómo construir objetos topológicos adecuados a partir de dichas muestras.

4. De nubes de puntos a objetos topológicos

En los capítulos anteriores se han introducido las herramientas algebraicas necesarias para estudiar la topología de un espacio a través de su homología. Sin embargo, en el contexto del análisis de datos el punto de partida suele ser una muestra finita de puntos en un espacio métrico. Surge entonces una cuestión fundamental: cómo pasar de una nube de puntos a un complejo simplicial sobre el cual sea posible calcular los invariantes algebraicos desarrollados anteriormente.

Esta cuestión constituye uno de los problemas centrales del análisis topológico de datos. La idea general consiste en asociar a una nube de puntos un complejo simplicial cuya estructura capture información topológica del objeto subyacente del que los datos proceden. Una manera de justificar este paso consiste en construir complejos cuyas realizaciones geométricas estén relacionadas, mediante equivalencia homotópica, con ciertos espacios continuos obtenidos a partir de la nube de puntos. En particular, los complejos de Čech y de Vietoris–Rips son dos construcciones fundamentales en este contexto [7, 15].

El objetivo de este capítulo es describir este paso de datos discretos a objetos topológicos. Para ello se introducirá primero el problema de reconstrucción topológica a partir de muestras finitas. A continuación se estudiarán los sistemas finitos de conjuntos, el nervio de una familia y el teorema del nervio, que proporcionan la justificación matemática de varias construcciones utilizadas en topología computacional. Posteriormente, se aplicará esta teoría al caso particular de uniones de bolas asociadas a nubes de puntos, dando lugar al complejo de Čech. Finalmente, se presentará el complejo de Vietoris–Rips y se explicará por qué la dependencia respecto al parámetro de escala conduce de forma natural a la homología persistente.

4.1. El problema de reconstrucción topológica

Sea $X \subseteq \mathbb{R}^d$ un subconjunto geométrico que representa el objeto que se quiere estudiar, y sea

$$S = \{x_1, \dots, x_n\} \subseteq \mathbb{R}^d$$

una muestra finita de puntos generada a partir de él.

En el caso ideal sin ruido se supone que

$$S \subseteq X.$$

En presencia de ruido, los puntos de S pueden encontrarse cerca de X sin pertenecer exactamente a él. En ambos casos, en la práctica solo se observa la muestra finita S , mientras que el objeto subyacente X no se conoce por completo.

El problema consiste en inferir, a partir de la nube de puntos S , alguna información sobre la topología de X , como su número de componentes conexas o la existencia de clases homológicas no triviales.

Desde un punto de vista conceptual, esta tarea presenta dos dificultades. La primera es que una nube de puntos finita, considerada aisladamente, posee una topología demasiado

4. De nubes de puntos a objetos topológicos

simple si se le dota de la topología discreta: cada punto queda aislado y no aparecen rasgos topológicos de dimensión superior. La segunda es que la reconstrucción de la topología del objeto subyacente depende de la densidad del muestreo y de la escala a la que se observen los datos [7].

Por tanto, no basta con estudiar el conjunto S únicamente como conjunto finito. Es necesario enriquecerlo mediante una construcción que incorpore relaciones de proximidad entre sus puntos. Una forma natural de hacerlo consiste en asociar a los puntos ciertos conjuntos geométricos, como bolas de radio fijo, y estudiar el patrón de intersecciones entre ellos. Esta idea conduce a los sistemas de conjuntos y a la noción de nervio.

4.2. Sistemas finitos de conjuntos

Los complejos simpliciales aparecen de forma natural al estudiar patrones de intersección entre colecciones finitas de conjuntos. Sea

$$\mathcal{F} = \{F_1, \dots, F_m\}$$

una familia finita de subconjuntos de $\mathbb{R}^d$. La información combinatoria básica de esta familia viene dada por qué subfamilias tienen intersección común no vacía.

Este punto de vista es especialmente útil cuando los conjuntos de $\mathcal{F}$ son convexos. En dimensión baja, la convexidad impone restricciones fuertes sobre los posibles patrones de intersección. Por ejemplo, en la recta real, si tres intervalos se intersectan dos a dos, entonces necesariamente tienen intersección común. En dimensiones superiores este fenómeno se generaliza mediante resultados clásicos como el teorema de Helly [15].

Observación 4.1. El papel de la convexidad en este contexto será garantizar que las intersecciones no vacías sean topológicamente simples, como se precisa en la siguiente observación.

Observación 4.2. Todo subconjunto convexo no vacío $C \subseteq \mathbb{R}^d$ es contráctil. En efecto, fijado un punto $c_0 \in C$, se puede contraer C sobre c_0 mediante la homotopía

$$H: C \times [0, 1] \rightarrow C, \quad H(x, t) = (1 - t)x + tc_0.$$

Como C es convexo, el segmento que une x con c_0 está contenido en C , luego la aplicación anterior está bien definida. Para $t = 0$ se obtiene la identidad y para $t = 1$ se obtiene la aplicación constante $x \mapsto c_0$.

En particular, todo símplice geométrico es contráctil, ya que es la envolvente convexa de sus vértices. Además, toda intersección no vacía de conjuntos convexos vuelve a ser convexa y, por tanto, también es contráctil.

El interés de estos sistemas de conjuntos en topología computacional reside en que permiten pasar de una colección geométrica continua a un objeto combinatorio: el complejo simplicial determinado por los solapamientos de los conjuntos.

4.3. El nervio de una familia de conjuntos

Sea $\mathcal{F} = \{F_1, \dots, F_m\}$ una familia finita de conjuntos. El nervio de $\mathcal{F}$ registra qué subfamilias tienen intersección común no vacía.

Definición 4.3. El *nervio* de la familia $\mathcal{F}$, denotado por $\text{Nrv}(\mathcal{F})$, es el complejo simplicial abstracto definido por

$$\text{Nrv}(\mathcal{F}) = \left\{ \mathcal{G} \subseteq \mathcal{F} \mid \mathcal{G} \neq \emptyset, \bigcap_{G \in \mathcal{G}} G \neq \emptyset \right\}.$$

Es decir, cada conjunto F_i de la familia determina un vértice del nervio, una intersección no vacía de dos conjuntos determina una arista, una intersección común no vacía de tres conjuntos determina un 2-símplice, y así sucesivamente. La **Figura 4.1** muestra un ejemplo sencillo de esta construcción.

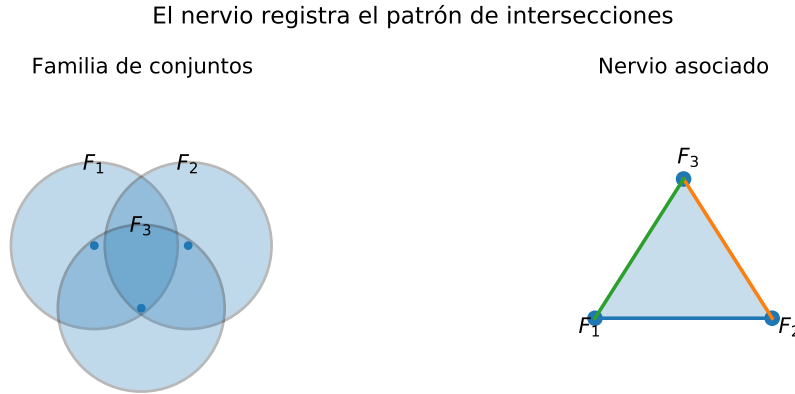


Figura 4.1.: Ejemplo de nervio asociado a una familia de conjuntos. El complejo simplicial de la derecha registra qué subfamilias tienen intersección común no vacía. Fuente: elaboración propia.

La definición anterior produce siempre un complejo simplicial abstracto. En efecto, si una subfamilia $\mathcal{G} \subseteq \mathcal{F}$ tiene intersección común no vacía, entonces cualquier subfamilia $\mathcal{H} \subseteq \mathcal{G}$ también tiene intersección común no vacía. Por tanto, toda cara de un símple de nervio vuelve a pertenecer al nervio [15].

Observación 4.4. El nervio conserva únicamente el patrón de intersecciones de los conjuntos de $\mathcal{F}$.

4.4. El teorema del nervio

La relevancia topológica del nervio viene dada por el teorema del nervio. Este resultado proporciona condiciones bajo las cuales el complejo simplicial construido a partir del patrón de intersecciones tiene el mismo tipo de homotopía que la unión de los conjuntos.

Teorema 4.5 (Teorema del nervio). *Sea $\mathcal{F}$ una familia finita de subconjuntos cerrados y convexos de un espacio euclídeo. Entonces la realización geométrica del nervio $|\text{Nrv}(\mathcal{F})|$ y la unión*

$$\bigcup_{F \in \mathcal{F}} F$$

4. De nubes de puntos a objetos topológicos

tienen el mismo tipo de homotopía.

En este contexto, decir que tienen el mismo tipo de homotopía significa que ambos espacios son homotópicamente equivalentes. En particular, sus grupos de homología son isomorfos.

La hipótesis de convexidad garantiza que todas las intersecciones finitas no vacías de elementos de $\mathcal{F}$ sean contráctiles. En efecto, la intersección de conjuntos convexos vuelve a ser convexa, y todo conjunto convexo es contráctil. Esta es la razón por la que el nervio puede llegar a conservar la información topológica de la unión.

Idea de la demostración. La demostración puede entenderse construyendo un espacio auxiliar $\mathcal{Y}$ que relaciona la unión de los conjuntos con el nervio. Este espacio auxiliar está formado por pares (x, u) , donde x es un punto de la unión

$$\bigcup_{F \in \mathcal{F}} F$$

y u es un punto de la realización geométrica del nervio. Se toman todos los pares en los que u pertenece al símple del nervio determinado por los conjuntos de $\mathcal{F}$ que contienen a x .

Por ejemplo, si un punto x pertenece a $F_1 \cap F_2 \cap F_4$, entonces los vértices correspondientes a F_1, F_2, F_4 forman un símple del nervio, y u puede ser un punto de ese símple.

A partir de este espacio auxiliar aparecen dos aplicaciones naturales:

$$\bigcup_{F \in \mathcal{F}} F \longleftarrow \mathcal{Y} \longrightarrow |\text{Nrv}(\mathcal{F})|.$$

La primera aplicación envía cada par (x, u) al punto x . Es decir, olvida en qué punto del nervio está u . La segunda aplicación envía cada par (x, u) al punto u . Es decir, olvida el punto geométrico x y conserva solo su posición en el nervio.

La razón por la que este espacio auxiliar funciona es que, al hacer cualquiera de estas dos proyecciones, lo que se pierde es topológicamente simple. Si se fija un punto x de la unión, los conjuntos de $\mathcal{F}$ que contienen a x determinan un símple del nervio, y todo símple es contráctil. Por otro lado, si se fija un punto u de la realización geométrica del nervio, este punto queda situado en alguna cara del complejo. Los vértices de esa cara corresponden a una subfamilia de conjuntos de $\mathcal{F}$ con intersección común no vacía. En el caso que estamos considerando, esa intersección es convexa y, por tanto, contráctil por la **Observación 4.2**.

De manera intuitiva, esto significa que el espacio auxiliar $\mathcal{Y}$ solo añade información contráctil al pasar hacia la unión y también al pasar hacia el nervio. Por ello, $\mathcal{Y}$ tiene el mismo tipo de homotopía que ambos espacios. Así,

$$\mathcal{Y} \simeq \bigcup_{F \in \mathcal{F}} F \quad \text{y} \quad \mathcal{Y} \simeq |\text{Nrv}(\mathcal{F})|.$$

Por transitividad de la equivalencia homotópica se obtiene

$$\bigcup_{F \in \mathcal{F}} F \simeq |\text{Nrv}(\mathcal{F})|.$$

La demostración completa requiere comprobar rigurosamente que las dos aplicaciones anteriores son equivalencias homotópicas. Hatcher demuestra una versión de este resultado

para recubrimientos abiertos cuyas intersecciones finitas no vacías son contráctiles [27]. La versión para familias finitas de conjuntos cerrados y convexos en espacios euclídeos es la que se utiliza habitualmente en topología computacional y aparece, por ejemplo, en Edelsbrunner y Harer [15].

Este resultado es uno de los puentes fundamentales entre geometría y topología computacional: permite sustituir una unión de conjuntos en un espacio euclídeo por un complejo simplicial abstracto sobre el que se pueden calcular grupos de homología isomorfos a los de la unión de conjuntos original (véase la **Observación 3.18**).

4.5. El complejo de Čech

Aplicando la construcción del nervio al caso particular de bolas euclídeas, se obtiene el complejo de Čech, una de las construcciones básicas en topología computacional [15]. Sea

$$S = \{x_1, \dots, x_n\} \subseteq \mathbb{R}^d$$

una nube finita de puntos y sea $r > 0$. Para cada $x \in S$, denotamos por

$$B_x(r) = \{y \in \mathbb{R}^d \mid \|x - y\| \leq r\}$$

la bola cerrada de centro x y radio r . Consideramos entonces la familia finita de bolas

$$\mathcal{B}_r(S) = \{B_x(r) \mid x \in S\}.$$

Definición 4.6. El *complejo de Čech* asociado a S y r , denotado por $\check{C}_r(S)$, es el nervio de la familia de bolas $\mathcal{B}_r(S)$. Es decir,

$$\check{C}_r(S) = \left\{ \emptyset \neq \sigma \subseteq S \mid \bigcap_{x \in \sigma} B_x(r) \neq \emptyset \right\}.$$

Por tanto, un subconjunto finito $\sigma = \{x_{i_0}, \dots, x_{i_k}\} \subseteq S$ genera un k -símplice en $\check{C}_r(S)$ si y solo si las bolas de radio r centradas en esos puntos tienen intersección común no vacía.

Esta construcción está directamente relacionada con el teorema del nervio. En efecto, las bolas euclídeas son convexas, y toda intersección no vacía de bolas convexas vuelve a ser convexa. En particular, dichas intersecciones son contráctiles. Por tanto, el teorema del nervio implica que el espacio topológico asociado a una realización geométrica del complejo de Čech,

$$|\check{C}_r(S)|,$$

tiene el mismo tipo de homotopía que la unión de bolas

$$U(S, r) = \bigcup_{x \in S} B_x(r).$$

Además, si $r_0 \leq r$, entonces cada bola $B_x(r_0)$ está contenida en $B_x(r)$. En consecuencia, toda intersección común que existe para radio r_0 también existe para radio r , y se obtiene la inclusión

$$\check{C}_{r_0}(S) \subseteq \check{C}_r(S).$$

Por tanto, al aumentar el radio se obtiene una familia creciente de complejos de Čech. Esta observación será fundamental más adelante para introducir la noción de filtración.

4.6. El complejo de Vietoris–Rips

Aunque el complejo de Čech posee una justificación topológica muy natural, decidir si una colección de varias bolas tiene intersección común puede resultar costoso. Una simplificación habitual consiste en comprobar únicamente distancias por pares. Esta idea conduce al complejo de Vietoris–Rips.

Definición 4.7. Sea $S = \{x_1, \dots, x_n\} \subseteq \mathbb{R}^d$ y sea $r > 0$. El *complejo de Vietoris–Rips* asociado a S y r , denotado por $VR_r(S)$, es el complejo simplicial abstracto dado por

$$VR_r(S) = \{\emptyset \neq \sigma \subseteq S \mid \text{diam}(\sigma) \leq 2r\},$$

donde

$$\text{diam}(\sigma) = \max\{\|x - y\| \mid x, y \in \sigma\}.$$

Equivalentemente, un subconjunto $\sigma \subseteq S$ pertenece a $VR_r(S)$ si todos sus pares de puntos están a distancia menor o igual que $2r$.

La ventaja principal de esta construcción es su simplicidad combinatoria: para decidir si un conjunto de vértices forma un símplexe basta comprobar distancias por pares. Por esta razón, el complejo de Vietoris–Rips aparece con frecuencia en homología persistente y en aplicaciones del análisis topológico de datos [7, 15].

Observación 4.8. En esta definición se utiliza la convención de comparar el diámetro con $2r$, coherente con la construcción de Čech mediante bolas de radio r . Con esta elección, los complejos de Čech y Vietoris–Rips tienen las mismas aristas. En otras referencias puede encontrarse la convención equivalente con umbral r .

Observación 4.9. A diferencia del complejo de Čech, el complejo de Vietoris–Rips no se obtiene, en general, como el nervio de una familia de bolas. Por tanto, no tiene una justificación directa mediante el teorema del nervio. Su relación con la topología de la unión de bolas se obtiene de forma indirecta, mediante inclusiones entre complejos de Čech y de Vietoris–Rips.

4.7. Comparación entre los complejos de Čech y de Vietoris–Rips

Los complejos de Čech y de Vietoris–Rips están estrechamente relacionados, aunque no coinciden en general. Con la convención anterior, ambos tienen las mismas aristas: dos puntos $x, y \in S$ determinan una arista en $\check{C}_r(S)$ si y solo si las bolas $B_x(r)$ y $B_y(r)$ se intersectan, lo que equivale a $\|x - y\| \leq 2r$, que es precisamente la condición para que formen una arista en $VR_r(S)$.

Además, se tiene siempre la inclusión

$$\check{C}_r(S) \subseteq VR_r(S).$$

En efecto, si una familia de bolas de radio r tiene intersección común no vacía, entonces cualquier par de centros está a distancia como máximo $2r$. Por tanto, todo símplexe de Čech es también un símplexe de Vietoris–Rips.

La inclusión contraria no es cierta en general para el mismo radio. El complejo de Vietoris–Rips puede contener s mplices determinados  nicamente por condiciones de distancia por pares, aunque las bolas correspondientes no tengan intersecci n com n. Por ejemplo, tres puntos equidistantes formando un tri ngulo equil tero de lado $2r$ determinan un 2-s mplice en $VR_r(S)$, pero sus tres bolas de radio r no tienen por qu  compartir un punto de intersecci n com n en $\check{C}_r(S)$. Esta diferencia se ilustra en la Figura 4.2.

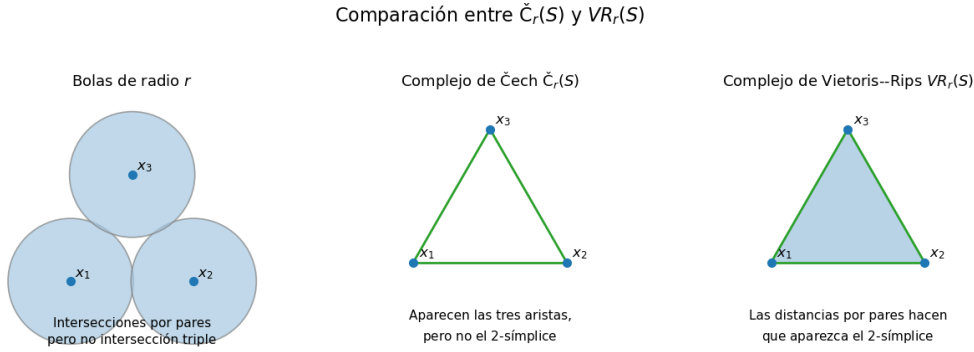


Figura 4.2.: Comparaci n entre los complejos de  ech y Vietoris–Rips para tres puntos equidistantes. Las bolas de radio r tienen intersecciones por pares, pero no intersecci n triple. Por ello, en $\check{C}_r(S)$ aparecen las tres aristas, pero no el 2-s mplice. En cambio, como en $VR_r(S)$ solo se comprueban distancias por pares, s  aparece el 2-s mplice. Fuente: elaboraci n propia.

Sin embargo, Edelsbrunner y Harer prueban que la inclusi n puede recuperarse aumentando el radio del complejo de  ech por una constante multiplicativa [15].

Proposici n 4.10 (Lema de Vietoris–Rips). *Sea S un conjunto finito de puntos en un espacio eucl deo y sea $r \geq 0$. Entonces*

$$VR_r(S) \subseteq \check{C}_{\sqrt{2}r}(S).$$

En conjunto, se obtiene la relaci n

$$\check{C}_r(S) \subseteq VR_r(S) \subseteq \check{C}_{\sqrt{2}r}(S).$$

Esta comparaci n muestra que el complejo de Vietoris–Rips puede entenderse como una construcci n combinatoriamente m s sencilla que queda intercalada entre dos complejos de  ech de radios distintos. Desde el punto de vista te rico,  ech est  m s directamente ligado a la topolog a de la uni n de bolas mediante el teorema del nervio. Desde el punto de vista pr ctico, Vietoris–Rips suele ser m s manejable, ya que solo requiere informaci n de distancias por pares.

4.8. Condiciones de muestreo y reconstrucci n

Las construcciones anteriores no garantizan por s  solas que la topolog a obtenida refleje fielmente la del objeto subyacente. Para que esto ocurra es necesario imponer hip tesis sobre la calidad del muestreo y sobre la elecci n del par metro de escala [38].

4. De nubes de puntos a objetos topológicos

Definición 4.11. Sea $X \subseteq \mathbb{R}^d$ y sea $\delta > 0$. Una muestra finita $S \subseteq X$ se dice δ -densa en X si para todo punto $x \in X$ existe un punto $s \in S$ tal que

$$\|x - s\| \leq \delta.$$

Esta definición se formula respecto a X , aunque en una aplicación real X no se observe completo. Su utilidad es teórica: permite expresar matemáticamente cuándo la muestra S está suficientemente repartida sobre el objeto que se quiere reconstruir.

La condición de δ -densidad expresa que la muestra no deja regiones grandes de X sin observar. Si S no es suficientemente densa, la unión de bolas puede no cubrir adecuadamente el espacio original, y entonces pueden perderse rasgos topológicos reales o aparecer rasgos artificiales.

La definición anterior corresponde al caso ideal en que los puntos de la muestra pertenecen al espacio X . En presencia de ruido, puede considerarse una condición análoga en la que los puntos de S se encuentran cerca de X , pero no necesariamente contenidos en él.

La elección del radio también influye en la reconstrucción, aunque esta cuestión se desarrollará con más detalle en la sección siguiente. Basta observar aquí que la topología obtenida depende también de la escala a la que se construyen los complejos.

Existen resultados de reconstrucción topológica que formalizan esta intuición bajo hipótesis geométricas adicionales sobre el espacio subyacente, por ejemplo cuando este es suficientemente regular y la muestra es densa [38]. En este trabajo no se desarrollarán esos resultados en detalle; bastará con retener la idea principal: la topología que se obtiene a partir de una nube de puntos depende simultáneamente de la densidad del muestreo, del ruido y de la escala elegida [7, 15].

4.9. Escala y filtraciones

Un rasgo esencial de las construcciones anteriores es su dependencia de un parámetro de escala r . Para valores pequeños de r , los complejos suelen tener muchas componentes conexas y pocos símlices de dimensión superior. Al aumentar r , aparecen nuevas aristas, se conectan componentes y pueden formarse ciclos. Para valores todavía mayores, algunos de esos ciclos pueden desaparecer al quedar rellenos por símlices de dimensión superior.

Por tanto, una escala demasiado pequeña puede hacer que el espacio parezca artificialmente desconectado, mientras que una escala demasiado grande puede ocultar rasgos topológicos relevantes.

En lugar de fijar un único valor de r , resulta natural considerar la familia completa de complejos obtenidos al variar la escala. En el caso de Čech, si $r_0 \leq r$, entonces

$$\check{C}_{r_0}(S) \subseteq \check{C}_r(S).$$

De manera análoga, los complejos de Vietoris–Rips también forman una familia creciente:

$$VR_{r_0}(S) \subseteq VR_r(S) \quad \text{si } r_0 \leq r.$$

Una familia anidada de complejos de la forma

$$K_{r_1} \subseteq K_{r_2} \subseteq \cdots \quad (r_1 \leq r_2 \leq \cdots)$$

recibe el nombre de *filtración*. El capítulo siguiente se dedicará a estudiar cómo evoluciona la homología a lo largo de una filtración, idea que constituye la base de la homología persistente.

5. Filtraciones y homología persistente

En el capítulo anterior se ha visto que, a partir de una nube finita de puntos, pueden construirse complejos simpliciales como los de Čech o Vietoris–Rips. Estos complejos dependen de un parámetro de escala r . Al variar r , pueden aparecer nuevos símlices y cambiar sus grupos de homología.

La homología persistente aparece precisamente para estudiar esta evolución. En lugar de escoger una única escala, se considera una familia creciente de complejos y se analiza cómo cambian sus grupos de homología a lo largo de dicha familia. Esta idea fue introducida en el contexto computacional por Edelsbrunner, Letscher y Zomorodian [16], y posteriormente desarrollada desde un punto de vista algebraico por Zomorodian y Carlsson mediante módulos de persistencia [49]. También pueden consultarse las exposiciones de Carlsson [7] y Edelsbrunner–Harer [15].

La idea central es distinguir los rasgos topológicos que aparecen solo durante un intervalo corto de escalas de aquellos que persisten durante un rango amplio. En aplicaciones al análisis de datos, los primeros suelen interpretarse como efectos locales, ruido o artefactos del muestreo, mientras que los segundos pueden reflejar estructuras geométricas más estables.

5.1. Filtraciones de complejos simpliciales

La noción básica de la homología persistente es la de filtración. Intuitivamente, una filtración describe un complejo simplicial que se construye de forma progresiva, añadiendo símlices sin eliminar los que ya estaban presentes.

Definición 5.1. Una *filtración* de un complejo simplicial K es una sucesión creciente de sub-complejos

$$K_0 \subseteq K_1 \subseteq \cdots \subseteq K_m = K.$$

En muchos casos se toma $K_0 = \emptyset$, aunque no será necesario imponerlo siempre.

La condición $K_i \subseteq K_{i+1}$ significa que cada complejo contiene todos los símlices de los anteriores. Por tanto, una filtración es una construcción ordenada del complejo final.

Ejemplo 5.2. Si $S \subseteq \mathbb{R}^d$ es una nube finita de puntos y

$$0 \leq r_0 \leq r_1 \leq \cdots \leq r_m,$$

entonces los complejos de Čech forman una filtración:

$$\check{C}_{r_0}(S) \subseteq \check{C}_{r_1}(S) \subseteq \cdots \subseteq \check{C}_{r_m}(S).$$

Del mismo modo, los complejos de Vietoris–Rips forman una filtración:

$$VR_{r_0}(S) \subseteq VR_{r_1}(S) \subseteq \cdots \subseteq VR_{r_m}(S).$$

5. Filtraciones y homología persistente

Otra forma habitual de obtener filtraciones consiste en asignar a cada s mplice un valor que indique el instante en el que aparece.

Definici n 5.3. Sea K un complejo simplicial. Una funci n

$$f: K \rightarrow \mathbb{R}$$

se dice *mon tona* si para toda cara $\tau \subseteq \sigma$ se cumple

$$f(\tau) \leq f(\sigma).$$

La monoton a garantiza que un s mplice no pueda aparecer antes que sus caras. Para cada $a \in \mathbb{R}$, se define

$$K(a) = \{\sigma \in K \mid f(\sigma) \leq a\}.$$

Entonces $K(a)$ es un subcomplejo de K , y si $a \leq b$, se tiene

$$K(a) \subseteq K(b).$$

Por tanto, los subcomplejos $K(a)$ forman una filtraci n [15].

5.2. Aplicaciones inducidas y homolog a persistente

La construcci n de la homolog a persistente se apoya en los homomorfismos inducidos en homolog a, introducidos en la **Observaci n 3.19**. En el caso de una filtraci n, estos homomorfismos aparecen de forma natural a partir de las inclusiones entre complejos.

Si

$$K_i \subseteq K_j \quad (i \leq j)$$

son dos complejos de una filtraci n, la inclusi n

$$\iota_i^j: K_i \hookrightarrow K_j$$

induce, para cada dimensi n p , un homomorfismo

$$(\iota_i^j)_*: H_p(K_i; \mathbb{F}) \rightarrow H_p(K_j; \mathbb{F}).$$

donde $\mathbb{F}$ es un cuerpo de coeficientes. De forma expl cita, si una clase de homolog a $[z] \in H_p(K_i; \mathbb{F})$ est  representada por un ciclo z , entonces

$$(\iota_i^j)_*([z]) = [(\iota_i^j)_\#(z)],$$

donde $(\iota_i^j)_\#$ es la aplicaci n inducida a nivel de cadenas. Esta definici n est  bien definida porque $(\iota_i^j)_\#$ es compatible con el operador borde y, por tanto, env a bordes en bordes [27, 15].

Geom tricamente, esto significa que una clase homol gica presente en K_i puede seguirse al pasar a un complejo posterior K_j . Al crecer el complejo, esa clase puede seguir siendo no trivial, puede anularse al convertirse en borde de una cadena de dimensi n superior, o puede identificarse con una clase que ya exist a previamente.

Por tanto, los homomorfismos inducidos permiten comparar la homolog a de distintas

escalas de una misma filtración. Esta es la idea básica de la homología persistente: estudiar qué clases sobreviven al pasar de K_i a complejos posteriores K_j .

Como se explicó en la [Sección 3.4](#), al trabajar con coeficientes en un cuerpo $\mathbb{F}$, los grupos de homología tienen estructura de espacios vectoriales. Esto permite definir los grupos de homología persistente como imágenes de aplicaciones lineales y los números de Betti persistentes como sus dimensiones.

Definición 5.4. Sea

$$K_0 \subseteq K_1 \subseteq \cdots \subseteq K_m$$

una filtración de complejos simpliciales. Para $i \leq j$, el p -ésimo grupo de homología persistente entre i y j se define como

$$H_p^{i,j} = \text{Im} \left((l_i^j)_* : H_p(K_i; \mathbb{F}) \rightarrow H_p(K_j; \mathbb{F}) \right).$$

El grupo $H_p^{i,j}$ recoge las clases de homología de dimensión p que están presentes en K_i y continúan siendo no triviales en K_j .

Definición 5.5. El p -ésimo número de Betti persistente entre i y j se define como

$$\beta_p^{i,j} = \dim_{\mathbb{F}} H_p^{i,j}.$$

Mientras que $\beta_p(K_i)$ cuenta las clases de homología presentes en una escala concreta, $\beta_p^{i,j}$ cuenta cuántas de ellas sobreviven desde K_i hasta K_j .

5.3. Nacimiento, muerte y persistencia

La definición de los grupos persistentes permite precisar qué significa que una clase homológica aparezca en una filtración y durante cuánto tiempo permanece. Para simplificar la notación, escribiremos

$$f_p^{i,j} : H_p(K_i; \mathbb{F}) \rightarrow H_p(K_j; \mathbb{F})$$

para el homomorfismo inducido por la inclusión $K_i \hookrightarrow K_j$, siempre que $i \leq j$ [15].

Para tratar también el caso $i = 0$, adoptaremos la convención

$$H_p(K_{-1}; \mathbb{F}) = 0.$$

De este modo, una clase que aparece ya en K_0 puede considerarse nacida en K_0 .

Definición 5.6. Una clase $\gamma \in H_p(K_i; \mathbb{F})$ se dice que *nace* en el complejo K_i si no procede de una clase existente en el complejo anterior, es decir, si

$$\gamma \notin \text{Im } f_p^{i-1,i}.$$

De forma intuitiva, una clase nace cuando aparece por primera vez un nuevo rasgo homológico de dimensión p . En dimensión 0, esto corresponde a la aparición de una nueva componente conexa; en dimensión 1, a la aparición de un ciclo no trivial; y en dimensión 2, a la aparición de una cavidad.

Definición 5.7. Supongamos que una clase $\gamma \in H_p(K_i; \mathbb{F})$ nace en K_i . Diremos que γ *muere* al entrar en K_j , con $j > i$, si K_j es el primer complejo de la filtración en el que la imagen de γ

5. Filtraciones y homología persistente

pasa a estar generada por clases nacidas antes que ella. Formalmente, esto significa que

$$f_p^{i,j-1}(\gamma) \notin \text{Im } f_p^{i-1,j-1},$$

pero

$$f_p^{i,j}(\gamma) \in \text{Im } f_p^{i-1,j}.$$

La primera condición dice que, justo antes de entrar en K_j , la clase γ todavía aporta información nueva respecto a las clases nacidas antes de K_j . La segunda condición dice que, al pasar a K_j , su imagen ya queda absorbida por clases más antiguas.

Geométricamente, esto puede ocurrir de dos maneras. Una clase puede morir porque se convierte en borde de una cadena de dimensión superior, como sucede cuando un ciclo de dimensión 1 se rellena con un 2-símplice. También puede morir porque pasa a identificarse con una clase nacida anteriormente. El ejemplo más claro de este segundo fenómeno aparece en dimensión 0, cuando dos componentes conexas se fusionan al añadir una arista.

Observación 5.8. En dimensión 0, cuando dos componentes conexas se fusionan, se adopta normalmente la convención conocida como *regla del anciano*: sobrevive la componente que nació antes y muere la más reciente. Esta regla permite asignar de forma coherente intervalos de vida a las componentes conexas de la filtración [15].

Supongamos ahora que la filtración viene parametrizada por valores reales

$$a_0 \leq a_1 \leq \dots \leq a_m,$$

de modo que K_i es el complejo que aparece al alcanzar el valor a_i . En una filtración de Čech o Vietoris–Rips, estos valores son radios; en una filtración definida por una función monótona, son los niveles de dicha función.

Definición 5.9. Si una clase γ nace en K_i y muere al entrar en K_j , su *persistencia* se define como

$$\text{pers}(\gamma) = a_j - a_i.$$

Si se trabaja únicamente con los índices de la filtración, se define la *persistencia en índice* como

$$j - i.$$

Si la clase no muere dentro de la filtración considerada, se dice que tiene persistencia infinita.

5.4. Un ejemplo elemental

Consideremos una filtración que construye un triángulo sólido a partir de sus vértices, aristas y cara. Este tipo de ejemplo es estándar para ilustrar el nacimiento y muerte de clases homológicas en persistencia [15].

Sean v_1, v_2, v_3 tres vértices. Tomamos la filtración

$$K_0 \subseteq K_1 \subseteq K_2 \subseteq K_3 \subseteq K_4,$$

donde

$$K_0 = \{\{v_1\}, \{v_2\}, \{v_3\}\},$$

$$K_1 = K_0 \cup \{\{v_1, v_2\}\},$$

$$K_2 = K_1 \cup \{\{v_2, v_3\}\},$$

$$K_3 = K_2 \cup \{\{v_3, v_1\}\},$$

y

$$K_4 = K_3 \cup \{\{v_1, v_2, v_3\}\}.$$

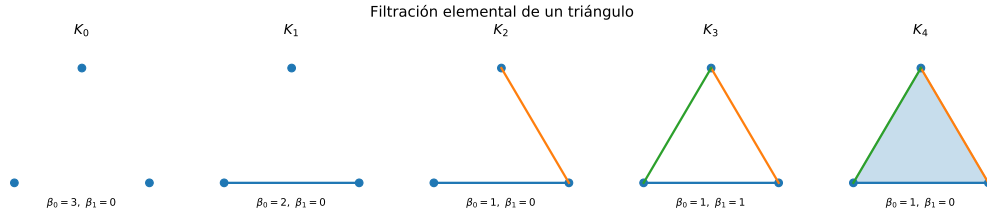


Figura 5.1.: Filtración elemental de un triángulo. Primero aparecen los tres vértices, después se añaden aristas, se forma un ciclo de dimensión 1 y finalmente el ciclo se rellena al añadir la cara triangular. Fuente: elaboración propia.

La evolución de los dos primeros números de Betti es:

i	simplex añadido	$\beta_0(K_i)$	$\beta_1(K_i)$
0	$\{v_1\}, \{v_2\}, \{v_3\}$	3	0
1	$\{v_1, v_2\}$	2	0
2	$\{v_2, v_3\}$	1	0
3	$\{v_3, v_1\}$	1	1
4	$\{v_1, v_2, v_3\}$	1	0

En dimensión 0, nacen tres clases al aparecer los tres vértices. Al añadir las dos primeras aristas, dos de esas componentes se fusionan con otra y mueren, quedando una única componente persistente.

En dimensión 1, al añadir la tercera arista se cierra un ciclo, de modo que nace una clase en $H_1(K_3)$. Al añadir la cara triangular, ese ciclo se convierte en borde y la clase muere en K_4 .

Este ejemplo muestra de forma elemental la lógica de la persistencia: las clases homológicas se registran por el intervalo de la filtración durante el que permanecen vivas.

5.5. Códigos de barras

La información de nacimiento y muerte puede representarse mediante intervalos. Si una clase nace en el índice b y muere en el índice d , se le asocia el intervalo

$$[b, d).$$

Si una clase no muere dentro de la filtración considerada, se representa mediante un intervalo de la forma

$$[b, \infty).$$

Definición 5.10. El *código de barras* de dimensión p asociado a una filtración es la colección de

intervalos que representan los tiempos de nacimiento y muerte de las clases de homología de dimensión p .

Cada barra describe la vida de una clase homológica. Una barra larga representa un rasgo que persiste durante un amplio rango de valores del parámetro; una barra corta representa un rasgo efímero. Esta representación permite visualizar de forma directa qué estructuras topológicas sobreviven a través de varias escalas [7, 15].

Retomando el ejemplo anterior del triángulo, con la convención de índices, los intervalos en dimensión 0 son

$$[0, 1), \quad [0, 2), \quad [0, \infty),$$

mientras que en dimensión 1 aparece el intervalo

$$[3, 4).$$

Si se usan valores reales a_i en lugar de índices, estos intervalos se sustituyen por

$$[a_0, a_1), \quad [a_0, a_2), \quad [a_0, \infty), \quad [a_3, a_4).$$

5.6. Diagramas de persistencia

Los códigos de barras admiten una representación equivalente mediante diagramas de persistencia. En lugar de representar una clase homológica mediante un intervalo, se representa mediante un punto en el plano nacimiento–muerte.

Definición 5.11. El *diagrama de persistencia* de dimensión p asociado a una filtración es el multiconjunto de puntos

$$(b, d) \in \overline{\mathbb{R}}^2$$

donde b es el valor de nacimiento y d el valor de muerte de una clase de homología de dimensión p .

Aquí

$$\overline{\mathbb{R}} = \mathbb{R} \cup \{-\infty, +\infty\}$$

denota la recta real extendida. Esta ampliación permite representar clases que no mueren durante la filtración considerada: si una clase nace en b y nunca muere, se representa mediante el punto

$$(b, +\infty).$$

Las clases homológicas de mayor interés suelen ser aquellas que sobreviven durante un intervalo apreciable de la filtración. Para estas clases se tiene que el valor de muerte es estrictamente mayor que el de nacimiento ($d > b$), por lo que sus correspondientes puntos se sitúan siempre por encima de la diagonal

$$\Delta = \{(x, x) \mid x \in \mathbb{R}\}.$$

La diferencia $d - b$ mide la persistencia de la clase correspondiente. Por tanto, los puntos cercanos a la diagonal representan clases de vida corta, mientras que los puntos alejados de la diagonal representan clases más persistentes.

El diagrama de persistencia no es únicamente una visualización. Bajo las hipótesis finitas habituales (que el complejo simplicial total esté formado por un número finito de símlices y que la filtración conste de un número finito de pasos), contiene la misma información que el código de barras: cada punto (b, d) corresponde al intervalo $[b, d)$. Por ello, el diagrama resume de forma exacta la información de nacimiento y muerte de las clases homológicas de una filtración [15, 7].

Observación 5.12. Los números de Betti persistentes pueden leerse directamente en el diagrama. En efecto, $\beta_p^{i,j}$ cuenta las clases que han nacido en K_i o antes y que siguen vivas en K_j . En el plano nacimiento–muerte, esto corresponde a contar los puntos (b, d) tales que

$$b \leq a_i \quad \text{y} \quad d > a_j.$$

Es decir, los puntos situados en el cuadrante superior izquierdo determinado por (a_i, a_j) , contando multiplicidades.

La **Figura 5.2** muestra el código de barras y el diagrama de persistencia correspondientes a la filtración del triángulo descrita anteriormente.

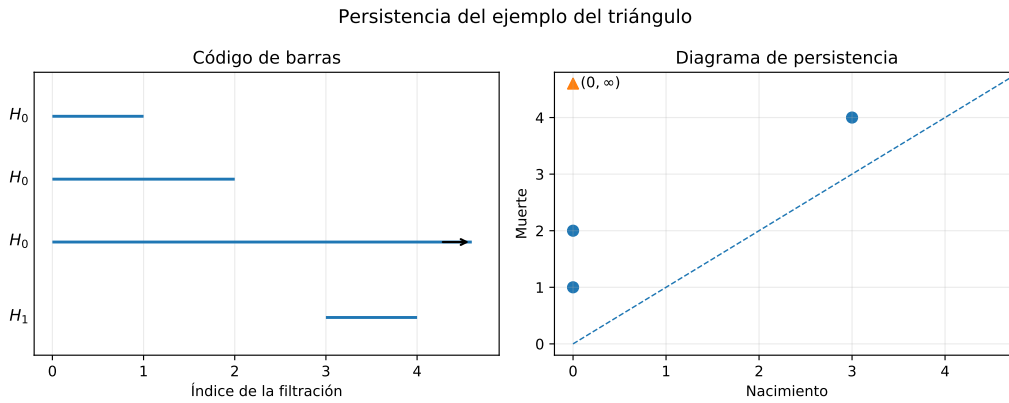


Figura 5.2.: Código de barras y diagrama de persistencia asociados a la filtración del triángulo. En dimensión 0 aparecen tres componentes, dos de las cuales mueren al conectarse con la componente persistente. En dimensión 1, el ciclo nace al cerrar el triángulo y muere al añadir la cara. Fuente: elaboración propia.

5.7. Paisajes de persistencia

Los diagramas de persistencia contienen la información de nacimiento y muerte de las clases homológicas, pero no son vectores ni funciones. Esto puede ser un inconveniente si después se quieren aplicar herramientas numéricas habituales, como calcular medias, normas o utilizar los diagramas como entrada de un algoritmo.

Una forma de resolver este problema es transformar el diagrama en una familia de funciones reales. Esta construcción se conoce como *paisaje de persistencia* (*persistence landscape*) y fue introducida por Bubenik [6]. La idea es asociar a cada punto del diagrama una función triangular sencilla y, después, combinar todas esas funciones siguiendo el orden de sus alturas.

5. Filtraciones y homología persistente

Sea (b, d) un punto de un diagrama de persistencia, con $d > b$. A este punto se le asocia la función

$$\Lambda_{(b,d)}(u) = \max\{0, \min(u - b, d - u)\}.$$

Esta función tiene forma de tienda: vale cero fuera del intervalo $[b, d]$, crece linealmente desde b hasta el punto medio del intervalo, y decrece linealmente desde ese punto medio hasta d . Su altura máxima es

$$\frac{d - b}{2},$$

por lo que una clase más persistente produce una tienda más ancha y más alta.

Si el diagrama contiene puntos

$$D = \{(b_i, d_i)\}_{i=1}^N,$$

se obtiene una función triangular $\Lambda_{(b_i, d_i)}$ para cada uno de ellos. Para cada valor fijo de u , se consideran las alturas

$$\Lambda_{(b_1, d_1)}(u), \dots, \Lambda_{(b_N, d_N)}(u).$$

Entonces $\lambda_1(u)$ se define como la mayor de esas alturas, $\lambda_2(u)$ como la segunda mayor, y así sucesivamente. Formalmente,

$$\lambda_k(u) = \text{kmax}_{i=1, \dots, N} \Lambda_{(b_i, d_i)}(u),$$

donde kmax denota el k -ésimo mayor valor.

De este modo, el paisaje de persistencia es la sucesión de funciones

$$\lambda_1, \lambda_2, \lambda_3, \dots$$

asociada al diagrama. En un diagrama finito solo hay un número finito de niveles no nulos. Intuitivamente, λ_1 recoge el contorno superior de todas las tiendas, λ_2 recoge el segundo contorno, y así sucesivamente.

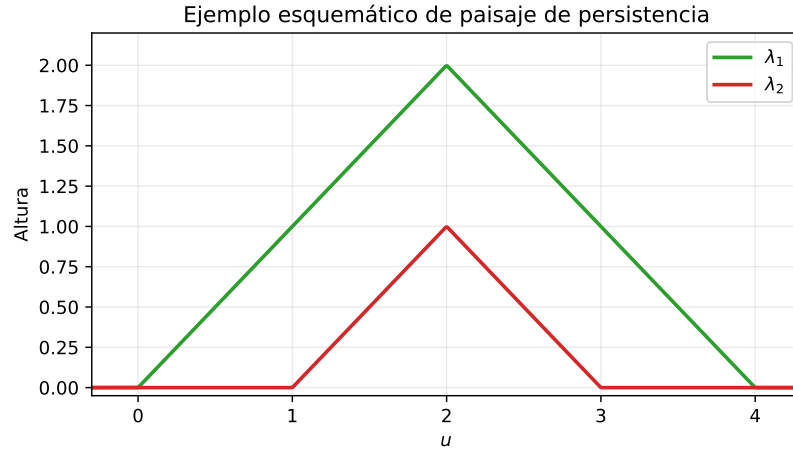


Figura 5.3.: Ejemplo esquemático de un paisaje de persistencia. Cada intervalo finito genera una función triangular, y las funciones $\lambda_1, \lambda_2, \dots$ se obtienen ordenando las alturas para cada valor de u . Fuente: elaboración propia.

Esta representación permite resumir el diagrama mediante cantidades numéricas. Por ejemplo, puede considerarse la norma

$$\|\lambda\|_2 = \left(\sum_{k \geq 1} \int_{\mathbb{R}} \lambda_k(u)^2 du \right)^{1/2}.$$

Esta norma mide el tamaño total del paisaje. Tiene en cuenta tanto la altura de las tiendas como el intervalo en el que son distintas de cero. Por eso, aumenta cuando hay clases con mayor persistencia o cuando aparecen varias clases con una contribución apreciable.

Observación 5.13. Las clases con persistencia infinita, representadas en el diagrama mediante puntos de la forma $(b, +\infty)$, no producen directamente una función triangular finita. Por ello, en aplicaciones computacionales suelen eliminarse o tratarse por separado antes de construir los paisajes de persistencia.

En la parte computacional de este trabajo se utilizará esta construcción sobre ventanas temporales de datos financieros. Para distinguir la variable interna del paisaje de la fecha de cálculo, se denotará por

$$\lambda_t^{(p)}$$

el paisaje de persistencia obtenido en la fecha t para la dimensión homológica p . En particular, se usará la norma

$$\|\lambda_t^{(p)}\|_2$$

como resumen numérico de la información persistente de esa ventana.

5.8. Estabilidad e interpretación

Una propiedad fundamental de la homología persistente es su estabilidad: pequeñas perturbaciones en los datos o en la función que define la filtración producen pequeñas perturbaciones en los diagramas de persistencia. Este resultado se formaliza mediante distancias entre diagramas, como la distancia bottleneck. Esta distancia mide el coste mínimo de emparejar los puntos de un diagrama con los puntos de otro, permitiendo emparejar puntos cercanos a la diagonal con la propia diagonal. En este sentido, los rasgos de baja persistencia pueden desaparecer bajo pequeñas perturbaciones sin alterar de forma significativa el diagrama.

La estabilidad justifica que la persistencia sea adecuada para datos reales, donde siempre existen ruido, errores de medición o pequeñas variaciones de la muestra. Los puntos cercanos a la diagonal representan clases de vida corta y pueden ser sensibles al ruido, mientras que los puntos alejados de la diagonal representan clases más persistentes y, por tanto, potencialmente más robustas [10, 14].

Además, esta estabilidad también se traslada a los paisajes de persistencia: si el diagrama cambia poco, las funciones del paisaje cambian de forma controlada. Por ello, los paisajes permiten trabajar con una representación numérica de la información persistente manteniendo la idea de robustez frente a pequeñas perturbaciones [6].

Parte II.

Aplicación computacional a finanzas

6. Motivación y objetivos computacionales

En la primera parte de este trabajo se han presentado los fundamentos matemáticos del análisis topológico de datos, especialmente los relacionados con complejos simpliciales, homología, filtraciones y homología persistente. A partir de esta base, la segunda parte de la memoria se centra en llevar estas ideas a un problema aplicado: el análisis de datos financieros y la construcción experimental de carteras.

El interés de aplicar TDA a finanzas aparece porque los datos de mercado suelen ser de alta dimensión, cambian con el tiempo y contienen relaciones complejas entre activos. En una cartera con muchos activos, cada activo puede verse como una serie temporal que evoluciona de forma relacionada con las demás. Medidas clásicas como la media, la volatilidad o la correlación resumen parte de esta información, pero pueden perder aspectos relacionados con la estructura global de los datos, la presencia de grupos o los cambios de régimen.

El análisis topológico de datos ofrece una forma distinta de estudiar este tipo de información. Su idea principal es representar los datos como nubes de puntos y analizar su forma mediante herramientas topológicas. En el contexto financiero, esta perspectiva permite estudiar relaciones entre activos desde un punto de vista geométrico y estructural. La literatura reciente ha utilizado TDA en problemas como detección de crisis, análisis de riesgo, selección de activos y construcción de carteras [20, 23, 22, 24]. Estos trabajos sirven como motivación para estudiar si las herramientas topológicas pueden aportar información complementaria a las medidas financieras tradicionales.

Dada esta motivación, se plantean los siguientes objetivos para la parte computacional del trabajo.

6.1. Objetivo general

El objetivo general de la parte computacional es desarrollar una librería en Python para aplicar técnicas de análisis topológico de datos a series financieras y evaluar su utilidad en un problema experimental de construcción de carteras.

El producto final esperado es una herramienta software capaz de integrar la preparación de los datos, la aplicación de Mapper [42] y homología persistente, la generación de pesos de cartera y la evaluación de las estrategias mediante un protocolo de backtesting reproducible.

6.2. Objetivos específicos

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos. Estos objetivos recogen las principales características que debe tener la solución desarrollada y los resultados que se espera obtener con ella.

6.2.1. Disponer de una herramienta modular y reutilizable para TDA aplicado a finanzas

El primer objetivo es disponer de una librería Python que integre las principales etapas necesarias para aplicar análisis topológico de datos a series financieras.

La herramienta debe permitir trabajar de forma ordenada desde los datos de entrada hasta la obtención de resultados experimentales, separando las distintas partes del proceso para facilitar su uso, mantenimiento y posible ampliación.

6.2.2. Representar las relaciones entre activos mediante herramientas topológicas

El segundo objetivo es obtener una representación de las relaciones entre activos financieros mediante técnicas de TDA.

Para ello, los activos se representan a partir de sus retornos en una ventana temporal y se aplican herramientas como Mapper y homología persistente. Mapper permite obtener una representación en forma de grafo, mientras que la homología persistente aporta información sobre la estructura topológica de ventanas recientes de mercado.

6.2.3. Obtener decisiones de cartera a partir de la información topológica

El tercer objetivo es conseguir que la información obtenida mediante TDA pueda traducirse en una decisión financiera evaluable.

En particular, el grafo generado por Mapper debe poder transformarse en pesos de cartera, de forma que la estructura topológica detectada pueda utilizarse dentro de una estrategia y evaluarse mediante backtesting.

6.2.4. Comprobar experimentalmente el funcionamiento del método sobre datos financieros reales

El cuarto objetivo es comprobar que la solución desarrollada puede aplicarse de forma adecuada sobre datos financieros reales.

Para ello, se utilizan las 49 carteras industriales de Kenneth French [19] y un conjunto de activos del S&P 500 obtenido a partir de CRSP/WRDS [8]. Estos dos conjuntos permiten evaluar el método en escenarios diferentes: un universo sectorial agregado y un universo formado por acciones individuales.

6.2.5. Evaluar la utilidad de la información topológica frente a una estrategia de referencia

El último objetivo es estudiar si las estrategias basadas en Mapper y homología persistente ofrecen un comportamiento competitivo frente a una referencia financiera clásica.

La comparación se realiza mediante un backtesting causal entre Mapper, Mapper con control de régimen basado en homología persistente y una cartera equiponderada. La evaluación considera métricas de rentabilidad y riesgo, como retorno anualizado, volatilidad, Sharpe, Sortino y máximo drawdown [13].

6.3. Requisitos del sistema

Además de los objetivos anteriores, la librería debe cumplir una serie de requisitos. Los requisitos funcionales describen qué debe poder hacer el sistema. Los requisitos no funcionales recogen propiedades de calidad, instalación, uso y reproducibilidad.

6.3.1. Requisitos funcionales

Los requisitos funcionales de la librería son los siguientes:

- Permitir trabajar con datos financieros en forma de matrices temporales de precios o retornos.
- Convertir retornos en índices de precios cuando sea necesario para ejecutar el backtesting.
- Incluir utilidades de preparación para los conjuntos de datos utilizados en el trabajo, especialmente KF49 y S&P 500/CRSP.
- Transformar ventanas temporales de retornos en nubes de puntos donde cada punto represente un activo financiero.
- Permitir aplicar Mapper a estas nubes de puntos y configurar sus principales parámetros, como el número de cubos, el solapamiento del recubrimiento y el método de clustering local.
- Calcular diagramas de persistencia a partir de matrices de distancia entre activos.
- Obtener características topológicas derivadas de la homología persistente, incluyendo una señal de régimen basada en paisajes de persistencia.
- Generar pesos de cartera a partir del grafo Mapper.
- Realizar backtests causales con ventana histórica, rebalanceo periódico y costes de transacción.
- Calcular métricas de evaluación como retorno anualizado, volatilidad anualizada, Sharpe, Sortino, máximo drawdown, NAV final y turnover.

6.3.2. Requisitos no funcionales

Los requisitos no funcionales de la librería son los siguientes:

- La librería debe ser independiente del sistema operativo, siempre que el entorno utilizado sea compatible con Python y con las dependencias del proyecto.
- El proyecto debe poder instalarse como paquete Python desde el repositorio y también desde el repositorio público de librerías PyPI¹.
- La estructura del código debe ser modular, separando preprocesamiento de datos, análisis topológico, construcción de carteras, backtesting y experimentación.

¹<https://pypi.org/project/tda-finance-mapper/>

6. Motivación y objetivos computacionales

- El uso básico debe estar orientado a un usuario que conozca los parámetros principales del experimento, aunque pueda ejecutar la librería sin conocer todo el código interno.
- El código debe ser legible y mantenible.
- La librería debe incluir documentación de instalación, explicación de parámetros principales y ejemplos de uso.
- Los experimentos deben ser reproducibles mediante configuraciones explícitas, semillas fijas cuando proceda, exportación de resultados y registro de versiones de dependencias.
- La gestión de datos debe respetar las restricciones de acceso y licencia, especialmente en el caso de CRSP/WRDS.

6.4. Alcance del trabajo

El trabajo tiene carácter académico y experimental. Su finalidad es estudiar si las técnicas topológicas introducidas en la parte matemática pueden integrarse en una librería funcional y aplicarse a datos financieros reales.

Desde el punto de vista informático, la aportación principal consiste en desarrollar una herramienta modular, instalable, documentada y reproducible. Desde el punto de vista experimental, el trabajo busca comprobar si la información topológica obtenida mediante Mapper y homología persistente puede compararse de forma razonable con una estrategia clásica de referencia.

7. Planificación y presupuesto

En este capítulo se resume la planificación temporal del trabajo y se realiza una estimación del esfuerzo dedicado. Aunque el capítulo se incluye dentro de la parte computacional de la memoria, la planificación recoge el trabajo completo realizado en el TFG: estudio teórico, revisión bibliográfica, implementación, experimentación y redacción.

7.1. Descomposición de tareas

El trabajo se ha dividido en varias tareas principales. En primer lugar, fue necesario estudiar los fundamentos matemáticos del análisis topológico de datos, especialmente complejos simpliciales, filtraciones, homología persistente, paisajes de persistencia y Mapper. A continuación, se revisaron aplicaciones de estas técnicas a datos financieros, con el objetivo de conectar la teoría con el problema de detección de regímenes y construcción de carteras.

Una vez fijado el enfoque, se desarrolló la parte informática del proyecto. Esta incluyó la preparación de los datos, la implementación del pipeline Mapper, la incorporación de herramientas de homología persistente, el diseño del motor de backtesting y la ejecución de experimentos comparativos. Finalmente, se trabajó en la organización de la librería, la documentación, el empaquetado y la redacción de la memoria.

La [Tabla 7.1](#) muestra una estimación de la dedicación por tareas.

7.2. Estimación económica

Además del tiempo dedicado, se realiza una estimación económica orientativa del coste del proyecto. Esta estimación permite valorar el esfuerzo realizado y los recursos utilizados durante el desarrollo.

Para valorar el trabajo personal se considera un coste/hora de 15 euros. Como referencia, el salario mínimo interprofesional anual en España para 2026 es de 17094 euros brutos [5]. Si se divide esa cantidad entre 52 semanas y 40 horas semanales, se obtiene una referencia aproximada:

$$c_{\text{SMI,h}} = \frac{17094}{52 \cdot 40} \approx 8.22 \text{ euros/hora.}$$

La tarifa de 15 euros/hora utilizada en este presupuesto se sitúa por encima de esa referencia, ya que el proyecto incluye tareas técnicas especializadas relacionadas con programación, análisis topológico de datos, tratamiento de series financieras y validación experimental.

Si se denota por H el número total de horas estimadas y por c_h el coste por hora, el coste de personal se calcula como

$$C_{\text{personal}} = H \cdot c_h.$$

En este caso, tomando $H = 740$ horas y $c_h = 15$ euros/hora, se obtiene:

$$C_{\text{personal}} = 740 \cdot 15 = 11100 \text{ euros.}$$

Tarea	Descripción	Horas
Estudio teórico de TDA	Revisión de complejos simpliciales, filtraciones, homología persistente, paisajes de persistencia y Mapper.	140
Revisión de aplicaciones financieras	Lectura de trabajos sobre TDA en finanzas, detección de crisis, regímenes de mercado y construcción de carteras.	55
Preprocesamiento de datos	Preparación de las 49 carteras industriales y de los datos de S&P 500/CRSP, incluyendo limpieza, transformación y alineación temporal.	40
Implementación de Mapper	Construcción del pipeline Mapper, funciones filtro, recubrimiento, clustering local y generación de pesos de cartera.	85
Implementación de homología persistente	Cálculo de diagramas de persistencia, extracción de características topológicas y señal de régimen.	55
Motor de backtesting	Implementación de validación temporal, rebalanceo, costes de transacción, métricas financieras y benchmark equiponderado.	70
Experimentos y comparación de modelos	Ejecución de pruebas intermedias, selección de configuraciones y comparación final entre Mapper, Mapper con homología persistente y cartera equiponderada.	70
Empaquetado y documentación	Organización como librería Python, configuración del paquete, documentación con Sphinx, publicación en PyPI y control de calidad.	45
Redacción de la memoria	Escritura, revisión y preparación final de los capítulos, tablas, figuras y apéndices.	180
Total		740

Tabla 7.1.: Estimación de dedicación por tareas principales.

También se asigna una parte del coste del equipo informático utilizado. El ordenador personal empleado durante el desarrollo tiene un coste aproximado de adquisición de 1000 euros. Suponiendo una vida útil de 5 años, es decir, 60 meses, y considerando un periodo de uso asociado al TFG de aproximadamente 10.5 meses, el coste amortizado imputable al proyecto es:

$$C_{\text{equipo}} = 1000 \cdot \frac{10.5}{60} = 175 \text{ euros.}$$

La [Tabla 7.2](#) recoge una estimación simplificada del presupuesto.

Concepto	Criterio aplicado	Coste estimado
Trabajo realizado	740 h a 15 euros/h	11100 euros
Equipo informático	Amortización de ordenador personal	175 euros
Software utilizado	Librerías libres y herramientas gratuitas	0 euros
Datos Kenneth French	Datos públicos	0 euros
Datos CRSP/WRDS	Acceso institucional durante el TFG	0 euros directos
Total estimado imputado		11275 euros

Tabla 7.2.: Estimación económica orientativa del proyecto.

El coste de CRSP/WRDS requiere una aclaración adicional. En este trabajo se ha utilizado acceso institucional, sin embargo, fuera de un entorno universitario o institucional, el acceso a estos datos requiere una suscripción o licencia específica [48, 9].

El coste total indicado debe interpretarse como una valoración aproximada del trabajo y de los recursos utilizados. En la práctica, el desarrollo se ha realizado con equipo personal, software libre y acceso institucional a las fuentes de datos necesarias.

7.3. Planificación temporal

El trabajo se desarrolló a lo largo de varios meses, combinando fases de estudio, implementación, experimentación y redacción. Algunas tareas se solaparon en el tiempo. Por ejemplo, la redacción de la memoria se fue realizando de forma progresiva conforme se cerraban partes del trabajo.

La [Figura 7.1](#) muestra el diagrama de Gantt resumido del proyecto. En él se observa que la primera parte estuvo más centrada en el estudio teórico y la revisión bibliográfica, mientras que la parte central del trabajo se dedicó principalmente a la implementación y experimentación. Finalmente, las últimas fases se centraron en la librería, la documentación y la revisión final de la memoria.

En conjunto, la planificación refleja que el trabajo combinó una fase inicial de estudio, una fase central de desarrollo y experimentación, y una fase final de documentación, empaquetado y revisión de la memoria.

7. Planificación y presupuesto

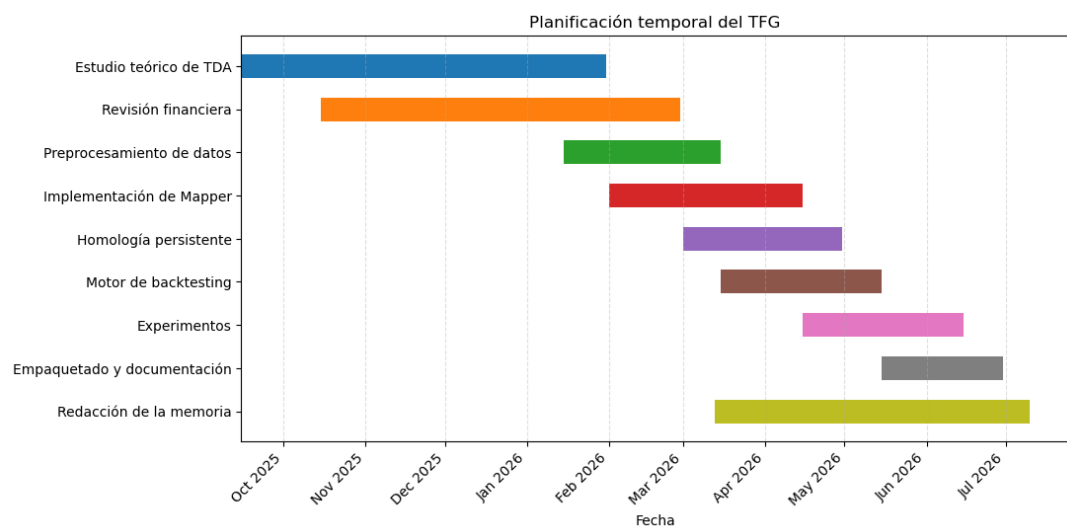


Figura 7.1.: Diagrama de Gantt del desarrollo del trabajo.

8. Implementación de la librería

En este capítulo se describe cómo se ha organizado e implementado la parte informática del trabajo. El objetivo de esta parte ha sido convertir las ideas desarrolladas en los capítulos anteriores en una librería de Python que permita aplicar TDA a datos financieros y ejecutar los experimentos de forma reproducible.

La implementación se encuentra disponible en el repositorio del proyecto bajo licencia MIT [12]. Además, se ha preparado como paquete instalable y se ha publicado en PyPI con el nombre `tda-finance-mapper` [11]. De esta forma, la librería puede instalarse desde el repositorio durante el desarrollo o directamente mediante `pip` para un uso más sencillo.

8.1. Objetivos de la implementación

La implementación se diseñó pensando en el flujo completo del trabajo. Era necesario repetir los experimentos con distintos conjuntos de datos, ventanas temporales y configuraciones de parámetros. Por este motivo, el proyecto se organizó como una librería Python dividida en módulos. Cada módulo recoge una parte concreta del proceso: preparación de datos, construcción topológica, generación de pesos, backtesting y análisis de resultados. Esta separación facilita entender el código, modificar una parte del pipeline y reutilizar funciones en nuevos experimentos.

También se buscó distinguir la aportación propia del trabajo respecto a las herramientas externas utilizadas. Algunas librerías proporcionan algoritmos ya existentes, como el cálculo de diagramas de persistencia, la reducción de dimensión o el clustering. La aportación del proyecto está en integrar estas herramientas en un flujo financiero causal, desde los datos de entrada hasta la comparación final de estrategias.

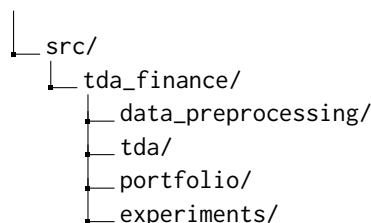
8.2. Estructura del repositorio

La estructura final del proyecto separa el código fuente, la documentación, los scripts auxiliares, los datos locales y los resultados experimentales.

De forma esquemática, la estructura principal del repositorio es la siguiente:

```
tda-finance-mapper/  
├── README.md  
├── LICENSE  
├── pyproject.toml  
├── requirements.txt  
├── requirements-lock.txt  
├── data/  
│   └── README.md  
├── docs/  
│   └── source/  
└── scripts/
```

8. Implementación de la librería



El directorio `src/tda_finance` contiene el paquete principal de Python. Dentro de él se agrupan los módulos reutilizables de la librería. El directorio `docs/source` contiene los ficheros fuente de la documentación generada con Sphinx. Por su parte, `scripts` contiene scripts auxiliares de preparación de datos que quedan fuera de la API principal de la librería, aunque resultan útiles para reproducir el flujo experimental.

La carpeta `data` se mantiene fuera del paquete y los datos financieros utilizados en los experimentos quedan fuera del repositorio. Esta decisión es importante porque algunos datos, especialmente los procedentes de CRSP/WRDS, están sujetos a restricciones de acceso y licencia. Por ello, el repositorio incluye únicamente un fichero informativo que explica qué datos deben generarse u obtenerse localmente.

8.3. Módulos principales

La librería se divide en cuatro bloques principales: preprocesamiento de datos, análisis topológico, construcción y evaluación de carteras, y ejecución de experimentos.

8.3.1. Preprocesamiento de datos

El módulo `data_preprocessing` contiene las funciones encargadas de transformar los datos financieros al formato requerido por el resto del pipeline. En el caso de las 49 carteras industriales de Kenneth French, el código permite descargar y preparar los datos mensuales, generando las series necesarias para los experimentos. También se preparan los factores de Fama-French utilizados posteriormente para el cálculo de algunas métricas financieras.

Por motivos de licencia, los datos procedentes de CRSP/WRDS quedan fuera del repositorio. El proyecto proporciona el código de preparación de los datos en matrices mensuales de precios o retornos, pero el usuario debe disponer de acceso propio a dichos datos para reproducir los experimentos basados en S&P 500.

Esta fase es necesaria porque las fuentes financieras utilizadas no tienen el mismo formato de entrada. Algunos conjuntos están expresados como precios, otros como retornos, y en muchos casos existen diferencias en fechas disponibles, activos presentes o valores ausentes.

8.3.2. Módulo topológico

El módulo `tda` agrupa las funciones relacionadas con el análisis topológico de datos. Incluye la construcción del grafo Mapper, el cálculo de diagramas de persistencia, la extracción de características topológicas y la detección de cambios de régimen mediante paisajes de persistencia.

La implementación de Mapper parte de una matriz de precios y construye, para cada ventana temporal, una representación de los activos mediante sus retornos recientes. En esta representación, cada activo se interpreta como un punto descrito por su comportamiento durante la ventana de observación. A continuación, se aplica una función filtro basada en

reducción de dimensión, un recubrimiento del espacio filtrado y clustering local. El resultado es un grafo que resume relaciones entre grupos de activos con comportamiento similar, siguiendo la idea general del algoritmo Mapper [42].

En la parte de homología persistente, el módulo calcula diagramas de persistencia a partir de matrices de distancia entre activos. En particular, se utilizan distancias derivadas de correlaciones, lo que permite construir complejos de Vietoris–Rips a partir de información por pares. Este tipo de complejos es habitual en TDA porque permite estudiar la estructura de una nube de puntos cuando se dispone de una matriz de distancias [7, 15]. Posteriormente, esta información topológica se transforma en magnitudes numéricas que pueden incorporarse al pipeline financiero.

En este trabajo, la homología persistente se utiliza como mecanismo auxiliar para detectar cambios de régimen y modular la estrategia basada en Mapper.

8.3.3. Módulo de cartera

El módulo portfolio contiene el motor de backtesting y las funciones de evaluación financiera. Su función es aplicar de forma secuencial una regla de construcción de pesos y calcular la evolución de la cartera a lo largo del tiempo.

El diseño del backtest sigue una validación temporal estricta. En cada fecha de rebalanceo, la estrategia solo utiliza información disponible hasta ese momento. Los pesos obtenidos a partir de la ventana histórica se aplican al periodo posterior, evitando el uso de información futura. Además, el motor incorpora costes de transacción proporcionales al turnover y permite comparar las estrategias topológicas con una cartera equiponderada bajo la misma frecuencia de rebalanceo y la misma convención de costes.

Gracias a la separación entre este módulo y el resto, sería posible sustituir en el futuro la regla de construcción de pesos, añadir nuevos benchmarks o incorporar otras métricas de evaluación sin modificar la construcción topológica.

8.3.4. Módulo de experimentos

El módulo experiments contiene los scripts utilizados para ejecutar las comparaciones finales del trabajo. En estos scripts se fijan los conjuntos de datos, los periodos de evaluación, los parámetros de Mapper, la activación del control mediante homología persistente y las rutas de salida.

El experimento principal compara tres estrategias: Mapper, Mapper con control de régimen basado en homología persistente, y cartera equiponderada. Los resultados se guardan en ficheros CSV para facilitar su análisis posterior y su inclusión en la memoria.

8.4. Librerías y herramientas utilizadas

La implementación se apoya en librerías científicas de Python y en herramientas específicas de topología computacional. Las versiones utilizadas se recogen en el fichero `requirements-lock.txt`, lo que permite reconstruir el entorno de trabajo con mayor precisión. En esta sección se indican las dependencias principales, la versión empleada y el motivo de su elección.

- **NumPy 2.4.6.** Se emplea para el cálculo numérico con arrays y operaciones vectorizadas. Se ha elegido porque es una de las librerías básicas del ecosistema científico de Python

y sirve como base para muchas de las demás dependencias utilizadas en el proyecto [26].

- **pandas 3.0.3.** Se utiliza para trabajar con tablas, fechas y series temporales financieras. Es adecuada para este proyecto porque facilita la alineación de fechas, el cálculo de retornos, la selección de ventanas temporales y la exportación de resultados en formato CSV [35].
- **scikit-learn 1.9.0.** Se utiliza en varias etapas del pipeline, especialmente para PCA, DBSCAN y clustering jerárquico aglomerativo. Se ha seleccionado porque proporciona implementaciones estables y ampliamente utilizadas de estos métodos de aprendizaje automático [39].
- **umap-learn 0.5.12.** Se emplea para la reducción de dimensión no lineal dentro de la construcción del *lens* de Mapper. Se ha elegido porque permite obtener representaciones de baja dimensión que pueden captar relaciones no lineales entre los activos [34].
- **knapper 2.1.0.** Se utiliza como base para construir el grafo Mapper. Esta librería se ha seleccionado porque está orientada específicamente a la implementación del algoritmo Mapper en Python y permite combinar funciones filtro, recubrimientos y clustering local [47].
- **ripser 0.6.15.** Se emplea para calcular diagramas de persistencia. Se ha elegido porque está especializada en homología persistente y permite obtener de forma eficiente los diagramas necesarios para la señal topológica de régimen [1].
- **GUDHI 3.12.0.** Se utiliza para complementar la parte de topología computacional, especialmente en representaciones asociadas a homología persistente. Se ha seleccionado porque es una librería consolidada en TDA y proporciona herramientas útiles para trabajar con objetos topológicos y sus vectorizaciones [31].
- **NetworkX 3.6.1.** Se emplea para manipular el grafo Mapper una vez construido. Se ha elegido porque ofrece una forma sencilla de trabajar con grafos, componentes conexas, nodos y relaciones entre nodos [25].
- **Sphinx 9.1.0.** Se utiliza para generar la documentación HTML de la librería. Se ha seleccionado porque es una herramienta habitual en proyectos Python y permite construir documentación de usuario y documentación de API a partir de ficheros fuente [44].
- **sphinx-rtd-theme 3.1.0.** Se utiliza como tema visual para la documentación generada con Sphinx. Se ha elegido porque proporciona un formato claro y habitual en documentación técnica de proyectos Python [44].
- **flake8 7.3.0.** Se emplea para revisar el estilo del código y detectar errores sencillos. Se ha seleccionado porque permite comprobar de forma automática aspectos relacionados con formato, convenciones de escritura y posibles problemas básicos en el código [40].
- **pyright 1.1.410.** Se utiliza para análisis estático de tipos durante el desarrollo. Se ha elegido porque ayuda a detectar incoherencias en el uso de funciones, argumentos y valores devueltos antes de ejecutar el código [36].

- **build 1.5.0.** Se emplea para construir la distribución del paquete Python. Se ha seleccionado porque forma parte del flujo recomendado para generar los ficheros necesarios antes de publicar un paquete [41].
- **twine 6.2.0.** Se utiliza para comprobar y subir la distribución del paquete a PyPI. Se ha elegido porque es la herramienta habitual para validar y publicar paquetes Python [41].

La **Tabla 8.1** resume las librerías y herramientas principales utilizadas en la implementación.

Herramienta	Versión	Uso principal	Referencia
NumPy	2.4.6	Cálculo numérico	[26]
pandas	3.0.3	Series temporales y tablas	[35]
scikit-learn	1.9.0	PCA y clustering	[39]
umap-learn	0.5.12	Reducción de dimensión	[34]
kmapper	2.1.0	Construcción de Mapper	[47]
ripser	0.6.15	Diagramas de persistencia	[1]
GUDHI	3.12.0	Topología computacional	[31]
NetworkX	3.6.1	Manipulación de grafos	[25]
Sphinx	9.1.0	Documentación HTML	[44]
sphinx-rtd-theme	3.1.0	Tema de documentación	[44]
flake8	7.3.0	Revisión de estilo	[40]
pyright	1.1.410	Análisis estático de tipos	[36]
build	1.5.0	Construcción del paquete	[41]
twine	6.2.0	Comprobación y subida a PyPI	[41]

Tabla 8.1.: Principales librerías y herramientas utilizadas en la implementación.

8.5. Empaquetado e instalación

El proyecto se ha preparado como paquete Python instalable. Para ello se utiliza el fichero `pyproject.toml` [41]. El paquete se distribuye con el nombre `tda-finance-mapper`, mientras que el módulo importable en Python se denomina `tda_finance`. Por tanto, la instalación desde PyPI se realiza con:

```
pip install tda-finance-mapper
```

Una vez instalado, las funciones principales pueden importarse desde Python. Por ejemplo:

```
from tda_finance.tda.mapper_clustering import MapperParams
from tda_finance.portfolio.backtest_engine import backtest_tda
```

Durante el desarrollo se utilizó la instalación en modo editable. Este modo permite trabajar sobre el código fuente del repositorio y probar los cambios sin reinstalar el paquete cada vez. Desde la raíz del repositorio se ejecuta:

```
pip install -e .
```

8. Implementación de la librería

También se incluye un fichero `requirements.txt` con las dependencias del proyecto. Para instalar el entorno a partir de ese fichero puede usarse:

```
pip install -r requirements.txt
```

La publicación en PyPI facilita que otros usuarios puedan instalar la librería sin clonar el repositorio completo. Para reproducir exactamente los experimentos de la memoria, además de instalar el paquete, es necesario disponer de los datos correspondientes y ejecutar los scripts incluidos en el repositorio.

8.6. Documentación

La documentación de la librería se ha generado con Sphinx [44]. El objetivo ha sido documentar tanto el uso básico del paquete como las funciones principales de la API.

La documentación de usuario explica cómo instalar la librería, preparar los datos de entrada, configurar los parámetros principales, ejecutar un experimento y guardar los resultados. Esta parte está pensada para alguien que quiera utilizar el paquete sin revisar todo el código fuente. Una versión de este manual se incluye en el Apéndice C.

La documentación de la API recoge los módulos, clases y funciones principales de la librería. Esta parte resulta útil si se quiere modificar el código, ampliar el pipeline o integrar alguna función en otros experimentos.

Los ficheros fuente de la documentación se almacenan en `docs/source`. La documentación HTML generada no se guarda en el repositorio. Para generarla localmente se utiliza:

```
sphinx-build -b html docs/source docs/_build/html
```

Además, la documentación se ha preparado para poder publicarse como página web estática mediante GitHub Pages.

8.7. Calidad del código y reproducibilidad

Además de implementar el pipeline, se han aplicado varias medidas para facilitar la revisión del código y la repetición de los experimentos:

- **Estructura modular.** El código se organiza en módulos separados para preprocesamiento, análisis topológico, construcción de carteras, backtesting y experimentación. Esta separación facilita localizar cada parte del pipeline y modificar componentes concretos.
- **Anotaciones de tipos.** Las funciones principales incluyen anotaciones de tipos. Esto ayuda a documentar qué entradas y salidas se esperan en cada caso.
- **Revisión de estilo.** El estilo del código se ha revisado con `flake8`, tomando como referencia las convenciones generales de PEP 8 [46].
- **Análisis estático.** Durante el desarrollo se ha utilizado `Pylance/Pyright` para detectar posibles errores de tipos o usos inconsistentes de funciones antes de ejecutar el código.
- **Docstrings.** Las funciones y clases principales incluyen docstrings. Esto facilita la lectura del código y permite generar documentación de API mediante Sphinx.

- **Configuraciones explícitas.** Los experimentos principales fijan de forma explícita los parámetros utilizados, como ventanas temporales, frecuencia de rebalanceo, costes de transacción, parámetros de Mapper y activación del control mediante homología persistente.
- **Exportación de resultados.** Los resultados experimentales se guardan en ficheros CSV. Esto permite generar tablas y figuras de la memoria a partir de salidas guardadas.
- **Control de dependencias.** El fichero `requirements-lock.txt` recoge las versiones exactas de las dependencias utilizadas en un entorno limpio de comprobación.
- **Comprobación del paquete.** Antes de publicar la librería en PyPI, se construyó la distribución del paquete y se validó mediante `twine check`.

9. Diseño experimental

En este capítulo se describe el diseño utilizado para evaluar las estrategias desarrolladas. Primero se presentan los conjuntos de datos, su preprocesamiento y la representación de los activos. Después, se explica el protocolo de backtesting y las métricas utilizadas para comparar las estrategias. Por último, se definen los tres modelos considerados en la evaluación final y las medidas adoptadas para mantener una validación temporal causal y facilitar la reproducibilidad de los experimentos.

9.1. Objetivo de la evaluación

El objetivo de la evaluación es estudiar si las estrategias construidas a partir de herramientas topológicas pueden ofrecer un comportamiento competitivo frente a una estrategia financiera sencilla de referencia. La comparación tiene en cuenta rentabilidad, riesgo, drawdown y estabilidad temporal.

Todos los modelos se evalúan mediante un protocolo causal. En cada fecha de rebalanceo solo se utiliza información disponible hasta ese momento, evitando incorporar datos futuros en la construcción de las carteras.

9.2. Conjuntos de datos

Se consideran dos universos financieros. El primero es KF49, formado por las 49 carteras industriales de Kenneth French. Este conjunto permite trabajar con un universo sectorial agregado y estable. Los datos proceden de la Kenneth R. French Data Library [19, 18].

El segundo universo está formado por activos del S&P 500 obtenidos a partir de CRSP/WRDS [8]. En este caso se trabaja con acciones individuales, lo que da lugar a un universo más amplio y con una estructura diferente a la de KF49.

Dataset	Tipo de activo	Frecuencia	Papel en el experimento
KF49	Carteras industriales	Mensual	Universo sectorial agregado
S&P 500	Acciones individuales	Mensual	Universo amplio de activos

Tabla 9.1.: Conjuntos de datos utilizados en los experimentos.

9.3. Preprocesamiento y representación de los activos

Antes de aplicar los métodos topológicos, los datos se transforman en matrices temporales homogéneas. En KF49, los retornos mensuales se obtienen en porcentaje y se convierten a escala decimal. Cuando se necesita trabajar con precios, se construye un índice sintético:

$$P_{i,t} = P_{i,t-1}(1 + r_{i,t}), \quad P_{i,0} = 100,$$

9. Diseño experimental

donde $r_{i,t}$ es el retorno del activo i en el mes t .

En el S&P 500, los datos originales están en frecuencia diaria. Para cada activo i y mes m , los retornos diarios se agregan mediante retorno compuesto:

$$R_{i,m} = \prod_{t \in m} (1 + r_{i,t}) - 1.$$

Se utiliza PERMNO como identificador del activo. En la matriz mensual se mantienen los valores ausentes asociados a entradas y salidas de activos, evitando rellenar hacia atrás observaciones que todavía no estaban disponibles.

Para aplicar Mapper, en cada fecha de rebalanceo se toma una ventana histórica de longitud L . Si hay N activos disponibles, la nube de puntos se representa como

$$X_t = \begin{pmatrix} r_{1,t-L+1} & \cdots & r_{1,t} \\ \vdots & \ddots & \vdots \\ r_{N,t-L+1} & \cdots & r_{N,t} \end{pmatrix} \in \mathbb{R}^{N \times L}.$$

Cada fila corresponde a un activo y cada columna a un mes de la ventana. Antes de construir el lens, los retornos se normalizan por activo:

$$\tilde{r}_{i,s} = \frac{r_{i,s} - \mu_i}{\sigma_i},$$

donde μ_i y σ_i son la media y desviación típica del activo i dentro de la ventana.

En la variante con homología persistente se construye una matriz de distancias entre activos basada en correlaciones. En los experimentos finales, Mapper utiliza una ventana de 60 meses y la señal de homología persistente se calcula con una ventana más corta de 20 meses. Esta elección busca que la señal topológica de régimen sea más sensible a cambios recientes en la estructura del mercado.

9.4. Protocolo experimental común

Todos los experimentos se realizan utilizando los mismos conjuntos de datos y el mismo esquema general de evaluación. De esta forma, tanto las decisiones estudiadas durante la evolución del modelo como la comparación final entre estrategias se evalúan bajo unas condiciones comunes.

Los experimentos se realizan con frecuencia mensual. Para evaluar el comportamiento a lo largo del tiempo, el periodo total se divide en cinco bloques:

$$1975\text{--}1984, \quad 1985\text{--}1994, \quad 1995\text{--}2004, \quad 2005\text{--}2014, \quad 2015\text{--}2025.$$

En cada bloque se aplica el mismo procedimiento de backtesting. La estrategia utiliza una ventana histórica de 60 meses para construir la cartera y se rebalancea cada 3 meses. Además, se incorporan costes de transacción de 5 puntos básicos proporcionales al turnover.

El backtesting se realiza de forma causal. En cada fecha de rebalanceo, los pesos se calculan utilizando únicamente información disponible hasta ese momento. Después, esos pesos se aplican al periodo posterior hasta el siguiente rebalanceo. Este esquema evita que las decisiones de cartera utilicen información futura.

Elemento	Configuración
Frecuencia de datos	Mensual
Ventana histórica	60 meses
Frecuencia de rebalanceo	3 meses
Tipo de cartera	Long-only
Costes de transacción	5 puntos básicos por turnover
Validación temporal	Causal
Periodos de evaluación	Cinco bloques temporales

Tabla 9.2.: Configuración común del backtesting utilizado en los experimentos.

9.4.1. Esquema común de construcción de carteras Mapper

En todos los experimentos basados en Mapper, el grafo se construye de nuevo en cada fecha de rebalanceo. Para una fecha t , se toma la ventana histórica de 60 meses inmediatamente anterior y se representa cada activo mediante sus retornos dentro de esa ventana. De esta forma, los activos forman una nube de puntos en la que cada punto corresponde a un activo financiero.

A partir de esta nube de puntos se aplica el pipeline Mapper: primero se calcula una función filtro o *lens*, después se recubre el espacio filtrado, se realiza clustering local dentro de cada elemento del recubrimiento y finalmente se construye el grafo Mapper. Los nodos del grafo representan clusters locales de activos, y las aristas aparecen cuando dos nodos comparten al menos un activo debido al solapamiento del recubrimiento [42].

Una vez obtenido el grafo en una fecha de rebalanceo, se necesita una regla adicional para convertirlo en pesos de cartera. Esta conversión no forma parte del algoritmo Mapper original, ya que Mapper proporciona una representación topológica de los datos. En este trabajo se estudian dos reglas para realizar ese paso: una basada en agrupaciones amplias del grafo y otra basada en la pertenencia de los activos a los nodos. La comparación entre ambas alternativas se presenta en la Sección 10.1.2.

9.5. Métricas de evaluación

Los resultados experimentales se resumen mediante medias calculadas sobre los cinco periodos de evaluación. Las métricas utilizadas son las siguientes.

El valor liquidativo o *NAV* mide la evolución acumulada de una cartera:

$$NAV_t = NAV_{t-1}(1 + R_t), \quad NAV_0 = 1,$$

donde R_t es el retorno de la cartera en el periodo t .

El retorno anualizado resume la rentabilidad de la estrategia en escala anual. Si T es el número de retornos mensuales evaluados, se calcula de forma geométrica como

$$R_{\text{anual}} = \left(\frac{NAV_T}{NAV_0} \right)^{12/T} - 1.$$

9. Diseño experimental

La volatilidad anualizada mide la variabilidad de los retornos en escala anual:

$$\sigma_{\text{anual}} = \sqrt{12} \sigma_R,$$

donde σ_R es la desviación típica de los retornos mensuales.

El ratio de Sharpe mide la rentabilidad obtenida por unidad de volatilidad:

$$\text{Sharpe} = \frac{\bar{R} - R_f}{\sigma_R}.$$

El ratio de Sortino es similar al Sharpe, pero utiliza únicamente la desviación negativa de los retornos. Por ello, resulta útil para distinguir entre volatilidad general y riesgo a la baja:

$$\text{Sortino} = \frac{\bar{R} - R_f}{\sigma_R^-}.$$

El máximo drawdown mide la mayor caída desde un máximo previo del valor liquidativo:

$$MDD = \min_t \left(\frac{NAV_t}{\max_{s \leq t} NAV_s} - 1 \right).$$

Esta métrica es especialmente importante en una estrategia financiera, ya que muestra la pérdida acumulada máxima sufrida durante el periodo evaluado.

Finalmente, el turnover mide cuánto cambian los pesos de la cartera entre rebalances. En este trabajo se utiliza para aplicar costes de transacción: cuanto mayor es el cambio en los pesos, mayor es el coste asignado a la estrategia.

Métrica	Interpretación
Retorno anualizado	Rentabilidad expresada en escala anual
Volatilidad anualizada	Variabilidad anualizada de los retornos
Sharpe	Rentabilidad ajustada por volatilidad
Sortino	Rentabilidad ajustada por riesgo a la baja
MDD	Mayor caída desde un máximo previo
NAV final	Valor acumulado final de la cartera
Turnover	Cambio medio de los pesos entre rebalances

Tabla 9.3.: Métricas utilizadas en las comparaciones experimentales.

9.6. Modelos comparados

La comparación final se realiza entre tres modelos: Mapper, Mapper con homología persistente y una cartera equiponderada.

9.6.1. Mapper

El primer modelo utiliza Mapper. En cada fecha de rebalanceo se construye una nube de puntos donde cada punto representa un activo y sus coordenadas son los retornos recientes

de ese activo. Sobre esta nube se aplica Mapper para obtener un grafo que resume relaciones entre activos.

Mapper fue introducido como un método para obtener descripciones simplificadas de datos de alta dimensión mediante funciones filtro, recubrimientos y clustering parcial [42]. En este trabajo, el grafo se utiliza como base para construir una cartera. Para ello, es necesario definir una regla que transforme la información contenida en los nodos del grafo en pesos para los activos. Las alternativas estudiadas y la elección final se presentan en la Sección 10.1.2.

9.6.2. Mapper con homología persistente

El segundo modelo mantiene Mapper como estrategia principal y añade una señal basada en homología persistente. Esta señal actúa como un mecanismo auxiliar de control de régimen.

En cada fecha de rebalanceo se calcula, sobre una ventana reciente de retornos, una matriz de distancias entre activos basada en correlaciones. A partir de esta matriz se obtienen los diagramas de persistencia y, siguiendo la representación introducida en el Capítulo 5, se calcula la norma L_2 del paisaje de persistencia en dimensión H_1 :

$$z_t = \|\lambda_t^{(1)}\|_2.$$

El valor z_t actúa como un resumen escalar de la información persistente en dimensión 1 para la ventana considerada. Este valor se compara con los observados en rebalanceos anteriores. En los experimentos se considera que la señal es anómala cuando la norma actual supera el percentil 95 de su historial reciente.

Cuando se detecta una anomalía, el modelo aplica temporalmente una acción defensiva. Durante el desarrollo se estudiaron dos posibles respuestas ante esta señal. La comparación entre ambas alternativas y la elección utilizada en la evaluación final se presentan en la Sección 10.1.3.

9.6.3. Cartera equiponderada

El tercer modelo es una cartera equiponderada, donde todos los activos disponibles reciben el mismo peso en cada fecha de rebalanceo. Esta estrategia se utiliza como benchmark principal.

La elección de este benchmark se debe a que la regla $1/N$ es sencilla, transparente y evita estimar medias, covarianzas o resolver un problema de optimización. Además, DeMiguel, Garlappi y Uppal muestran que esta estrategia es una referencia exigente fuera de muestra frente a modelos más complejos de asignación de activos [13].

Modelo	Descripción	Papel en el experimento
Mapper	Grafo Mapper transformado en pesos	Modelo topológico principal
Mapper + PH	Mapper con señal topológica de régimen	Extensión con control de régimen
Equal-weight	Pesos iguales para todos los activos	Benchmark clásico

Tabla 9.4.: Modelos comparados en la evaluación final.

9.7. Control temporal y reproducibilidad

La evaluación final sigue el protocolo descrito en la Sección 9.4, manteniendo el mismo esquema causal en todas las estrategias comparadas.

La configuración de los experimentos se fija antes de obtener las tablas finales y se mantiene constante dentro de cada conjunto de datos. Los resultados se guardan en ficheros CSV, lo que permite generar las tablas y figuras de la memoria a partir de las salidas obtenidas.

10. Resultados experimentales

En este capítulo se presentan, en primer lugar, las comparaciones realizadas durante la evolución del modelo y las decisiones que llevaron a seleccionar su configuración final. A continuación, se recogen los resultados obtenidos al comparar las tres estrategias definidas en el capítulo anterior: Mapper, Mapper con homología persistente y la cartera equiponderada utilizada como referencia.

Los resultados completos por periodo se incluyen en los apéndices correspondientes. El Apéndice A recoge los resultados asociados a la evolución del modelo, mientras que el Apéndice B contiene los resultados completos de la comparación final.

Además de las métricas financieras, se incluye una comparación de tiempos de ejecución para valorar el coste computacional asociado a cada estrategia.

10.1. Evolución del modelo

La configuración final del modelo se obtuvo tras comparar varias alternativas razonables durante el desarrollo del trabajo. En particular, se estudiaron distintos métodos de clustering local para Mapper, dos reglas de asignación de pesos, dos respuestas posibles ante la señal de homología persistente y varias configuraciones del recubrimiento.

Estas decisiones son necesarias porque Mapper no determina todos los elementos de una aplicación concreta. El algoritmo define la construcción general basada en una función filtro, un recubrimiento y clustering local, pero deja abiertas distintas elecciones que dependen del problema considerado [42]. Además, en este trabajo es necesario transformar el grafo resultante en pesos de cartera, lo que introduce una decisión adicional específica de la aplicación financiera.

En las siguientes subsecciones se presentan las comparaciones realizadas para seleccionar la configuración utilizada en la evaluación final.

10.1.1. Clustering local: DBSCAN frente a HACA

La primera decisión estudiada fue el algoritmo de clustering local utilizado dentro de Mapper. En una primera versión se empleó DBSCAN, un algoritmo basado en densidad que permite encontrar grupos sin fijar previamente el número de clusters y que puede identificar puntos considerados ruido [17]. Esta propiedad puede ser útil en muchos problemas de análisis de datos, pero en este trabajo tiene una consecuencia importante: los puntos de la nube representan activos financieros, por lo que marcar un punto como ruido puede hacer que ciertos activos reciban un tratamiento menos estable dentro del grafo Mapper.

Como alternativa, se probó un método de clustering jerárquico aglomerativo, que en este trabajo se denomina HACA. Los métodos jerárquicos construyen agrupaciones de forma progresiva, fusionando observaciones o grupos según una medida de distancia [37]. En este contexto, resultan atractivos porque permiten agrupar todos los activos sin descartarlos explícitamente como ruido. En la implementación se utilizan herramientas de `scikit-learn`, librería ampliamente utilizada para aprendizaje automático en Python [39].

10. Resultados experimentales

La **Tabla 10.1** resume la comparación entre ambas alternativas. Las métricas se han calculado como medias sobre los cinco periodos de evaluación utilizados en los experimentos.

Dataset	Variante	Ret. anual.	Vol. anual.	Sharpe	Sortino	MDD	NAV final
KF49	DBSCAN	12.81 %	16.84 %	0.552	0.822	-30.78 %	3.49
KF49	HACA	13.45 %	16.01 %	0.609	0.922	-28.99 %	3.71
S&P 500	DBSCAN	9.48 %	10.86 %	0.489	0.736	-22.16 %	2.62
S&P 500	HACA	8.56 %	9.11 %	0.478	0.692	-17.61 %	2.37

Tabla 10.1.: Comparación resumida entre DBSCAN y HACA. Las métricas son medias sobre los cinco periodos de evaluación.

En las 49 carteras industriales, HACA mejora a DBSCAN en retorno anualizado, volatilidad, Sharpe, Sortino, máximo drawdown y valor liquidativo final. En el S&P 500, DBSCAN obtiene una rentabilidad media y un Sharpe ligeramente superiores, pero HACA reduce de forma clara la volatilidad y el máximo drawdown. Como se busca una regla estable y razonable para distintos universos de activos, se seleccionó HACA como método de clustering local para la versión final.

La **Figura 10.1** muestra de forma visual la comparación del Sharpe medio entre ambas alternativas.

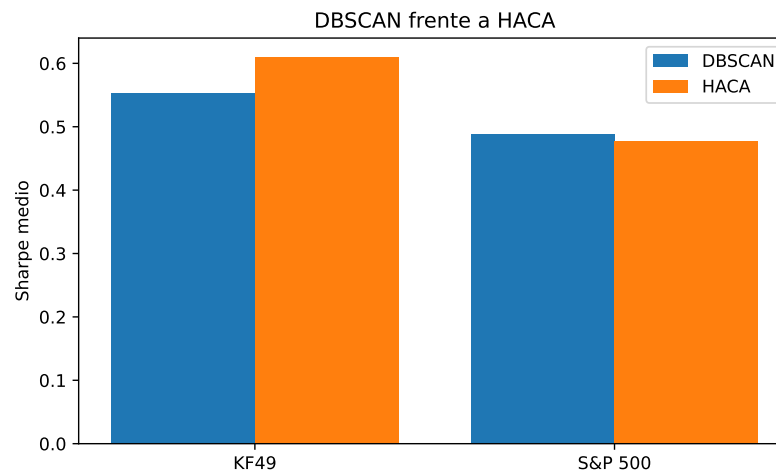


Figura 10.1.: Comparación del Sharpe medio entre DBSCAN y HACA.

10.1.2. Asignación de pesos: macrocluster frente a solapamiento de nodos

Como se ha explicado en la Sección 9.4.1, en cada fecha de rebalanceo se construye un grafo Mapper a partir de la ventana histórica de retornos. El siguiente paso consiste en transformar ese grafo en pesos de cartera. Este paso es específico del proyecto, ya que Mapper genera una representación topológica de la nube de puntos, mientras que una cartera necesita asignar un peso numérico a cada activo.

En este trabajo se estudiaron dos reglas para convertir el grafo en pesos. La primera, denominada *macrocluster*, utiliza agrupaciones amplias del grafo Mapper. La idea es identificar grupos globales de nodos conectados y asignar peso a los activos según el grupo al que pertenecen. Por tanto, esta regla utiliza la información del grafo de forma agregada.

La segunda regla, denominada *node overlap*, utiliza directamente la pertenencia de los activos a los nodos del grafo. Debido al solapamiento del recubrimiento, un mismo activo puede aparecer en más de un nodo. En la implementación desarrollada, un activo que aparece en varios nodos recibe mayor presencia inicial en la cartera. Después, los pesos se normalizan y se aplica un límite máximo por activo para evitar concentraciones excesivas.

La [Tabla 10.2](#) muestra la comparación resumida entre ambas reglas.

Dataset	Variante	Ret. anual.	Vol. anual.	Sharpe	Sortino	MDD	NAV final
KF49	Macrocluster	13.55 %	15.87 %	0.620	0.945	-28.31 %	3.76
KF49	Node overlap	13.45 %	16.01 %	0.609	0.922	-28.99 %	3.71
S&P 500	Macrocluster	7.42 %	8.21 %	0.388	0.553	-16.40 %	2.12
S&P 500	Node overlap	8.56 %	9.11 %	0.478	0.692	-17.61 %	2.37

Tabla 10.2.: Comparación resumida entre macrocluster y solapamiento de nodos. Las métricas son medias sobre los cinco periodos de evaluación.

Los resultados no son idénticos en ambos conjuntos de datos. En KF49, macrocluster obtiene valores ligeramente mejores en las métricas principales. Sin embargo, en el S&P 500, node overlap mejora de forma clara el retorno anualizado, el Sharpe, el Sortino y el NAV final. Además, node overlap resulta más coherente con la estructura de Mapper, ya que utiliza de forma directa la información de solapamiento generada por el recubrimiento.

Por este motivo, se seleccionó node overlap como regla final de asignación de pesos.

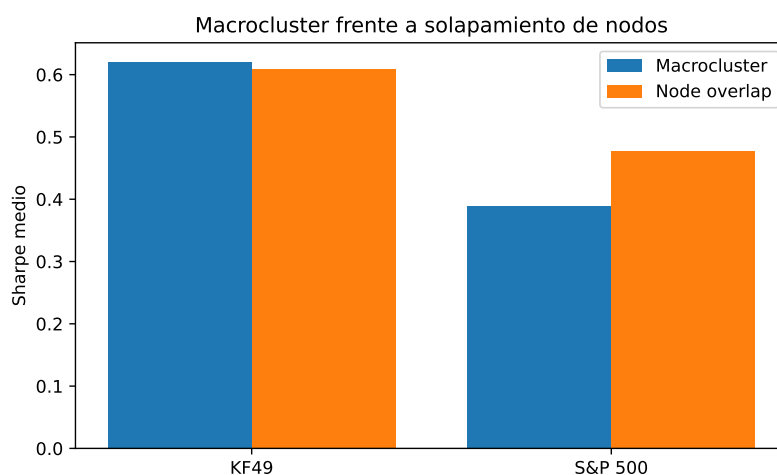


Figura 10.2.: Comparación del Sharpe medio entre macrocluster y solapamiento de nodos.

10.1.3. Control de régimen: liquidez frente a cartera equiponderada

Además del modelo basado únicamente en Mapper, se desarrolló una variante que incorpora homología persistente como señal auxiliar. La motivación procede de trabajos que aplican TDA a series financieras mediante ventanas deslizantes, diagramas de persistencia y persistence landscapes. En particular, Gidea y Katz utilizan normas L^p de persistence landscapes para estudiar cambios topológicos asociados a episodios de inestabilidad financiera [20].

En este proyecto, la homología persistente se utiliza como un mecanismo complementario de control de régimen. Cuando la señal topológica indica un comportamiento anómalo, el modelo puede modificar temporalmente la cartera generada por Mapper. Durante el desarrollo se compararon dos posibles respuestas: pasar a liquidez o sustituir temporalmente la cartera Mapper por una cartera equiponderada.

La opción de liquidez es más agresiva, ya que reduce completamente la exposición a los activos de riesgo durante los periodos señalados. La opción equiponderada es más conservadora: mantiene la inversión en el universo de activos, pero desactiva temporalmente la señal específica de Mapper. Además, la cartera $1/N$ es una referencia habitual en asignación de activos. DeMiguel, Garlappi y Uppal la utilizan como benchmark precisamente porque no requiere estimar momentos ni resolver un problema de optimización, y muestran que no es trivial superar de forma consistente esta regla fuera de muestra [13].

La comparación entre ambas acciones defensivas se recoge en la [Tabla 10.3](#).

Dataset	Variante	Ret. anual.	Vol. anual.	Sharpe	Sortino	MDD	NAV final
KF49	Liquidez	12.46 %	15.08 %	0.574	0.901	-27.64 %	3.48
KF49	Equal-weight	13.50 %	16.01 %	0.612	0.922	-28.98 %	3.72
S&P 500	Liquidez	7.47 %	8.76 %	0.375	0.549	-16.65 %	2.15
S&P 500	Equal-weight	8.39 %	9.38 %	0.450	0.648	-18.04 %	2.33

Tabla 10.3.: Comparación resumida entre las dos acciones defensivas asociadas a la señal de homología persistente. Las métricas son medias sobre los cinco periodos de evaluación.

Los resultados muestran que la acción basada en liquidez reduce algo el máximo draw-down, pero a costa de menor retorno anualizado, menor Sharpe y menor NAV final en ambos conjuntos de datos. La sustitución temporal por equal-weight ofrece un equilibrio más razonable entre mantener exposición al mercado y reducir la dependencia de la señal Mapper en periodos detectados como anómalos. Por ello, la versión final con homología persistente utiliza equal-weight como acción defensiva.

10.1.4. Sensibilidad respecto al recubrimiento

El último bloque de decisiones está relacionado con el recubrimiento del espacio filtro. En Mapper, el recubrimiento controla cómo se divide el espacio sobre el que se proyectan los datos. Si el recubrimiento es demasiado grueso, puede ocultar estructura local; si es demasiado fino, puede producir grafos demasiado fragmentados. El artículo original de Mapper ya destaca que la construcción depende de una cubierta del espacio de parámetros y que puede verse como una representación a distintos niveles de resolución [42].

En la implementación utilizada, el recubrimiento se controla mediante dos parámetros principales: el número de cubos, n_cubes , y el porcentaje de solapamiento, $perc_overlap$. Se

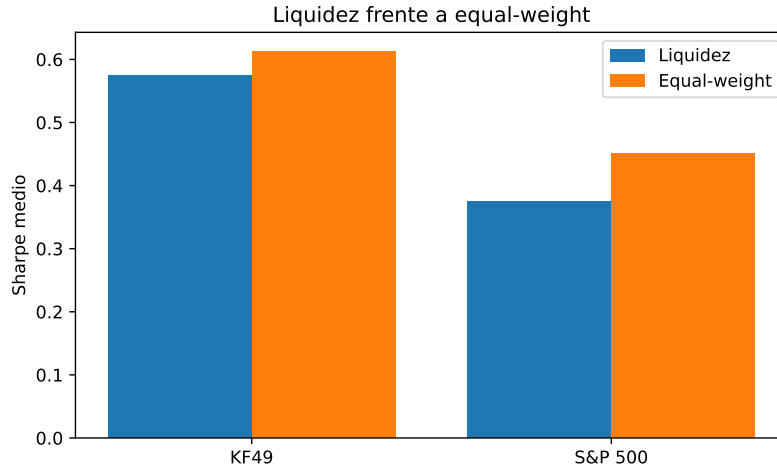


Figura 10.3.: Comparación del Sharpe medio entre liquidez y cartera equiponderada como acción defensiva.

consideraron los siguientes valores:

$$n_cubes \in \{8, 12, 16\}, \quad \text{perc_overlap} \in \{0.15, 0.25, 0.40\}.$$

Las [Figura 10.4](#) y [Figura 10.5](#) muestran el Sharpe medio obtenido para cada combinación de parámetros en los dos conjuntos de datos.

A partir de este análisis se seleccionaron las configuraciones de la [Tabla 10.4](#). En KF49, la mejor configuración según Sharpe medio fue $n_cubes = 12$ y $\text{perc_overlap} = 0.25$. En el S&P 500, la mejor configuración dentro de la parrilla fue $n_cubes = 16$ y $\text{perc_overlap} = 0.15$.

Las tablas completas del estudio de sensibilidad del recubrimiento se incluyen también en el [Apéndice A](#).

Dataset	n_cubes	perc_overlap
KF49	12	0.25
S&P 500	16	0.15

Tabla 10.4.: Configuración final del recubrimiento seleccionada tras el análisis de sensibilidad.

Las configuraciones seleccionadas son distintas en ambos conjuntos de datos. Esto indica que el comportamiento del recubrimiento depende del universo considerado y que una misma elección de parámetros no tiene por qué ser la más adecuada en todos los casos. Por este motivo, se mantiene una configuración específica para KF49 y otra para el S&P 500 en la evaluación final.

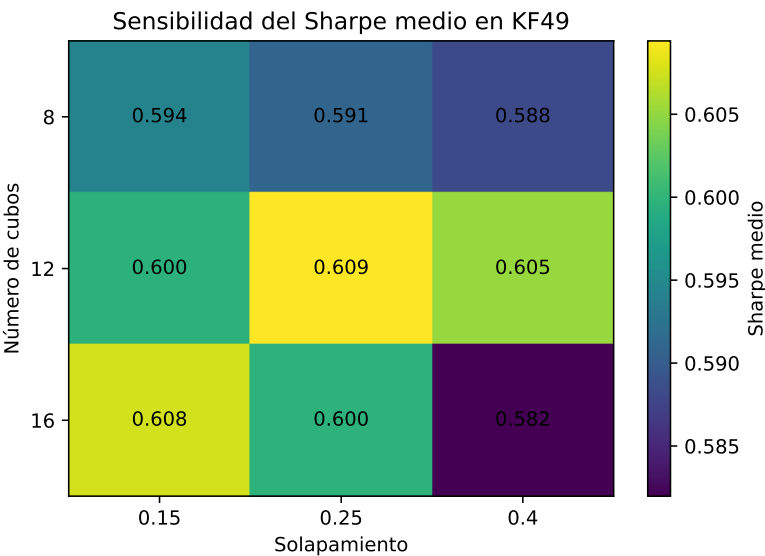


Figura 10.4.: Sensibilidad del Sharpe medio respecto a los parámetros del recubrimiento en KF49.

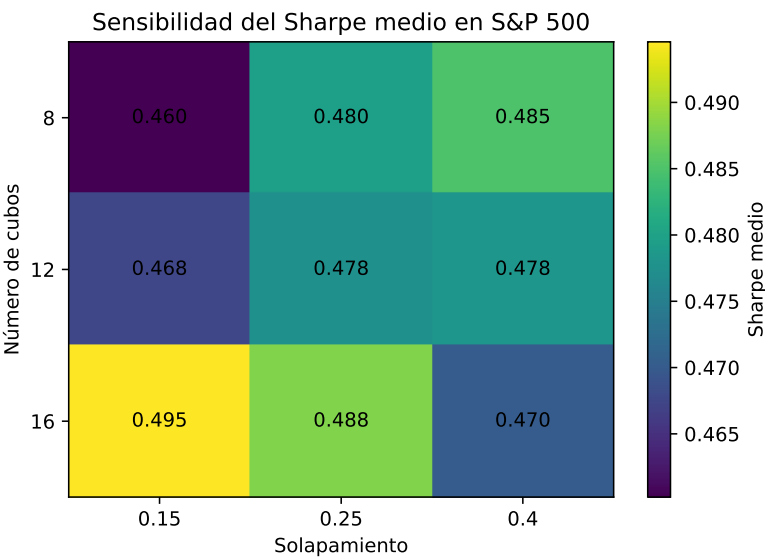


Figura 10.5.: Sensibilidad del Sharpe medio respecto a los parámetros del recubrimiento en S&P 500.

10.2. Configuración final de los modelos

La **Tabla 10.5** resume las decisiones seleccionadas a partir de las comparaciones presentadas en la Sección 10.1. Esta configuración se mantiene fija en la evaluación final.

Componente	KF49	S&P 500
Clustering local	HACA	HACA
Regla de pesos	Node overlap	Node overlap
Acción ante alerta PH	Equal-weight	Equal-weight
n_cubes	12	16
perc_overlap	0.25	0.15

Tabla 10.5.: Configuración seleccionada tras las comparaciones realizadas durante la evolución del modelo.

10.3. Resultados sobre las 49 carteras industriales

La **Tabla 10.6** resume los resultados medios obtenidos sobre las 49 carteras industriales. Las métricas se calculan como media sobre los cinco periodos de evaluación.

Modelo	Ret. anual	Vol. anual	Sharpe	Sortino	MDD	NAV final
Mapper	13.45 %	16.01 %	0.609	0.922	-28.99 %	3.71
Mapper + PH	13.50 %	16.01 %	0.612	0.922	-28.98 %	3.72
Equal-weight	13.04 %	16.44 %	0.575	0.860	-30.36 %	3.55

Tabla 10.6.: Resultados medios sobre las 49 carteras industriales. Las métricas son medias sobre los cinco periodos de evaluación.

En este conjunto de datos, los dos modelos topológicos obtienen resultados ligeramente superiores a la cartera equiponderada. Mapper alcanza un retorno anualizado medio del 13.45 %, frente al 13.04 % del benchmark. Al incorporar homología persistente, el retorno anualizado medio aumenta ligeramente hasta el 13.50 %. La mejora también aparece en las métricas ajustadas por riesgo: el Sharpe medio pasa de 0.575 en equal-weight a 0.609 en Mapper y 0.612 en Mapper con PH.

La diferencia es pequeña, aunque apunta a que, en un universo sectorial agregado, la estructura de co-movimientos entre activos puede contener información útil para construir carteras. Además, el máximo drawdown medio de los modelos topológicos es algo menor que el del benchmark: -28.99 % en Mapper y -28.98 % en Mapper con PH, frente al -30.36 % de la cartera equiponderada.

La **Figura 10.6** muestra la evolución encadenada del valor liquidativo. La curva se ha construido concatenando los periodos de evaluación, por lo que debe interpretarse como una visualización comparativa de la trayectoria de los modelos entre bloques temporales.

La figura muestra que las tres estrategias siguen trayectorias parecidas, pero los modelos topológicos terminan ligeramente por encima de la cartera equiponderada. La variante con homología persistente apenas modifica el comportamiento de Mapper, aunque mejora levemente el resultado medio.

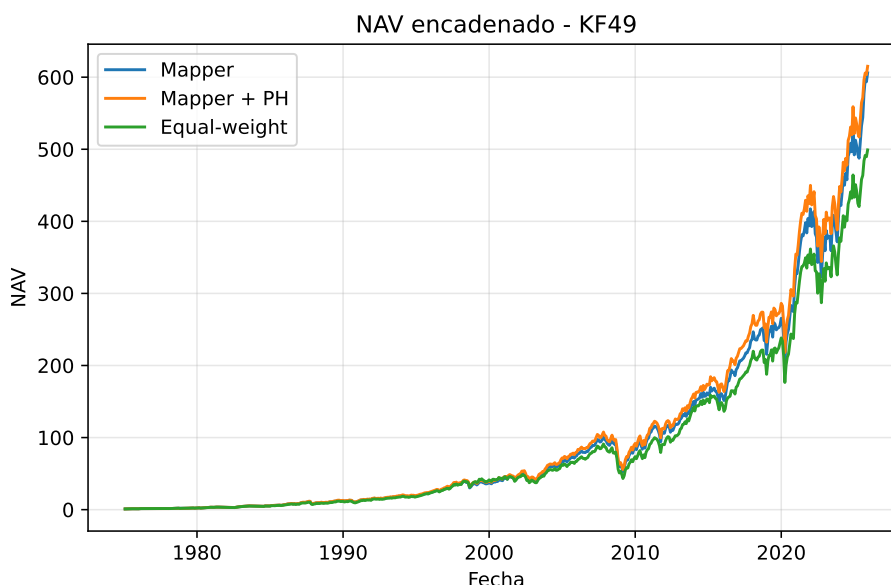


Figura 10.6.: Evolución encadenada del valor liquidativo para los modelos comparados en KF49.

10.4. Resultados sobre el S&P 500

La [Tabla 10.7](#) recoge los resultados medios obtenidos sobre el universo del S&P 500.

Modelo	Ret. anual	Vol. anual	Sharpe	Sortino	MDD	NAV final
Mapper	8.76 %	9.16 %	0.495	0.717	-17.34 %	2.41
Mapper + PH	8.66 %	9.42 %	0.475	0.686	-18.41 %	2.39
Equal-weight	11.19 %	12.58 %	0.574	0.871	-23.70 %	3.01

Tabla 10.7.: Resultados medios sobre el S&P 500. Las métricas son medias sobre los cinco periodos de evaluación.

En este caso, el comportamiento es distinto al observado en KF49. La cartera equiponderada obtiene el mayor retorno anualizado medio, con un 11.19 %, frente al 8.76 % de Mapper y el 8.66 % de Mapper con PH. También presenta el mejor Sharpe medio, con un valor de 0.574.

Los modelos topológicos reducen claramente el máximo drawdown medio. Mapper presenta un drawdown medio de -17.34 %, mientras que la cartera equiponderada alcanza -23.70 %. Por tanto, en el S&P 500 el modelo basado en Mapper queda por debajo del benchmark en rentabilidad, pero muestra un comportamiento más defensivo.

También se observa que la incorporación de homología persistente no mejora el resultado de Mapper en este universo. Mapper sin PH obtiene mayor retorno anualizado, menor volatilidad, mayor Sharpe, mayor Sortino, menor máximo drawdown y mayor NAV final. Esto indica que, para el S&P 500, la señal de régimen basada en homología persistente no aporta una mejora adicional bajo la configuración utilizada. Una posible explicación es que

la señal detecta cambios topológicos que no coinciden necesariamente con momentos en los que convenga sustituir Mapper por una cartera equiponderada. Por ello, en este conjunto de datos la versión con PH debe interpretarse como una prueba de control de régimen que no resulta beneficiosa en los resultados finales.

La **Figura 10.7** representa la evolución encadenada del valor liquidativo para los tres modelos.

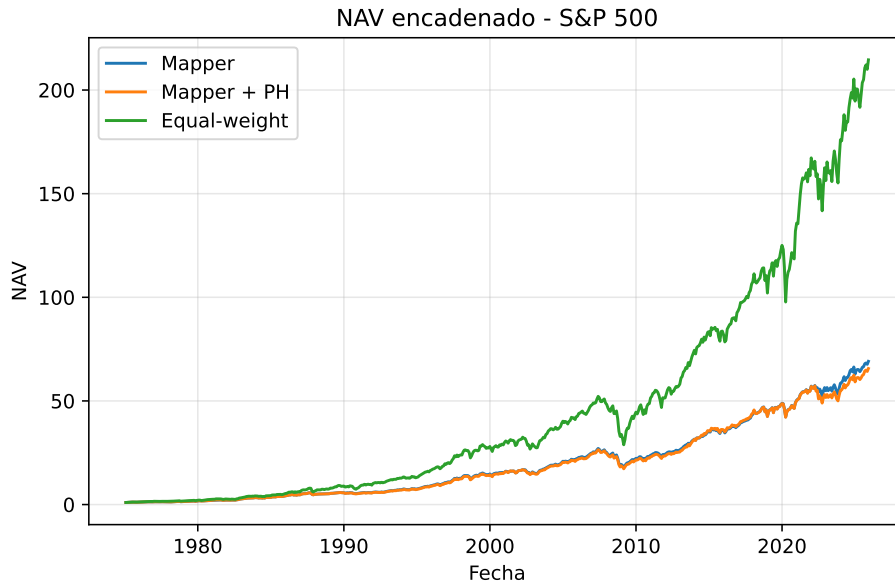


Figura 10.7.: Evolución encadenada del valor liquidativo para los modelos comparados en S&P 500.

La figura confirma que equal-weight obtiene una trayectoria final superior en este universo. Las estrategias topológicas terminan con menor NAV final, aunque presentan menor volatilidad y menor drawdown medio.

10.5. Comparación global

La **Figura 10.8** compara el retorno anualizado medio de los modelos en ambos conjuntos de datos. En KF49, las diferencias entre modelos son reducidas, aunque Mapper con PH obtiene el mayor retorno medio. En el S&P 500, la cartera equiponderada se mantiene claramente por encima de los modelos topológicos en términos de rentabilidad.

La **Figura 10.9** muestra el Sharpe medio. En KF49, Mapper con PH alcanza el valor más alto, con un Sharpe medio de 0.612. En el S&P 500, equal-weight obtiene el mejor Sharpe medio, con un valor de 0.574. Esto indica que el rendimiento relativo de los modelos topológicos depende del universo financiero considerado.

La **Figura 10.10** compara el máximo drawdown medio en valor absoluto. En ambos conjuntos de datos, los modelos basados en Mapper presentan drawdowns medios inferiores o muy próximos a los de la cartera equiponderada. Esta diferencia es especialmente clara en el S&P 500, donde Mapper reduce el drawdown medio de 23.70 % a 17.34 %.

La **Tabla 10.8** resume qué modelo obtiene el mejor resultado en cada métrica principal.

10. Resultados experimentales

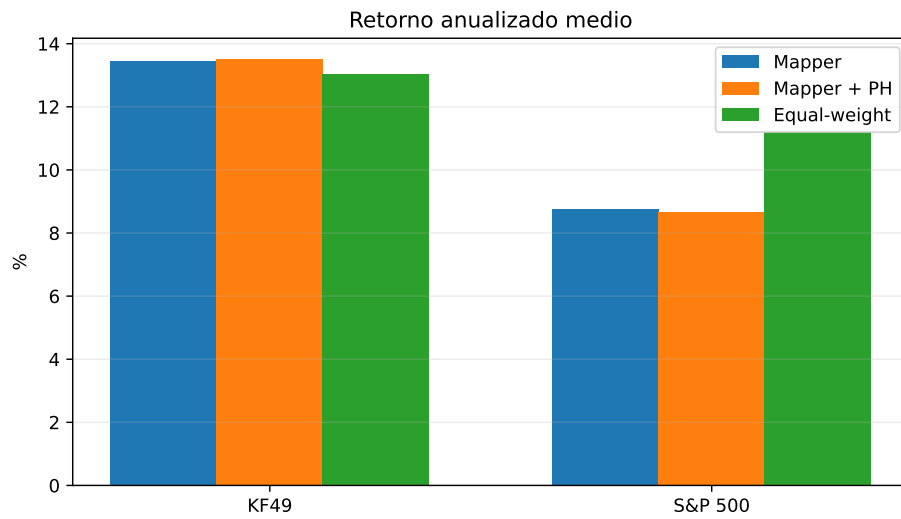


Figura 10.8.: Retorno anualizado medio por modelo y conjunto de datos.

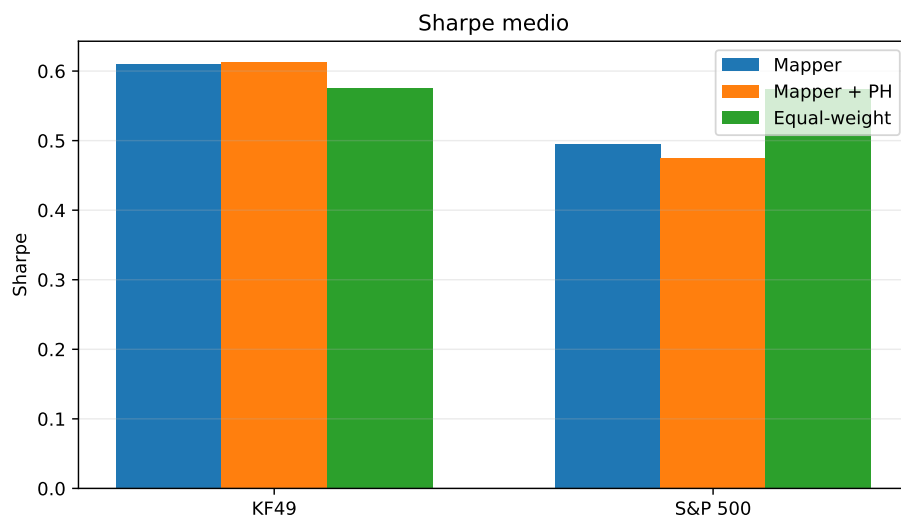


Figura 10.9.: Sharpe medio por modelo y conjunto de datos.

Dataset	Mejor retorno anual	Mejor Sharpe	Menor MDD medio
KF49	Mapper + PH (13.50 %)	Mapper + PH (0.612)	Mapper + PH (-28.98 %)
S&P 500	Equal-weight (11.19 %)	Equal-weight (0.574)	Mapper (-17.34 %)

Tabla 10.8.: Resumen de los mejores modelos por métrica principal.

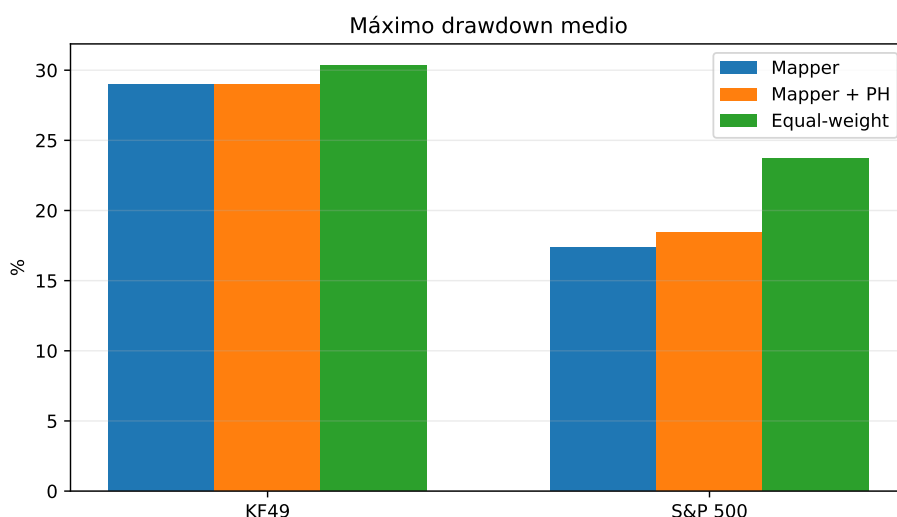


Figura 10.10.: Máximo drawdown medio en valor absoluto por modelo y conjunto de datos.

10.6. Tiempos de ejecución

Además de las métricas financieras, se compararon los tiempos de ejecución de las tres estrategias. Esta comparación es relevante porque los modelos tienen costes computacionales distintos. La cartera equiponderada solo necesita repartir el peso entre los activos disponibles. Mapper, en cambio, debe construir una nube de puntos, aplicar una función filtro, recubrir el espacio filtrado, realizar clustering local y transformar el grafo resultante en pesos de cartera. La variante Mapper con PH añade además el cálculo de una señal topológica de régimen.

Los tiempos se midieron en el mismo equipo y con el mismo entorno de Python para todos los modelos. El equipo utilizado fue un ordenador personal con procesador 13th Gen Intel® Core™ i7-1360P × 16, 16 GiB de memoria RAM y sistema operativo Ubuntu 24.04.4 LTS.

Modelo	Tiempo medio por periodo (s)	Tiempo total (s)
Mapper	8.52	42.61
Mapper + PH	5.69	28.45
Equal-weight	0.05	0.24

Tabla 10.9.: Tiempos de ejecución de los modelos en KF49.

En KF49, los dos modelos topológicos presentan tiempos de ejecución reducidos, de pocos segundos por periodo. Mapper con PH aparece más rápido en media que Mapper. Esto se debe a que, en algunos rebalanceos, la señal de régimen activa la acción defensiva y la estrategia pasa directamente a una cartera equiponderada. En esos casos se evita construir el grafo Mapper, cuyo cálculo es más costoso. Como KF49 es un universo pequeño, este ahorro puede compensar el coste adicional de calcular la señal PH.

En el S&P 500, los tiempos aumentan claramente respecto a KF49. Esto es esperable porque

Modelo	Tiempo medio por periodo (s)	Tiempo total (s)
Mapper	44.25	221.23
Mapper + PH	50.61	253.07
Equal-weight	0.19	0.97

Tabla 10.10.: Tiempos de ejecución de los modelos en S&P 500.

el universo contiene muchos más activos individuales. Al aumentar el número de activos, crece la nube de puntos sobre la que se construye Mapper, aumenta el número de distancias y relaciones que se deben analizar, y el clustering local también resulta más costoso.

En este caso, Mapper con PH tarda más que Mapper sin PH. Aunque la implementación evita construir Mapper cuando se activa la acción defensiva, el coste de calcular la señal persistente es mayor en un universo amplio como el S&P 500. Por ello, el ahorro obtenido en algunos rebalanceos no compensa completamente el coste adicional de la homología persistente.

La cartera equiponderada sigue teniendo un tiempo prácticamente despreciable en ambos conjuntos de datos, ya que su regla de construcción es directa. En conjunto, los resultados muestran que las estrategias topológicas tienen un coste computacional mayor que una regla equiponderada, especialmente cuando aumenta el número de activos. Aun así, los tiempos obtenidos son asumibles para el tipo de experimento realizado, ya que se trabaja con datos mensuales y rebalanceo trimestral, no con una estrategia de alta frecuencia.

10.7. Análisis por periodos

Además de comparar medias, es importante observar si el comportamiento de los modelos se mantiene estable a lo largo del tiempo. Para ello, la [Figura 10.11](#) muestra el Sharpe obtenido en cada periodo de evaluación para KF49.

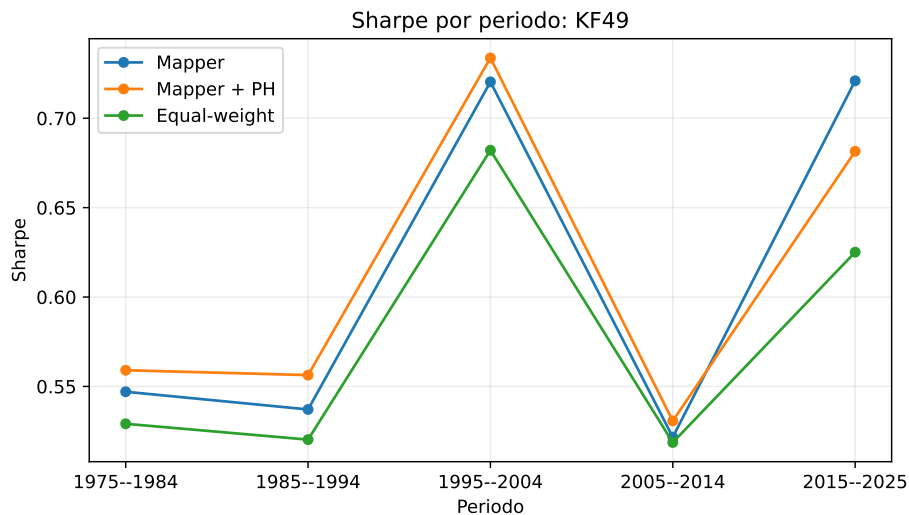


Figura 10.11.: Sharpe por periodo para los modelos comparados en KF49.

En KF49, los modelos topológicos son competitivos en varios periodos y presentan un comportamiento bastante próximo al benchmark. La ventaja media de Mapper con PH procede de la regularidad de sus resultados a lo largo de las distintas ventanas temporales.

La Figura 10.12 muestra el mismo análisis para el S&P 500.

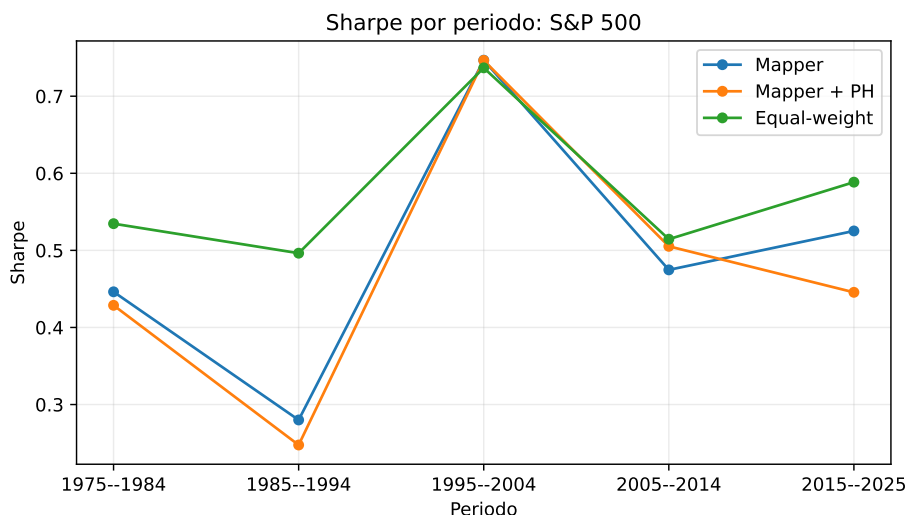


Figura 10.12.: Sharpe por periodo para los modelos comparados en S&P 500.

En el S&P 500, la cartera equiponderada obtiene mejores valores de Sharpe en varios periodos. Esto refuerza la idea de que, en un universo amplio de acciones individuales, la regla $1/N$ sigue siendo un benchmark difícil de superar. Aun así, Mapper mantiene un perfil de riesgo más moderado, especialmente en términos de drawdown.

10.8. Discusión de los resultados

Los resultados muestran que el comportamiento de las estrategias depende claramente del universo financiero considerado. En KF49, los modelos topológicos superan ligeramente a la cartera equiponderada en retorno anualizado, Sharpe, Sortino, máximo drawdown y NAV final. En el S&P 500, la cartera equiponderada obtiene mejores resultados de rentabilidad y Sharpe, mientras que Mapper presenta un menor drawdown medio.

Una posible explicación es que las 49 carteras industriales forman un universo más agregado y estable. Cada activo representa una industria completa, por lo que parte del ruido específico de cada compañía queda suavizado. En este contexto, las relaciones entre sectores pueden ser más fáciles de capturar mediante una representación topológica.

El caso del S&P 500 es distinto. Aunque contiene muchas acciones individuales, el índice original está ponderado por capitalización bursátil ajustada por las acciones disponibles para negociación, lo que se conoce como *free float* [43]. Esto hace que unas pocas compañías grandes puedan tener mucha influencia sobre el comportamiento del índice. Además, en los últimos años esa concentración ha estado muy relacionada con empresas tecnológicas o ligadas a tecnología. Por ejemplo, en el SPDR S&P 500 ETF Trust, que replica el S&P 500, el sector de tecnología de la información representaba el 37.37% del fondo a 29 de junio de 2026 [45].

Aunque en este trabajo las carteras se construyen asignando pesos a los activos disponibles, el universo del S&P 500 procede de un índice influido por grandes compañías y por sectores dominantes. En ese contexto, la estructura detectada por Mapper puede ser menos estable, mientras que una regla sencilla como equal-weight puede funcionar bien al repartir el peso de manera uniforme entre los activos.

La homología persistente debe interpretarse dentro de este trabajo como una señal auxiliar de control de régimen. En KF49 produce una mejora ligera respecto a Mapper, tanto en Sharpe como en retorno anualizado medio. En el S&P 500 ocurre lo contrario: Mapper sin PH obtiene mejores métricas principales. Esto sugiere que la señal persistente no funciona como una mejora automática del modelo, ya que su utilidad depende del universo de activos y de la forma en que se traduzca la detección de anomalías en una acción de cartera. En estos experimentos, la acción defensiva basada en pasar temporalmente a equal-weight funciona mejor en KF49 que en S&P 500.

En conjunto, los resultados no permiten afirmar que el enfoque topológico supere de forma general a una cartera equiponderada. Aun así, muestran que Mapper puede utilizarse de forma causal para construir carteras y que, en el caso de las carteras industriales, obtiene resultados ligeramente mejores que el benchmark. En el S&P 500 no alcanza la rentabilidad de equal-weight, pero reduce el drawdown medio, por lo que su comportamiento puede entenderse como más defensivo.

10.9. Limitaciones

Los resultados deben interpretarse con cautela. En primer lugar, solo se han considerado dos universos financieros: uno formado por carteras sectoriales y otro por acciones individuales. Esto permite comparar dos escenarios distintos, pero no garantiza que las mismas conclusiones se mantengan en otros mercados, otros periodos históricos o con otra frecuencia de datos.

Además, KF49 está formado por carteras académicas, no por activos que se puedan comprar directamente. Esto lo hace muy útil para probar el método en un conjunto estable, pero limita su uso práctico. Si se quisiera llevar una estrategia parecida al mercado real, habría que buscar activos negociables relacionados con esos sectores.

En segundo lugar, la configuración final se fijó tras un análisis de sensibilidad limitado. Se evitó hacer una búsqueda muy amplia de hiperparámetros porque, en un backtesting, probar muchas combinaciones puede acabar ajustando el modelo demasiado a los periodos históricos utilizados. El objetivo era evaluar una configuración razonable y reproducible, no encontrar la combinación que mejor encajase con cada periodo. Aun así, podrían existir otras configuraciones con mejores resultados.

En tercer lugar, el backtesting incluye costes de transacción proporcionales al turnover, pero sigue siendo una simulación simplificada. En una aplicación real habría que tener en cuenta otros factores, como la liquidez de los activos, la diferencia entre precio de compra y venta, el posible impacto de mercado y aspectos fiscales.

Por último, estos experimentos tienen carácter académico y exploratorio. El objetivo es estudiar la viabilidad de incorporar herramientas de análisis topológico de datos a un proceso reproducible de construcción y evaluación de carteras.

Parte III.

Cierre del trabajo

11. Conclusiones y vías futuras

11.1. Conclusiones generales

El objetivo general de este Trabajo Fin de Grado era estudiar el Análisis Topológico de Datos y aplicarlo a un problema real de alta dimensión. Durante el desarrollo del trabajo, se optó por centrar la parte aplicada en finanzas, utilizando herramientas de TDA para estudiar relaciones entre activos y construir estrategias experimentales de cartera.

La primera conclusión es que el TDA ofrece una forma distinta de analizar datos complejos. La parte matemática ha permitido entender cómo pasar de una nube de puntos a objetos topológicos, cómo calcular información mediante homología y cómo estudiar la evolución de esa información con filtraciones y homología persistente. Esta base teórica era necesaria para que la parte aplicada tuviera sentido y para interpretar qué se estaba calculando en cada etapa.

La segunda conclusión es que estas herramientas pueden incorporarse a un flujo computacional completo. Con el objetivo de facilitar su uso sin necesidad de conocer en profundidad todos los detalles matemáticos internos, se ha desarrollado una librería en Python capaz de preparar datos financieros, construir grafos Mapper, calcular información de homología persistente, generar pesos de cartera, ejecutar backtests causales y guardar resultados reproducibles. El resultado final combina así el estudio teórico del TDA con una herramienta software funcional.

La tercera conclusión procede de los resultados experimentales. En las 49 carteras industriales de Kenneth French, los modelos topológicos obtienen resultados ligeramente mejores que la cartera equiponderada. En el S&P 500, la cartera equiponderada consigue mayor rentabilidad y mejor Sharpe, aunque Mapper reduce el máximo drawdown medio. Esto muestra que el enfoque topológico puede ser competitivo en algunos contextos, pero que su comportamiento depende bastante del universo financiero utilizado.

El resultado principal del trabajo es haber conseguido conectar la teoría topológica con un problema financiero evaluable. Incluso cuando Mapper o Mapper con PH no superan al benchmark, los experimentos aportan información útil: permiten ver en qué casos el enfoque funciona mejor, en qué casos pierde frente a una regla sencilla y qué limitaciones aparecen al llevar una idea matemática a un backtesting realista.

11.2. Cumplimiento de los objetivos

Los objetivos planteados inicialmente se han cumplido de forma general, aunque algunos se fueron concretando a medida que avanzaba el trabajo.

El primer objetivo era familiarizarse con los conceptos matemáticos necesarios para entender el Análisis Topológico de Datos. Este objetivo se ha cumplido mediante el estudio de conectividad, homotopía, complejos simpliciales, homología simplicial, filtraciones, homología persistente, códigos de barras, diagramas de persistencia y paisajes de persistencia. Estos contenidos forman la base matemática de la memoria.

El segundo objetivo era comprender la motivación y el funcionamiento general del TDA. Este objetivo también se ha alcanzado. A lo largo de la memoria se explica por qué puede ser útil mirar los datos desde un punto de vista topológico, especialmente cuando son de alta dimensión o difíciles de visualizar. También se ha visto que la homología persistente permite estudiar una nube de puntos a varias escalas, lo que evita depender de una única elección del parámetro de escala.

El tercer objetivo era entender el funcionamiento de Mapper y utilizarlo en un caso aplicado. Este objetivo se ha cumplido principalmente en la parte computacional. Mapper se ha usado para construir grafos que resumen relaciones entre activos financieros. Además, se han estudiado varias decisiones prácticas necesarias para aplicarlo: la función filtro, el recubrimiento, el clustering local y la forma de transformar el grafo en pesos de cartera.

El cuarto objetivo era aplicar las herramientas estudiadas a datos reales. Este objetivo se ha alcanzado utilizando dos conjuntos financieros: las 49 carteras industriales de Kenneth French y un universo de acciones del S&P 500 obtenido a partir de CRSP/WRDS. Estos datos han permitido probar el método en dos escenarios diferentes: uno formado por carteras sectoriales agregadas y otro formado por acciones individuales.

El quinto objetivo planteaba una posible comparación con técnicas clásicas. En la versión final del trabajo, esta comparación se ha concretado mediante una cartera equiponderada. Esta elección resulta adecuada porque la cartera $1/N$ es una referencia sencilla, transparente y exigente en problemas de asignación de activos. Además, PCA se ha utilizado dentro del propio pipeline, como parte de la construcción del *lens* de Mapper.

Además de los objetivos iniciales, durante el desarrollo se decidió dar mayor importancia a la parte informática mediante la creación de una librería Python. El objetivo era reunir las distintas etapas del trabajo en una herramienta que pudiera instalarse y utilizarse sin necesidad de conocer en profundidad toda la implementación interna. Esto llevó también a publicar el paquete en PyPI, generar documentación con Sphinx, preparar un manual de usuario y organizar los experimentos de forma reproducible.

11.3. Aprendizaje académico y profesional

Desde el punto de vista académico, este trabajo me ha permitido unir varias áreas que al principio parecían bastante separadas. Por una parte, ha sido necesario estudiar conceptos de topología algebraica, como homología y homología persistente. Por otra, ha hecho falta llevar esas ideas a un problema de análisis de datos y, finalmente, interpretarlas dentro de un contexto financiero.

Una de las partes más difíciles ha sido pasar de la teoría a la implementación. Entender una definición matemática es un primer paso, pero después hay que decidir cómo se representa en código, qué datos se necesitan, qué parámetros hay que fijar y cómo se interpreta la salida. Esto se ve claramente con la homología persistente: en la parte teórica se estudian diagramas y paisajes de persistencia, mientras que en la parte aplicada se usa una norma del paisaje como señal de régimen dentro de un backtest.

También ha sido importante comprobar que usar una técnica más avanzada no garantiza mejores resultados. Mapper y la homología persistente aportan una forma distinta de analizar los datos, pero su rendimiento depende del conjunto de datos, de la representación elegida, de los parámetros y de la regla usada para transformar la información topológica en una cartera. Esta idea aparece claramente al comparar los resultados de KF49 y S&P 500.

Desde el punto de vista profesional, el trabajo ha servido para desarrollar un proyecto

software completo. Ha sido necesario organizar el código en módulos, preparar una estructura de paquete, controlar dependencias, documentar funciones, generar documentación con Sphinx, publicar la librería en PyPI y preparar un manual de usuario. Todo este proceso ha sido útil para entender mejor cómo se organiza un proyecto reproducible y reutilizable.

La parte experimental también me ha servido para ver la importancia del protocolo de evaluación. En un problema financiero, pequeños detalles pueden cambiar mucho la interpretación de los resultados. Por eso ha sido necesario cuidar la validación temporal, evitar el uso de información futura, aplicar los mismos costes de transacción a todos los modelos y comparar siempre con una referencia sencilla. Esta parte me ha ayudado a entender que un buen resultado experimental depende tanto del modelo como del protocolo de evaluación.

A nivel personal, el desarrollo del TFG ha supuesto trabajar en un proyecto largo, con varias fases y con cambios respecto a la idea inicial. Al principio el objetivo era más general: estudiar TDA y aplicarlo a datos reales. Con el tiempo, el trabajo se fue concretando hasta centrarse en finanzas y en el desarrollo de una librería propia. Esto me ha ayudado a entender mejor cómo evoluciona un proyecto real: se parte de una idea inicial, se prueban opciones y poco a poco se va cerrando una dirección concreta.

11.4. Limitaciones

Los resultados obtenidos deben interpretarse con cautela. Las principales limitaciones del trabajo son las siguientes:

- **Número de universos financieros estudiados.** Solo se han utilizado dos universos financieros. KF49 y S&P 500 permiten comparar dos escenarios distintos, pero no bastan para sacar conclusiones generales sobre otros mercados, otros periodos o distintos tipos de activos.
- **Naturaleza de las carteras KF49.** Las 49 carteras industriales de Kenneth French son carteras académicas construidas a partir de datos de mercado. Son útiles para probar el método en un universo estable, pero no corresponden directamente a activos que se puedan comprar de forma inmediata. Por tanto, los resultados sobre KF49 deben entenderse como una prueba experimental.
- **Selección de hiperparámetros.** La configuración final de Mapper y de la señal de homología persistente se ha elegido tras un análisis de sensibilidad limitado. Se evitó hacer una búsqueda muy amplia de hiperparámetros para reducir el riesgo de ajustar demasiado el modelo a los periodos históricos utilizados. Aun así, podrían existir otras configuraciones con mejores resultados.
- **Simplificaciones del backtesting.** Aunque se incluyen costes de transacción proporcionales al turnover, el backtesting sigue siendo una simplificación de la realidad. Quedan fuera otros elementos importantes, como la liquidez de los activos, la diferencia entre precio de compra y venta, el posible impacto de mercado y aspectos fiscales.
- **Uso de la homología persistente como señal de régimen.** La señal de homología persistente se ha utilizado de una forma concreta: como mecanismo auxiliar de control de régimen. Los resultados muestran que puede aportar una pequeña mejora en KF49, pero en el S&P 500 empeora ligeramente a Mapper. Esto indica que la señal PH necesita más estudio antes de considerarla una mejora robusta del modelo.

11.5. Vías futuras

Una primera vía de continuación sería ampliar la evaluación a otros conjuntos de datos. En esta memoria se han utilizado KF49 y S&P 500, que permiten comparar un universo sectorial agregado con un universo amplio de acciones individuales. Como continuación natural, se podrían estudiar otros mercados, otros periodos temporales o distintas frecuencias de datos. Esto ayudaría a comprobar si las conclusiones obtenidas dependen mucho de los datos elegidos o si se mantienen en contextos diferentes.

Otra línea de trabajo sería profundizar en la forma de transformar el grafo Mapper en pesos de cartera. En este TFG se han probado dos reglas concretas: macrocluster y solapamiento de nodos. Los resultados muestran que esta decisión influye en el comportamiento final de la estrategia. Por tanto, una continuación razonable sería estudiar con más detalle esta parte del pipeline y comparar nuevas reglas de asignación de pesos, manteniendo siempre un esquema de backtesting causal.

También sería interesante analizar con más detalle el papel de la homología persistente. En este trabajo se ha utilizado como una señal auxiliar de régimen basada en paisajes de persistencia, siguiendo la idea de que ciertas magnitudes topológicas pueden reflejar cambios en la estructura del mercado [20]. Los resultados muestran que esta señal mejora ligeramente el comportamiento en KF49, pero no en el S&P 500. Por ello, una vía futura sería estudiar cuándo esta señal aporta información útil y cómo debería traducirse en una decisión de cartera.

Desde el punto de vista experimental, sería conveniente ampliar la comparación con otros métodos de referencia. En esta memoria se ha utilizado la cartera equiponderada porque es una regla sencilla y difícil de superar de forma consistente fuera de muestra [13]. En trabajos posteriores podrían añadirse otras referencias clásicas de construcción de carteras, como modelos de media-varianza inspirados en Markowitz [32]. Esto permitiría situar mejor el comportamiento de Mapper frente a un conjunto más amplio de métodos financieros.

Por último, desde el punto de vista informático, la librería podría seguir desarrollándose para facilitar su uso. Algunas mejoras naturales serían añadir más ejemplos, ampliar la documentación, incorporar más pruebas automáticas y simplificar la ejecución de experimentos propios. De esta forma, el paquete podría ser más útil para otros usuarios interesados en aplicar TDA a datos financieros.

En resumen, este trabajo deja una base sobre la que se puede continuar. Se ha estudiado la teoría necesaria, se ha desarrollado una librería funcional y se han realizado experimentos sobre datos financieros reales. Las vías futuras más claras consisten en probar el método en más datos, estudiar mejor las decisiones del pipeline y seguir mejorando la librería para que sea más fácil de usar y extender.

A. Resultados auxiliares de la evolución del proyecto

En este apéndice se recogen las tablas completas que complementan el análisis del capítulo 9. Se detallan los resultados por periodo y las configuraciones evaluadas en el estudio del recubrimiento.

A.1. Comparación completa entre DBSCAN y HACA

Dataset	Periodo	Método	Ret. anual.	Vol. anual.	Sharpe	Sortino	MDD
KF49	1975–1984	DBSCAN	17.34 %	16.92 %	0.528	0.848	-17.79 %
KF49	1985–1994	DBSCAN	13.63 %	16.83 %	0.517	0.725	-31.57 %
KF49	1995–2004	DBSCAN	12.95 %	15.40 %	0.624	0.907	-24.55 %
KF49	2005–2014	DBSCAN	8.87 %	18.15 %	0.484	0.691	-55.51 %
KF49	2015–2025	DBSCAN	11.27 %	16.89 %	0.608	0.942	-24.48 %
KF49	1975–1984	HACA	17.78 %	17.07 %	0.547	0.881	-18.79 %
KF49	1985–1994	HACA	13.75 %	16.20 %	0.537	0.751	-30.92 %
KF49	1995–2004	HACA	13.69 %	13.96 %	0.720	1.050	-23.44 %
KF49	2005–2014	HACA	9.00 %	16.51 %	0.522	0.767	-47.92 %
KF49	2015–2025	HACA	13.03 %	16.29 %	0.721	1.160	-23.87 %
S&P 500	1975–1984	DBSCAN	15.32 %	12.03 %	0.540	0.880	-11.06 %
S&P 500	1985–1994	DBSCAN	8.50 %	9.48 %	0.318	0.440	-18.36 %
S&P 500	1995–2004	DBSCAN	11.56 %	10.91 %	0.709	1.138	-15.57 %
S&P 500	2005–2014	DBSCAN	5.81 %	10.69 %	0.451	0.609	-42.99 %
S&P 500	2015–2025	DBSCAN	6.18 %	11.17 %	0.427	0.613	-22.81 %
S&P 500	1975–1984	HACA	13.19 %	10.13 %	0.435	0.652	-10.15 %
S&P 500	1985–1994	HACA	7.95 %	8.69 %	0.279	0.374	-17.24 %
S&P 500	1995–2004	HACA	10.39 %	9.19 %	0.709	1.073	-13.04 %
S&P 500	2005–2014	HACA	5.12 %	8.63 %	0.458	0.627	-33.23 %
S&P 500	2015–2025	HACA	6.18 %	8.93 %	0.506	0.731	-14.39 %

Tabla A.1.: Comparación completa por periodo entre DBSCAN y HACA.

A.2. Comparación completa entre macrocluster y solapamiento de nodos

Dataset	Periodo	Método	Ret. anual.	Vol. anual.	Sharpe	Sortino	MDD
KF49	1975–1984	Macrocluster	18.43 %	17.29 %	0.574	0.933	-19.74 %
KF49	1985–1994	Macrocluster	14.42 %	15.83 %	0.583	0.815	-28.68 %
KF49	1995–2004	Macrocluster	14.20 %	13.66 %	0.765	1.142	-21.22 %
KF49	2005–2014	Macrocluster	8.77 %	16.44 %	0.510	0.754	-47.56 %
KF49	2015–2025	Macrocluster	11.96 %	16.11 %	0.668	1.081	-24.35 %
KF49	1975–1984	Node overlap	17.78 %	17.07 %	0.547	0.881	-18.79 %
KF49	1985–1994	Node overlap	13.75 %	16.20 %	0.537	0.751	-30.92 %
KF49	1995–2004	Node overlap	13.69 %	13.96 %	0.720	1.050	-23.44 %
KF49	2005–2014	Node overlap	9.00 %	16.51 %	0.522	0.767	-47.92 %
KF49	2015–2025	Node overlap	13.03 %	16.29 %	0.721	1.160	-23.87 %
S&P 500	1975–1984	Macrocluster	11.09 %	8.74 %	0.277	0.390	-8.42 %
S&P 500	1985–1994	Macrocluster	6.36 %	7.75 %	0.111	0.143	-15.63 %
S&P 500	1995–2004	Macrocluster	10.03 %	8.78 %	0.702	1.059	-12.76 %
S&P 500	2005–2014	Macrocluster	4.18 %	7.87 %	0.381	0.506	-31.58 %
S&P 500	2015–2025	Macrocluster	5.45 %	7.92 %	0.472	0.666	-13.60 %
S&P 500	1975–1984	Node overlap	13.19 %	10.13 %	0.435	0.652	-10.15 %
S&P 500	1985–1994	Node overlap	7.95 %	8.69 %	0.279	0.374	-17.24 %
S&P 500	1995–2004	Node overlap	10.39 %	9.19 %	0.709	1.073	-13.04 %
S&P 500	2005–2014	Node overlap	5.12 %	8.63 %	0.458	0.627	-33.23 %
S&P 500	2015–2025	Node overlap	6.18 %	8.93 %	0.506	0.731	-14.39 %

Tabla A.2.: Comparación completa por periodo entre macrocluster y solapamiento de nodos.

A.3. Comparación completa entre liquidez y cartera equiponderada

Dataset	Periodo	Método	Ret. anual.	Vol. anual.	Sharpe	Sortino	MDD
KF49	1975–1984	Liquidez	19.27 %	15.47 %	0.668	1.164	-15.25 %
KF49	1985–1994	Liquidez	11.98 %	15.52 %	0.452	0.633	-30.92 %
KF49	1995–2004	Liquidez	10.96 %	13.57 %	0.556	0.797	-21.90 %
KF49	2005–2014	Liquidez	8.07 %	15.97 %	0.480	0.704	-46.26 %
KF49	2015–2025	Liquidez	12.01 %	14.86 %	0.714	1.207	-23.87 %
KF49	1975–1984	Equal-weight	17.97 %	16.96 %	0.559	0.910	-19.09 %
KF49	1985–1994	Equal-weight	14.13 %	16.25 %	0.556	0.776	-30.92 %
KF49	1995–2004	Equal-weight	13.87 %	13.90 %	0.734	1.065	-22.58 %
KF49	2005–2014	Equal-weight	9.20 %	16.60 %	0.531	0.780	-48.42 %
KF49	2015–2025	Equal-weight	12.33 %	16.32 %	0.682	1.080	-23.87 %
S&P 500	1975–1984	Liquidez	11.50 %	9.65 %	0.297	0.445	-10.15 %
S&P 500	1985–1994	Liquidez	6.55 %	8.44 %	0.131	0.172	-17.24 %
S&P 500	1995–2004	Liquidez	10.13 %	8.93 %	0.699	1.072	-13.02 %
S&P 500	2005–2014	Liquidez	3.09 %	8.12 %	0.242	0.322	-28.46 %
S&P 500	2015–2025	Liquidez	6.07 %	8.66 %	0.506	0.735	-14.39 %
S&P 500	1975–1984	Equal-weight	13.00 %	10.53 %	0.406	0.606	-10.29 %
S&P 500	1985–1994	Equal-weight	7.59 %	8.94 %	0.237	0.314	-17.24 %
S&P 500	1995–2004	Equal-weight	10.32 %	9.34 %	0.694	1.053	-14.22 %
S&P 500	2005–2014	Equal-weight	5.40 %	8.80 %	0.482	0.664	-34.03 %
S&P 500	2015–2025	Equal-weight	5.62 %	9.29 %	0.433	0.605	-14.39 %

Tabla A.3.: Comparación completa por periodo entre liquidez y cartera equiponderada como acción defensiva.

A.4. Resultados del estudio de sensibilidad del recubrimiento

n_cubes	perc_overlap	Ret. anual.	Vol. anual.	Sharpe	Sortino	MDD
8	0.15	13.06 %	15.80 %	0.594	0.896	-29.41 %
8	0.25	13.05 %	15.84 %	0.591	0.896	-29.17 %
8	0.40	12.95 %	15.74 %	0.588	0.883	-28.83 %
12	0.15	13.27 %	16.00 %	0.600	0.902	-29.27 %
12	0.25	13.45 %	16.01 %	0.609	0.922	-28.99 %
12	0.40	13.27 %	15.86 %	0.605	0.915	-29.45 %
16	0.15	13.33 %	15.89 %	0.608	0.920	-29.07 %
16	0.25	13.17 %	15.85 %	0.600	0.907	-29.13 %
16	0.40	12.91 %	15.91 %	0.582	0.877	-29.89 %

Tabla A.4.: Resultados medios del estudio de sensibilidad del recubrimiento en KF49.

n_cubes	perc_overlap	Ret. anual.	Vol. anual.	Sharpe	Sortino	MDD
8	0.15	8.35 %	9.02 %	0.460	0.665	-17.18 %
8	0.25	8.55 %	9.07 %	0.480	0.697	-17.38 %
8	0.40	8.62 %	9.09 %	0.485	0.702	-17.62 %
12	0.15	8.51 %	9.20 %	0.468	0.673	-18.11 %
12	0.25	8.56 %	9.11 %	0.478	0.692	-17.61 %
12	0.40	8.52 %	8.99 %	0.478	0.689	-17.32 %
16	0.15	8.76 %	9.16 %	0.495	0.717	-17.34 %
16	0.25	8.68 %	9.17 %	0.488	0.705	-17.46 %
16	0.40	8.49 %	9.12 %	0.470	0.679	-17.55 %

Tabla A.5.: Resultados medios del estudio de sensibilidad del recubrimiento en S&P 500.

B. Resultados completos de la comparación final

Este apéndice recoge las tablas completas por periodo correspondientes a la comparación final entre Mapper, Mapper con homología persistente y la cartera equiponderada. En el cuerpo principal de la memoria se han incluido únicamente los resultados medios y las figuras principales, para no sobrecargar la exposición.

B.1. Resultados completos por periodo en KF49

Tabla B.1.: Resultados completos por periodo para KF49.

Periodo	Modelo	Ret. anual.	Sharpe	Sortino	MDD	NAV final
1975-1984	Mapper	17.78 %	0.547	0.881	-18.79 %	5.14
1985-1994	Mapper	13.75 %	0.537	0.751	-30.92 %	3.63
1995-2004	Mapper	13.69 %	0.720	1.050	-23.44 %	3.61
2005-2014	Mapper	9.00 %	0.522	0.767	-47.92 %	2.37
2015-2025	Mapper	13.03 %	0.721	1.160	-23.87 %	3.81
1975-1984	Mapper + PH	17.97 %	0.559	0.910	-19.09 %	5.22
1985-1994	Mapper + PH	14.13 %	0.556	0.776	-30.92 %	3.75
1995-2004	Mapper + PH	13.87 %	0.734	1.065	-22.58 %	3.66
2005-2014	Mapper + PH	9.20 %	0.531	0.780	-48.42 %	2.41
2015-2025	Mapper + PH	12.33 %	0.682	1.080	-23.87 %	3.56
1975-1984	Equal-weight	17.36 %	0.529	0.856	-18.76 %	4.96
1985-1994	Equal-weight	13.59 %	0.520	0.727	-31.44 %	3.58
1995-2004	Equal-weight	13.46 %	0.682	1.002	-23.01 %	3.54
2005-2014	Equal-weight	9.32 %	0.519	0.750	-52.63 %	2.44
2015-2025	Equal-weight	11.47 %	0.625	0.968	-25.94 %	3.27

B.2. Resultados completos por periodo en S&P 500

Tabla B.2.: Resultados completos por periodo para S&P 500.

Periodo	Modelo	Ret. anual.	Sharpe	Sortino	MDD	NAV final
1975-1984	Mapper	13.28 %	0.446	0.677	-8.98 %	3.48
1985-1994	Mapper	7.98 %	0.280	0.374	-17.36 %	2.15
1995-2004	Mapper	10.81 %	0.747	1.137	-12.74 %	2.79

Continued on next page

A. Resultados auxiliares de la evolución del proyecto

Tabla B.2.: Resultados completos por periodo para S&P 500.

Periodo	Modelo	Ret. anual.	Sharpe	Sortino	MDD	NAV final
2005–2014	Mapper	5.33 %	0.475	0.643	-34.11 %	1.68
2015–2025	Mapper	6.39 %	0.525	0.755	-13.51 %	1.97
1975–1984	Mapper + PH	13.23 %	0.429	0.646	-10.73 %	3.46
1985–1994	Mapper + PH	7.70 %	0.248	0.328	-17.36 %	2.10
1995–2004	Mapper + PH	10.90 %	0.746	1.142	-14.56 %	2.81
2005–2014	Mapper + PH	5.70 %	0.505	0.692	-34.84 %	1.74
2015–2025	Mapper + PH	5.76 %	0.446	0.620	-14.54 %	1.84
1975–1984	Equal-weight	15.71 %	0.535	0.875	-10.80 %	4.30
1985–1994	Equal-weight	11.80 %	0.497	0.701	-23.33 %	3.05
1995–2004	Equal-weight	11.87 %	0.737	1.132	-17.75 %	3.07
2005–2014	Equal-weight	7.57 %	0.515	0.751	-44.76 %	2.07
2015–2025	Equal-weight	9.04 %	0.589	0.898	-21.85 %	2.57

C. Manual de usuario

Este apéndice recoge una guía práctica de uso de la librería desarrollada en el trabajo. El objetivo es que un usuario externo pueda instalar el paquete, preparar sus propios datos, ejecutar las estrategias principales y obtener resultados de backtesting sin necesidad de revisar el código fuente de la librería.

La librería `tda-finance-mapper` está pensada para aplicar herramientas de análisis topológico de datos a series financieras. En concreto, permite construir carteras basadas en Mapper, añadir un control de régimen mediante homología persistente y comparar los resultados con una cartera equiponderada.

El paquete se utiliza como una librería de Python. Por tanto, el usuario crea un script de Python, importa las funciones del paquete y las aplica a una matriz de precios.

C.1. Formas de uso

Existen dos formas principales de utilizar el proyecto:

- instalar el paquete desde PyPI y ejecutar un experimento propio con datos externos;
- clonar el repositorio para inspeccionar el código, modificarlo o reproducir los experimentos de la memoria.

Para un usuario que quiera probar el método con sus propios datos, la primera opción es la más sencilla.

C.2. Uso del paquete con datos propios

C.2.1. Crear una carpeta de trabajo

En primer lugar, se recomienda crear una carpeta independiente para el experimento. Por ejemplo:

```
my_tda_experiment/  
├── data/  
│   └── input_data.csv  
├── results/  
└── run_my_experiment.py
```

El archivo `input_data.csv` contiene los datos de entrada. El archivo `run_my_experiment.py` contiene el código que carga los datos, ejecuta las estrategias y guarda los resultados.

Todos los comandos de esta sección deben ejecutarse desde la carpeta `my_tda_experiment/`.

C.2.2. Crear un entorno virtual

Se recomienda utilizar un entorno virtual para aislar las dependencias del proyecto:

C. Manual de usuario

```
python -m venv .venv
```

En Windows, se activa con:

```
.venv\Scripts\activate
```

En macOS o Linux, se activa con:

```
source .venv/bin/activate
```

C.2.3. Instalar el paquete

Con el entorno virtual activado, el paquete puede instalarse desde PyPI mediante:

```
pip install tda-finance-mapper
```

Para comprobar que la instalación se ha realizado correctamente, puede ejecutarse:

```
python -c "import tda_finance; print('tda_finance imported successfully')"
```

Debe tenerse en cuenta que el nombre del paquete en PyPI es `tda-finance-mapper`, mientras que en Python se importa como `tda_finance`.

C.3. Formato de los datos de entrada

Las funciones de backtesting esperan recibir una matriz de precios o un índice de precios. Esta matriz debe tener fechas en las filas, activos en las columnas y valores numéricos.

En la práctica, los datos financieros pueden aparecer en dos formatos habituales: precios o retornos. Ambos pueden utilizarse, pero si el archivo contiene retornos, estos deben convertirse previamente en un índice de precios.

C.3.1. Caso 1: el archivo contiene precios

Un archivo de precios puede tener la siguiente estructura:

```
date,AAPL,MSFT,JPM
2015-01-31,100.0,100.0,100.0
2015-02-28,101.2,98.7,102.5
2015-03-31,99.8,97.1,104.0
2015-04-30,103.4,99.5,105.2
```

En este caso, los datos pueden cargarse directamente como precios.

C.3.2. Caso 2: el archivo contiene retornos

Algunos conjuntos de datos financieros se distribuyen como retornos. Por ejemplo, las 49 carteras industriales de Kenneth French suelen proporcionarse como retornos mensuales.

Un archivo de retornos en formato decimal puede tener esta forma:


```
date,Agric,Food,Soda
1970-01-01,0.0101,-0.0277,-0.0279
1970-02-01,0.0993,0.0601,0.0388
1970-03-01,-0.1339,-0.0056,-0.0110
```

Aquí, el valor 0.0101 representa un retorno del 1.01%. Para utilizar estos datos en el backtesting, deben convertirse en un índice de precios:

```
prices = (1.0 + returns).cumprod() * 100.0
```

Si los retornos están expresados en porcentaje, por ejemplo 1.01 en lugar de 0.0101, primero deben dividirse entre 100:

```
returns = returns / 100.0
prices = (1.0 + returns).cumprod() * 100.0
```

Las tablas anteriores son solo ejemplos de formato. Un backtesting real necesita suficientes observaciones. Por ejemplo, con datos mensuales y `lookback_days=60`, la estrategia necesita al menos 60 meses de histórico antes del primer rebalanceo.

C.4. Script de ejemplo

El siguiente script ejecuta las tres estrategias principales: Mapper, Mapper con control de régimen mediante homología persistente y cartera equiponderada.

El usuario debe crear un archivo llamado `run_my_experiment.py` dentro de la carpeta de trabajo y pegar el siguiente código:

```
import os

import pandas as pd

from tda_finance.portfolio.backtest_engine import (
    backtest_equal_weight_rebalanced,
    backtest_tda,
    perf_summary,
)
from tda_finance.tda.mapper_clustering import MapperParams

# -----
# 1. Load input data
# -----

DATA_FILE = "data/input_data.csv"

# Set this to True if the CSV contains returns.
# Set this to False if the CSV already contains prices.
DATA_ARE_RETURNS = True

# Set this to True if returns are written as percentages, for example 1.01.
```

C. Manual de usuario

```
# Set this to False if returns are written as decimals, for example 0.0101.
RETURNS_ARE_PERCENTAGES = False

data = pd.read_csv(
    DATA_FILE,
    index_col=0,
    parse_dates=True,
)

data = data.sort_index()
data = data.astype(float)

print("Loaded input data:")
print(data.head())
print("Input shape:", data.shape)

if DATA_ARE_RETURNS:
    returns = data.copy()

    if RETURNS_ARE_PERCENTAGES:
        returns = returns / 100.0

    if (returns <= -1.0).any().any():
        raise ValueError(
            "Some returns are lower than or equal to -100%. "
            "Check whether the input data are really returns."
        )

    # The package expects prices. Returns are converted into a price index
    # starting at 100.
    prices = (1.0 + returns).cumprod() * 100.0

else:
    prices = data.copy()

    if (prices <= 0.0).any().any():
        raise ValueError(
            "Some prices are lower than or equal to zero. "
            "Check the input price matrix."
        )

print("\nPrice matrix used by the backtest:")
print(prices.head())
print("Price shape:", prices.shape)

# -----
# 2. Define Mapper parameters
# -----
```

```

params = MapperParams(
    pca_var=0.80,
    umap_dim=1,
    n_cubes=12,
    perc_overlap=0.25,
    clusterer="haca",
    haca_distance_threshold=0.6,
    haca_linkage="average",
    random_state=1,
)

# -----
# 3. Run Mapper
# -----

mapper_result = backtest_tda(
    prices=prices,
    lookback_days=60,
    rebalance_days=3,
    params=params,
    tc_bps=5.0,
    use_ph_control=False,
)

mapper_metrics = perf_summary(
    mapper_result["port_ret"],
    periods_per_year=12,
)

# -----
# 4. Run Mapper with PH regime control
# -----

mapper_ph_result = backtest_tda(
    prices=prices,
    lookback_days=60,
    rebalance_days=3,
    params=params,
    tc_bps=5.0,
    use_ph_control=True,
    regime_action="equal_weight",
)

mapper_ph_metrics = perf_summary(
    mapper_ph_result["port_ret"],
    periods_per_year=12,
)

```

```
# -----
# 5. Run equal-weight benchmark
# -----

eqw_result = backtest_equal_weight_rebalanced(
    prices=prices,
    lookback_days=60,
    rebalance_days=3,
    tc_bps=5.0,
)

eqw_metrics = perf_summary(
    eqw_result["port_ret"],
    periods_per_year=12,
)

# -----
# 6. Save results
# -----

os.makedirs("results", exist_ok=True)

comparison = pd.DataFrame(
    [
        {"model": "Mapper", **mapper_metrics},
        {"model": "Mapper + PH", **mapper_ph_metrics},
        {"model": "Equal-weight", **eqw_metrics},
    ]
)

comparison.to_csv("results/comparison_metrics.csv", index=False)

nav_curves = pd.DataFrame(
    {
        "Mapper": mapper_result["port_nav"],
        "Mapper + PH": mapper_ph_result["port_nav"],
        "Equal-weight": eqw_result["port_nav"],
    }
)

nav_curves.to_csv("results/nav_curves.csv")

mapper_result.to_csv("results/mapper_result.csv")
mapper_ph_result.to_csv("results/mapper_ph_result.csv")
eqw_result.to_csv("results/equal_weight_result.csv")

print("\nPerformance comparison:")
```

```
print(comparison)
```

```
print("\nResults saved in the 'results/' folder.")
```

Si el archivo de entrada contiene precios, debe utilizarse:

```
DATA_ARE_RETURNS = False
```

Si el archivo contiene retornos en formato decimal, por ejemplo 0.0101 para representar 1.01 %, debe utilizarse:

```
DATA_ARE_RETURNS = True
RETURNS_ARE_PERCENTAGES = False
```

Si el archivo contiene retornos en porcentaje, por ejemplo 1.01 para representar 1.01 %, debe utilizarse:

```
DATA_ARE_RETURNS = True
RETURNS_ARE_PERCENTAGES = True
```

En el ejemplo se utiliza `periods_per_year=12`, porque se asume que los datos son mensuales. Con datos diarios, lo habitual sería utilizar `periods_per_year=252`. Además, los parámetros `lookback_days` y `rebalance_days` deben interpretarse como número de observaciones. Por tanto, con datos mensuales, `lookback_days=60` representa 60 meses y `rebalance_days=3` representa un rebalanceo cada tres meses.

C.5. Ejecución del experimento

Desde la carpeta de trabajo, el script se ejecuta con:

```
python run_my_experiment.py
```

Después de la ejecución, se genera la carpeta `results/` con los siguientes archivos:

```
results/
├── comparison_metrics.csv
├── nav_curves.csv
├── mapper_result.csv
├── mapper_ph_result.csv
└── equal_weight_result.csv
```

C.6. Interpretación de los resultados

El archivo `comparison_metrics.csv` contiene una fila por modelo y una columna por métrica. Las métricas principales son:

El archivo `nav_curves.csv` contiene la evolución acumulada de cada estrategia. La curva NAV empieza en 1. Un valor final mayor que 1 indica que la estrategia ha aumentado de valor durante el periodo evaluado.

La NAV permite comparar la evolución acumulada de las estrategias, pero debe interpretarse junto con otras métricas. Una estrategia puede terminar con menor rentabilidad acumulada y, aun así, presentar menor volatilidad o menor drawdown.

Métrica	Interpretación
total_return	Retorno total durante el periodo evaluado.
ann_return_geo	Retorno anualizado geométrico.
ann_vol	Volatilidad anualizada.
sharpe	Rentabilidad ajustada por volatilidad total.
sortino	Rentabilidad ajustada por volatilidad negativa.
max_dd	Máxima caída desde un máximo anterior de la cartera.

Tabla C.1.: Métricas principales generadas por el script de ejemplo.

C.7. Representación gráfica de las curvas NAV

Para representar las curvas NAV, puede crearse un segundo archivo llamado `plot_results.py`:

```
import pandas as pd
import matplotlib.pyplot as plt

nav = pd.read_csv(
    "results/nav_curves.csv",
    index_col=0,
    parse_dates=True,
)

nav.plot()
plt.title("Portfolio NAV comparison")
plt.xlabel("Date")
plt.ylabel("NAV")
plt.tight_layout()
plt.show()
```

El gráfico se genera ejecutando:

```
python plot_results.py
```

Este gráfico permite comparar visualmente la evolución acumulada de Mapper, Mapper con control PH y la cartera equiponderada.

C.8. Uso del repositorio

El repositorio puede utilizarse si se desea inspeccionar el código fuente, modificarlo o reproducir los experimentos de la memoria.

Para clonar el repositorio:

```
git clone https://github.com/jaimecz3/tda-finance-mapper.git
cd tda-finance-mapper
```

Después se crea y activa un entorno virtual:

```
python -m venv .venv
```

En Windows:

```
.venv\Scripts\activate
```

En macOS o Linux:

```
source .venv/bin/activate
```

La instalación en modo editable se realiza con:

```
pip install -e .
```

Y las dependencias del proyecto pueden instalarse mediante:

```
pip install -r requirements.txt
```

Dentro del repositorio también pueden ejecutarse experimentos propios. Por ejemplo:

```
tda-finance-mapper/  
├── data/  
│   └── input_data.csv  
├── results/  
├── scripts/  
│   └── run_my_experiment.py
```

En ese caso, el script debe ejecutarse desde la raíz del repositorio:

```
python scripts/run_my_experiment.py
```

Esto es importante porque la ruta `data/input_data.csv` se interpreta respecto a la carpeta desde la que se lanza el comando.

C.9. Reproducción de los experimentos de la memoria

El repositorio incluye un script diseñado para reproducir los experimentos finales de la memoria:

```
python -m tda_finance.experiments.run_mapper_ph_experiments
```

Este script compara Mapper, Mapper con control de régimen mediante homología persistente y la cartera equiponderada. Está preparado para los conjuntos de datos utilizados en el trabajo: las 49 carteras industriales de Kenneth French y el conjunto S&P 500/CRSP.

Por tanto, este script no debe interpretarse como una entrada genérica para cualquier conjunto de datos. Para utilizar datos nuevos, se recomienda preparar una matriz de entrada propia y seguir el flujo descrito en las secciones anteriores.

C.10. Parámetros principales

Los parámetros de Mapper se configuran mediante `MapperParams`. Los más relevantes son:

Los parámetros principales del backtesting son:

Parámetro	Significado
pca_var	Varianza explicada retenida en PCA antes de aplicar UMAP.
umap_dim	Dimensión de salida de UMAP utilizada para construir la función filtro.
n_cubes	Número de intervalos del recubrimiento de Mapper.
perc_overlap	Porcentaje de solapamiento entre intervalos.
clusterer	Método de clustering local. En los experimentos finales se utiliza haca.
haca_distance_threshold	Umbral de distancia para el clustering jerárquico aglomerativo.
haca_linkage	Criterio de enlace utilizado en el clustering jerárquico.
dbscan_eps	Parámetro de radio utilizado por DBSCAN si se elige clusterer="dbscan".
dbscan_min_samples	Número mínimo de observaciones utilizado por DBSCAN.
random_state	Semilla utilizada para reproducibilidad.
min_assets	Número mínimo de activos necesarios en una ventana.

Tabla C.2.: Parámetros principales de MapperParams.

Parámetro	Significado
lookback_days	Número de observaciones utilizadas como ventana histórica.
rebalance_days	Número de observaciones entre dos rebalanceos consecutivos.
tc_bps	Coste de transacción en puntos básicos, aplicado proporcionalmente al turnover.
use_ph_control	Indica si se activa el control de régimen mediante homología persistente.
regime_action	Acción defensiva cuando PH detecta un régimen anómalo. Los valores admitidos son cash y equal_weight.
weight_method	Regla utilizada para transformar los clústeres de Mapper en pesos de cartera. Los valores admitidos son node_overlap y macro_cluster.

Tabla C.3.: Parámetros principales del backtesting.

C.11. Descripción de las estrategias

C.11.1. Mapper

La estrategia Mapper construye una nube de puntos a partir de los retornos recientes de los activos. Cada punto representa un activo y sus coordenadas son los retornos observados en la ventana histórica. A partir de esta nube se construye un grafo Mapper, que posteriormente se transforma en pesos de cartera.

C.11.2. Mapper con control PH

La variante Mapper con PH utiliza Mapper como método principal de construcción de cartera y añade una señal de régimen basada en homología persistente. Si esta señal detecta un régimen anómalo, la estrategia aplica una acción defensiva. En los ejemplos anteriores, esta acción defensiva es una cartera equiponderada.

C.11.3. Cartera equiponderada

La cartera equiponderada asigna el mismo peso a todos los activos disponibles en cada fecha de rebalanceo. Se utiliza como benchmark sencillo y transparente.

C.12. Problemas frecuentes

C.12.1. ModuleNotFoundError: No module named 'tda_finance'

Este error suele indicar que el paquete no está instalado en el entorno de Python activo. Debe activarse el entorno virtual correcto y volver a instalar el paquete:

```
pip install tda-finance-mapper
```

o, si se trabaja desde el repositorio:

```
pip install -e .
```

C.12.2. Serie temporal demasiado corta

Si el conjunto de datos tiene pocas observaciones para el valor seleccionado de `lookback_days`, el backtesting no podrá comenzar. Por ejemplo, con datos mensuales y `lookback_days=60`, se necesitan al menos 60 meses de histórico antes del primer rebalanceo.

C.12.3. Ruta incorrecta al archivo de datos

Si Python no encuentra el archivo `data/input_data.csv`, debe comprobarse desde qué carpeta se está ejecutando el script. En el uso con PyPI, el comando debe lanzarse desde la carpeta `my_tda_experiment/`. En el uso desde el repositorio, si el script está en `scripts/`, debe lanzarse desde la raíz del repositorio.

C.12.4. Resultados inesperados

Si aparecen valores `inf`, `NaN` o métricas imposibles, conviene comprobar que:

- las fechas se han leído correctamente;
- los datos están ordenados temporalmente;
- las columnas representan activos;
- la matriz pasada al backtesting representa precios o un índice de precios;
- si el archivo original contiene retornos, estos se han convertido correctamente;
- la frecuencia de rebalanceo es coherente con la frecuencia de los datos;
- los costes de transacción se han fijado correctamente;
- existen suficientes observaciones para la ventana histórica elegida.

C.13. Advertencia sobre uso financiero

La librería se ha desarrollado con finalidad académica y experimental. Los resultados obtenidos mediante el paquete no deben interpretarse como recomendación de inversión. Cualquier uso operativo requeriría una validación adicional, análisis de liquidez, modelización realista de costes de transacción y evaluación fuera de muestra más amplia.

Glosario

Conceptos matemáticos y topológicos

Análisis Topológico de Datos (TDA) Conjunto de técnicas que utilizan herramientas de la topología algebraica para estudiar la forma global de los datos. En este trabajo se emplea para analizar nubes de puntos y extraer información topológica mediante complejos simpliciales, homología persistente y Mapper.

Nube de puntos Conjunto finito de puntos en un espacio métrico o euclídeo. En TDA suele representar una muestra discreta de un objeto geométrico o de una estructura subyacente que se quiere estudiar.

Camino Aplicación continua $\gamma: [0,1] \rightarrow X$. Se interpreta como una forma continua de unir dos puntos de un espacio topológico X .

Conexidad por caminos Propiedad de un espacio topológico según la cual cualesquiera dos puntos pueden unirse mediante un camino. Las clases de equivalencia determinadas por esta relación se llaman componentes conexas por caminos.

Homotopía Deformación continua entre dos aplicaciones continuas. Si dos aplicaciones $f, g: X \rightarrow Y$ son homotópicas, se escribe $f \simeq g$.

Equivalencia homotópica Relación entre dos espacios topológicos que tienen la misma forma desde el punto de vista de la homotopía. Es menos restrictiva que el homeomorfismo y conserva invariantes como la homología.

Espacio contráctil Espacio topológico homotópicamente equivalente a un punto. Intuitivamente, es un espacio que puede deformarse continuamente hasta un único punto.

Símplice Generalización de puntos, segmentos, triángulos y tetraedros a dimensión arbitraria. Un 0-símplice es un punto, un 1-símplice es un segmento, un 2-símplice es un triángulo y un 3-símplice es un tetraedro.

Complejo simplicial Colección de símlices unidos de forma compatible. Sirve para construir objetos combinatorios sobre los que se puede calcular homología.

Complejo simplicial abstracto Familia de subconjuntos finitos de un conjunto de vértices, cerrada por inclusión. Si un conjunto pertenece al complejo, entonces todos sus subconjuntos también pertenecen al complejo.

Realización geométrica Espacio topológico obtenido al representar geométricamente un complejo simplicial abstracto. Se suele denotar por $|K|$.

Cadena Combinación lineal formal de símlices de una misma dimensión.

Operador borde Aplicación que asocia a cada símplex la suma alternada de sus caras de codimensión uno. Se denota por $\partial_n: C_n(K) \rightarrow C_{n-1}(K)$.

Ciclo Cadena cuyo borde es nulo. El grupo de n -ciclos se denota por $Z_n(K) = \ker \partial_n$.

Borde Cadena que aparece como borde de una cadena de dimensión superior. El grupo de n -bordes se denota por $B_n(K) = \operatorname{im} \partial_{n+1}$.

Grupo de homología Cociente

$$H_n(K) = Z_n(K) / B_n(K).$$

Mide las clases de ciclos que no son bordes, es decir, rasgos topológicos que no quedan rellenos por simplices de dimensión superior. Con coeficientes en un cuerpo, puede tratarse como un espacio vectorial.

Número de Betti Dimensión del grupo de homología con coeficientes en un cuerpo:

$$\beta_n = \dim_{\mathbb{F}} H_n(K; \mathbb{F}).$$

En bajas dimensiones, β_0 cuenta componentes conexas, β_1 cuenta clases independientes de ciclos y β_2 cuenta clases independientes de cavidades.

Nervio Complejo simplicial que registra qué subfamilias tienen intersección común no vacía dentro de una familia de conjuntos. Cada conjunto de la familia da lugar a un vértice, y una intersección común no vacía da lugar a un simplex.

Teorema del nervio Resultado que, bajo ciertas hipótesis, afirma que el nervio de una familia de conjuntos y la unión de dichos conjuntos tienen el mismo tipo de homotopía. Es una de las justificaciones teóricas del uso de complejos simpliciales en TDA.

Complejo de Čech Complejo simplicial obtenido como nervio de una familia de bolas centradas en los puntos de una nube de datos.

Complejo de Vietoris–Rips Complejo simplicial en el que un conjunto de puntos forma un simplex cuando todos sus pares están suficientemente cerca. Es muy utilizado en TDA por su sencillez computacional, ya que puede construirse a partir de distancias por pares.

Filtración Sucesión creciente de complejos simpliciales

$$K_0 \subseteq K_1 \subseteq \cdots \subseteq K_m.$$

Permite estudiar cómo cambia la topología al variar un parámetro, normalmente una escala.

Homología persistente Técnica que estudia cómo nacen y mueren las clases de homología a lo largo de una filtración. Permite distinguir rasgos topológicos persistentes de rasgos efímeros o asociados al ruido.

Nacimiento Instante de una filtración en el que aparece por primera vez una clase de homología.

Muerte Instante de una filtración en el que una clase de homología deja de representar una clase independiente, por ejemplo porque se convierte en borde o porque se fusiona con una clase nacida anteriormente.

Persistencia Diferencia entre el valor de muerte y el valor de nacimiento de una clase homológica. Mide durante cuánto tiempo permanece viva una clase a lo largo de una filtración.

Código de barras Representación de la homología persistente mediante intervalos. Cada barra representa el intervalo de vida de una clase homológica.

Diagrama de persistencia Representación de la homología persistente como un multiconjunto de puntos (b, d) , donde b es el valor de nacimiento y d el valor de muerte de una clase homológica.

Distancia bottleneck Distancia entre diagramas de persistencia que mide el coste mínimo de emparejar sus puntos, permitiendo emparejar puntos cercanos a la diagonal con la propia diagonal. Se utiliza para formular resultados de estabilidad.

Paisaje de persistencia Representación funcional de un diagrama de persistencia. Transforma los puntos del diagrama en una familia de funciones, lo que facilita calcular normas, medias o utilizar la información persistente en algoritmos numéricos.

Mapper Algoritmo de TDA que construye un resumen en forma de grafo de una nube de datos. Utiliza una función filtro, un recubrimiento del espacio imagen y clustering local para representar la estructura global de los datos.

Función filtro o lens Aplicación utilizada en Mapper para proyectar o resumir la nube de datos antes de construir el recubrimiento. En este trabajo se emplea una reducción de dimensión basada en PCA y UMAP.

Recubrimiento Familia de subconjuntos que cubren un espacio. En Mapper, el recubrimiento se aplica sobre la imagen de la función filtro y controla el nivel de resolución del grafo obtenido.

Clustering local Agrupamiento aplicado dentro de cada elemento del recubrimiento en Mapper. Permite construir los nodos del grafo a partir de grupos locales de puntos.

Conceptos informáticos

Pipeline Secuencia ordenada de pasos que transforman unos datos de entrada en unos resultados finales. En este trabajo, el pipeline incluye preprocesamiento, construcción topológica, generación de pesos, backtesting y almacenamiento de resultados.

Librería Conjunto organizado de módulos y funciones reutilizables. En este trabajo, el código se organiza como una librería Python para facilitar su instalación, documentación y ampliación.

Módulo Fichero o bloque de código que agrupa funciones y clases relacionadas. Por ejemplo, la librería contiene módulos para preprocesamiento, TDA, backtesting y experimentos.

API Interfaz de programación que define cómo puede utilizarse una librería desde otro código. En este trabajo, la API permite importar clases y funciones como MapperParams o backtest_tda.

Script Fichero ejecutable que realiza una tarea concreta. En el proyecto se utilizan scripts para preparar datos y lanzar experimentos.

Entorno virtual Entorno aislado de Python donde se instalan las dependencias de un proyecto. Permite separar las librerías de este trabajo de las instaladas en el sistema.

Dependencia Librería externa que el proyecto necesita para funcionar. Ejemplos de dependencias en este trabajo son `numpy`, `pandas`, `scikit-learn`, `ripser` o `gudhi`.

pyproject.toml Fichero de configuración utilizado en proyectos Python modernos. En este trabajo contiene metadatos del paquete, dependencias y configuración de herramientas auxiliares.

Paquete Python Proyecto preparado para instalarse e importarse desde Python. En este trabajo, el paquete se distribuye con el nombre `tda-finance-mapper` y se importa como `tda_finance`.

PyPI Repositorio público de paquetes Python. Permite instalar la librería mediante `pip install tda-finance-mapper`.

Sphinx Herramienta para generar documentación. En este trabajo se utiliza para construir documentación HTML de usuario y de API.

PEP 8 Guía de estilo para código Python. Recoge convenciones sobre formato, nombres y organización del código.

flake8 Herramienta que comprueba el estilo del código Python y ayuda a detectar errores sencillos.

Pyright/Pylance Herramientas de análisis estático de tipos. Ayudan a detectar posibles inconsistencias antes de ejecutar el código.

Reproducibilidad Capacidad de repetir un experimento y obtener los mismos resultados o resultados equivalentes bajo la misma configuración.

requirements-lock.txt Fichero que recoge las versiones exactas de las dependencias utilizadas en un entorno concreto. Sirve como referencia para reconstruir el entorno de desarrollo o validación.

Conceptos financieros

Activo financiero Instrumento financiero que puede formar parte de una cartera. En este trabajo los activos son carteras industriales o acciones individuales.

Precio Valor de mercado de un activo en una fecha determinada. A partir de los precios se calculan retornos.

Retorno simple Variación relativa del precio de un activo entre dos fechas:

$$r_t = \frac{P_t}{P_{t-1}} - 1.$$

Retorno logarítmico Retorno definido como

$$\log \left(\frac{P_t}{P_{t-1}} \right).$$

Es habitual en análisis financiero por sus propiedades aditivas en el tiempo.

Cartera Combinación de varios activos financieros. Cada activo recibe un peso que indica qué proporción del capital se invierte en él.

Peso de cartera Proporción del capital asignada a un activo. Si w_i es el peso del activo i , en una cartera completamente invertida se cumple $\sum_i w_i = 1$.

Cartera long-only Cartera en la que todos los pesos son no negativos. Esto evita posiciones cortas y hace que la interpretación sea más sencilla.

Cartera equiponderada Cartera en la que todos los activos disponibles reciben el mismo peso. Si hay N activos, cada uno recibe peso $1/N$.

Benchmark Estrategia o referencia utilizada para comparar el rendimiento de un modelo. En este trabajo se utiliza como benchmark principal la cartera equiponderada.

Rebalanceo Proceso de actualizar los pesos de una cartera en fechas concretas. En este trabajo el rebalanceo se realiza cada tres meses.

Backtesting Simulación histórica de una estrategia de inversión. Permite evaluar cómo se habría comportado una regla de construcción de cartera usando datos pasados.

Sesgo de look-ahead Error que aparece cuando una estrategia utiliza información futura para tomar decisiones en una fecha pasada. En este trabajo se evita utilizando ventanas históricas causales.

Turnover Medida de cuánto cambian los pesos de una cartera entre dos fechas de rebalanceo. Un turnover alto suele implicar mayores costes de transacción.

Costes de transacción Costes asociados a comprar o vender activos. En este trabajo se modelan de forma proporcional al turnover.

Valor liquidativo (NAV) Valor acumulado de una cartera a lo largo del tiempo. Si R_t es el retorno de la cartera, se calcula mediante

$$NAV_t = NAV_{t-1}(1 + R_t).$$

Retorno anualizado Rentabilidad expresada en escala anual. Permite comparar estrategias evaluadas durante periodos de distinta longitud.

Volatilidad anualizada Medida de variabilidad de los retornos expresada en escala anual. Se utiliza como indicador de riesgo.

Ratio de Sharpe Medida de rentabilidad ajustada por riesgo. Relaciona el exceso de retorno de una cartera con su volatilidad.

Ratio de Sortino Variante del ratio de Sharpe que penaliza únicamente la volatilidad negativa o riesgo a la baja.

Drawdown Caída de una cartera desde un máximo previo de su valor liquidativo. Sirve para medir pérdidas acumuladas durante una fase negativa.

Máximo drawdown Mayor caída observada desde un máximo previo del valor liquidativo. Es una medida importante del riesgo sufrido por una estrategia.

Tipo libre de riesgo Rentabilidad asociada a un activo considerado sin riesgo en el periodo analizado. Se utiliza para calcular métricas como el ratio de Sharpe.

KF49 Abreviatura utilizada en este trabajo para referirse a las 49 carteras industriales de Kenneth French.

S&P 500 Índice bursátil compuesto por grandes empresas estadounidenses. En este trabajo se utiliza como universo amplio de acciones individuales a partir de datos CRSP/WRDS.

CRSP/WRDS Fuente de datos financieros utilizada para obtener información histórica de activos estadounidenses. Sus datos están sujetos a restricciones de acceso y licencia.

Bibliografía

- [1] U. Bauer. Ripser: efficient computation of vietoris–rips persistence barcodes. *Journal of Applied and Computational Topology*, 5:391–423, 2021.
- [2] F. Belchi, M. Pirashvili, J. Conway, M. Bennett, R. Djukanovic, and J. Brodzki. Lung topology characteristics in patients with chronic obstructive pulmonary disease. *Scientific Reports*, 8:5341, 2018.
- [3] R. Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, Princeton, NJ, 1961.
- [4] K. Beyer, J. Goldstein, R. Ramakrishnan, and U. Shaft. When is “nearest neighbor” meaningful? In *International Conference on Database Theory*, pages 217–235. Springer, 1999.
- [5] Boletín Oficial del Estado. Real decreto 126/2026, de 18 de febrero, por el que se fija el salario mínimo interprofesional para 2026. <https://www.boe.es/buscar/doc.php?id=BOE-A-2026-3815>, 2026. Último acceso: 6 de julio de 2026.
- [6] P. Bubenik. Statistical topological data analysis using persistence landscapes. *Journal of Machine Learning Research*, 16(3):77–102, 2015.
- [7] G. Carlsson. Topology and data. *Bull. Amer. Math. Soc.*, 46(2):255–308, 2009.
- [8] Center for Research in Security Prices. Crsp us stock databases. <https://www.crsp.org/>. University of Chicago Booth School of Business. Accedido el DD de mes de 2026.
- [9] Center for Research in Security Prices. Subscription information request. <https://www.crsp.org/subscription-information/>, 2026. Último acceso: 6 de julio de 2026.
- [10] D. Cohen-Steiner, H. Edelsbrunner, and J. Harer. Stability of persistence diagrams. *Discrete & Computational Geometry*, 37(1):103–120, 2007.
- [11] J. Corzo Galdó. tda-finance-mapper: Topological data analysis tools applied to financial portfolios. <https://pypi.org/project/tda-finance-mapper/>, 2026. Python Package Index. Último acceso: 16 de junio de 2026.
- [12] J. Corzo Galdó. tda-finance-mapper: librería para aplicar análisis topológico de datos a finanzas. <https://github.com/jaimecrz3/tda-finance-mapper.git>, 2026. Repositorio del Trabajo Fin de Grado.
- [13] V. DeMiguel, L. Garlappi, and R. Uppal. Optimal versus naive diversification: How inefficient is the $1/n$ portfolio strategy? *The Review of Financial Studies*, 22(5):1915–1953, 2009.
- [14] H. Edelsbrunner and J. Harer. Persistent homology—a survey. In J. E. Goodman, J. Pach, and R. Pollack, editors, *Surveys on Discrete and Computational Geometry: Twenty Years Later*, volume 453 of *Contemporary Mathematics*, pages 257–282. American Mathematical Society, Providence, Rhode Island, 2008.
- [15] H. Edelsbrunner and J. Harer. *Computational Topology: An Introduction*. American Mathematical Society, Providence, RI, 2010.
- [16] H. Edelsbrunner, D. Letscher, and A. Zomorodian. Topological persistence and simplification. *Discrete and Computational Geometry*, 28:511–533, 2002.
- [17] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, pages 226–231, 1996.

- [18] E. F. Fama and K. R. French. Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics*, 33(1):3–56, 1993.
- [19] K. R. French. Kenneth r. french data library. https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html. Accedido el DD de mes de 2026.
- [20] M. Gidea and Y. Katz. Topological data analysis of financial time series: Landscapes of crashes. *Physica A: Statistical Mechanics and its Applications*, 491:820–834, 2018.
- [21] C. Giusti, E. Pastalkova, C. Curto, and V. Itskov. Clique topology reveals intrinsic geometric structure in neural correlations. *Proceedings of the National Academy of Sciences*, 112(44):13455–13460, 2015.
- [22] A. Goel, D. Filipović, and P. Pasricha. Sparse portfolio selection via topological data analysis-based clustering, 2024. Version of 16 December 2024.
- [23] A. Goel, P. Pasricha, and A. Mehra. Topological data analysis in investment decisions. *Expert Systems with Applications*, 147:113222, 2020.
- [24] A. Goel, A. Sharma, and J. Kannianen. Class of topological portfolios: Are they better than classical portfolios?, 2026. Preprint. arXiv: 2601.03974 [q-fin.PM].
- [25] A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the 7th Python in Science Conference*, pages 11–15, 2008.
- [26] C. R. Harris, K. J. Millman, S. J. van der Walt, et al. Array programming with numpy. *Nature*, 585:357–362, 2020.
- [27] A. Hatcher. *Algebraic Topology*. Cambridge University Press, Cambridge, 2002.
- [28] T. Ichinomiya, I. Obayashi, and Y. Hiraoka. Protein-folding analysis using features obtained by persistent homology. *Scientific Reports*, 118:2926–2937, 2020.
- [29] D. Lo and B. Park. Modeling the spread of the zika virus using topological data analysis. *Scientific Reports*, 13:1–12, 2018.
- [30] P. Y. Lum, G. Singh, A. Carlsson, T. Ishkhanov, M. Vejdemo-Johansson, M. Alagappan, J. Carlsson, and G. Carlsson. Extracting insights from the shape of complex data using topology. *Scientific Reports*, 3(1236), 2013.
- [31] C. Maria, J.-D. Boissonnat, M. Glisse, and M. Yvinec. The gudhi library: Simplicial complexes and persistent homology. In *Mathematical Software – ICMS 2014*, pages 167–174. Springer, 2014.
- [32] H. Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.
- [33] V. Marx. The big challenges of big data. *Nature*, 498(7453):255–260, 2013.
- [34] L. McInnes, J. Healy, and J. Melville. Umap: Uniform manifold approximation and projection for dimension reduction, 2018.
- [35] W. McKinney. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.
- [36] Microsoft. Pyright: Static type checker for python. <https://microsoft.github.io/pyright/>. Último acceso: 16 de junio de 2026.
- [37] F. Murtagh. A survey of recent advances in hierarchical clustering algorithms. *The Computer Journal*, 26(4):354–359, 1983.
- [38] P. Niyogi, S. Smale, and S. Weinberger. Finding the homology of submanifolds with high confidence from random samples. *Discrete & Computational Geometry*, 39(1-3):419–441, 2008.
- [39] F. Pedregosa, G. Varoquaux, A. Gramfort, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [40] Python Code Quality Authority. Flake8 documentation. <https://flake8.pycqa.org/>. Último acceso: 16 de junio de 2026.

- [41] Python Packaging Authority. Python packaging user guide. <https://packaging.python.org/>. Último acceso: 16 de junio de 2026.
- [42] G. Singh, F. Mémoli, and G. E. Carlsson. Topological methods for the analysis of high dimensional data sets and 3d object recognition. In *SPBG*, pages 91–100, 2007.
- [43] S&P Dow Jones Indices. S&p 500. <https://www.spglobal.com/spdji/en/indices/equity/sp-500/>, 2026. Último acceso: 7 de julio de 2026.
- [44] Sphinx Developers. Sphinx documentation. <https://www.sphinx-doc.org/>. Último acceso: 16 de junio de 2026.
- [45] State Street Global Advisors. Spdr s&p 500 etf trust: Holdings and sector allocation. <https://www.ssga.com/us/en/intermediary/etfs/state-street-spdr-sp-500-etf-trust-spy>, 2026. Datos de posiciones y sectores consultados el 7 de julio de 2026.
- [46] G. Van Rossum, B. Warsaw, and N. Coghlan. Pep 8 – style guide for python code. <https://peps.python.org/pep-0008/>, 2001. Accedido el DD de mes de 2026.
- [47] H. J. van Veen, N. Saul, D. Eargle, and S. Mangul. Kepler mapper: A flexible python implementation of the mapper algorithm. *Journal of Open Source Software*, 4(42):1315, 2019.
- [48] Wharton Research Data Services. What is wrds? <https://wrds-www.wharton.upenn.edu/pages/about/what-wrds/>, 2026. Último acceso: 6 de julio de 2026.
- [49] A. Zomorodian and G. Carlsson. Computing persistent homology. *Discrete & Computational Geometry*, 33(2):249–274, 2005.